

密 级： 公开

加密论文编号：

论文题目： 面向云原生应用权限提升漏洞的动态防
控技术研究

学 号： M202310652

研 究 生： 韩晓阳

专 业 名 称： 计算机科学与技术

2026 年 5 月 1 日

面向云原生应用权限提升漏洞的动态防控技术研究

**Research on Dynamic Defense and Monitoring
Technologies for Privilege Escalation
Vulnerabilities in Cloud-Native Applications**

研究生：韩晓阳

指导教师：孙昌爱

北京科技大学 计算机与通信工程学院

北京 100083，中国

Doctor Degree Candidate: Xiaoyang Han

Supervisor: Chang'ai Sun

School of Computer and Communication Engineering

University of Science and Technology Beijing

30 Xueyuan Road, Haidian District

Beijing 100083, P.R.CHINA

中图分类号:

学校代码:

10008

U D C:

密 级:

公开

北京科技大学硕士学位论文

论文题目: 面向云原生应用权限提升漏洞的动态防控技术研究

研究生: 韩晓阳

指 导 教 师: 孙昌爱 单位: 北京科技大学 职称: 教授

指 导 小 组 成 员: 单位: 职称:

 单位: 职称:

论文提交日期: 2026 年 5 月 1 日

学位授予单位: 北 京 科 技 大 学

摘 要

Kubernetes 生态中的第三方开源应用普遍存在过度授权、访问控制缺陷、敏感信息泄露和网络隔离不足等安全漏洞，易被利用实施权限提升攻击。受开源社区漏洞治理能力差异影响，生产环境在补丁窗口期内会暴露于权限提升攻击风险之下。本文统计的云原生权限提升漏洞中 30.4% 在披露后存在攻击窗口。现有方法偏重通用静态核查、配置审计或单项检测，难以为具体漏洞快速生成有效的防控方案，且策略制定依赖专家经验，复用性差。

本文围绕补丁窗口期内云原生权限提升漏洞的临时防控需求，利用大语言模型的语义分析与代码生成能力，提出一套涵盖漏洞建模、策略生成与策略评价的自动化方法。主要贡献包括以下三方面。

(1) 漏洞分析与策略生成方法：首先建立漏洞分类框架，并整合代码仓库、漏洞公告、问题报告等多源证据构建融合上下文的安全知识库，为漏洞成因分析与利用路径研判提供结构化支撑。防控策略分为防护策略与监控策略两类，通过检索增强、思维链推理与反馈优化的协同流程实现防护策略智能化生成。进一步地，构建多模型防护策略评价机制，并关注有效性、无干扰性和可部署性三个不同维度。

(2) 原型系统实现：基于上述方法设计并实现原型系统 KPEAgent，集成漏洞信息采集、知识库构建、策略生成与策略评价各模块。

(3) 实验验证与案例分析：在 55 个漏洞样本上开展对比实验和消融实验。对比静态规则化方法，本方法结合具体漏洞场景生成更具针对性的防控策略；消融实验显示反馈优化对策略质量影响最大，去掉后有效性降 79.2%；大语言模型评审聚合排序较单一评审的人工评审一致性更高；案例分析表明，生成的策略可在准入控制、权限收敛、网络隔离和运行时监测等环节落地实施，封堵补丁窗口期内的攻击路径。

本文的研究可为云原生环境下权限提升漏洞补丁窗口期的临时防控提供方法参考与工程借鉴。

关键词：云原生安全；权限提升漏洞；策略代码生成；补丁窗口期防控；多智能体协同

Research on Dynamic Defense and Monitoring Technologies for Privilege Escalation Vulnerabilities in Cloud-Native Applications

ABSTRACT

Third-party open-source applications in the Kubernetes ecosystem commonly suffer from security issues such as excessive privileges, access-control flaws, sensitive information exposure, and insufficient network isolation, making them vulnerable to privilege escalation attacks. Due to differences in vulnerability governance capabilities across open-source communities, production environments are often exposed to privilege escalation risks during the patch-gap window. In the cloud-native privilege escalation vulnerabilities surveyed in this thesis, 30.4% still had an attack window after disclosure. Existing methods mainly rely on generic static checks, configuration auditing, or point-wise detection, and therefore struggle to rapidly generate effective defense-and-control strategies for specific vulnerabilities. In addition, strategy formulation heavily depends on expert experience, which limits reusability.

To address the need for interim defense and control of cloud-native privilege escalation vulnerabilities during the patch-gap window, this thesis leverages the semantic analysis and code generation capabilities of large language models to propose an automated method covering vulnerability modeling, strategy generation, and strategy evaluation. The main contributions are as follows.

(1) Vulnerability analysis and strategy generation method: This thesis first establishes a vulnerability classification framework and then integrates multi-source evidence from code repositories, vulnerability advisories, and issue reports to build a context-enriched security knowledge base. This provides structured support for vulnerability root-cause analysis and exploit-path reasoning. Defense-and-control strategies are divided into protection strategies and monitoring strategies. Through a collaborative pipeline of retrieval augmentation, chain-of-thought reasoning, and feedback refinement, the method realizes the intelligent generation of protection strategies. Furthermore, a multi-model evaluation mechanism for protection strategies is constructed, focusing on three dimensions: effectiveness, non-interference,

and deployability.

(2) Prototype system implementation: A prototype system called KPEAgent is designed and implemented on the basis of the above method. The system integrates modules for vulnerability information collection, knowledge base construction, strategy generation, and strategy evaluation.

(3) Experimental validation and case studies: Comparative and ablation experiments are conducted on 55 vulnerability samples. Compared with static rule-based methods, the proposed method generates more targeted defense-and-control strategies by incorporating specific vulnerability scenarios. Ablation experiments show that feedback refinement has the greatest impact on strategy quality: removing it reduces effectiveness by 79.2%. The complete method improves effectiveness by 32.5% and deployability by 41.2%. In addition, multi-model judge aggregation improves ranking stability by 23.7% compared with a single judge and shows good consistency with human evaluation. Case studies further demonstrate that the generated strategies can be applied in admission control, privilege reduction, network isolation, and runtime monitoring to block attack paths during the patch-gap window.

This research provides both methodological reference and engineering guidance for interim defense and control of privilege escalation vulnerabilities in cloud-native environments during the patch-gap window.

Key Words: Cloud-native security; Privilege escalation vulnerability; Strategy code generation; Patch-gap defense and control; Multi-agent collaboration

目 录

摘要	I
ABSTRACT	III
1 引言	1
1.1 背景与意义	1
1.2 研究内容与成果	2
1.3 论文组织结构	4
2 背景介绍	5
2.1 相关概念与技术	5
2.1.1 容器	5
2.1.2 容器集群系统 Kubernetes	6
2.1.3 Kubernetes 权限模型	7
2.1.4 Kubernetes 权限提升漏洞	7
2.2 云原生安全工具	8
2.2.1 监控工具：Falco	9
2.2.2 防护工具：准入控制工具	9
2.2.3 网络防护工具：NetworkPolicy	10
2.2.4 静态扫描与配置审计工具	10
2.3 国内外研究现状	11
2.3.1 系统权限安全问题	12
2.3.2 Kubernetes 安全问题	13
2.3.3 本课题组研究基础	14
2.4 小结	16
3 面向云原生权限提升漏洞的智能防控技术	17
3.1 形式化定义	17
3.1.1 基本对象	17
3.1.2 评价与选择	18
3.2 总体方法流程	18
3.3 开源应用漏洞安全知识库构建	19
3.3.1 数据采集	20
3.3.2 漏洞筛选	20
3.3.3 漏洞分类	21
3.3.4 知识库构建	24

3.4 基于多智能体协同的策略生成与优化	24
3.4.1 安全上下文构建	25
3.4.2 基于漏洞分析思维链的策略生成	26
3.4.3 基于评审反馈的策略优化	28
3.5 策略评估与质量控制	30
3.5.1 评价算子	30
3.5.2 多智能体评估体系	31
3.5.3 多源排名聚合模型	32
3.6 小结	34
4 面向云原生权限提升漏洞的智能化防控系统设计与实现	35
4.1 需求分析	35
4.1.1 用户角色与用例分析	35
4.1.2 功能需求	35
4.2 总体设计	36
4.2.1 系统架构设计	36
4.2.2 数据模型设计	37
4.2.3 数据流转与处置流程设计	38
4.2.4 交互时序设计	38
4.3 详细设计与实现	39
4.3.1 CVE 知识库管理	39
4.3.2 策略生成与优化	40
4.3.3 专家评审与排序	43
4.3.4 策略部署管理	46
4.4 小结	47
5 实验研究	49
5.1 研究问题	49
5.2 实验设置	49
5.3 RQ1: 消融实验与 workflow 模块贡献	50
5.3.1 workflow 跨模型稳健性验证	52
5.4 RQ2: 生成模型能力评估	52
5.4.1 三维质量指标的能力分化	53
5.4.2 输出稳定性评估	54
5.4.3 生成模型能力评估小结	55
5.5 RQ3: 评审方法可信度评估	55
5.5.1 不同大语言模型的评审一致性验证	56

5.5.2 跨维度评审差异分析	57
5.5.3 与人工评审的对比验证	58
5.5.4 小结	62
5.6 RQ4: 静态扫描的处置价值	62
5.7 RQ5: 资源消耗与效率评估	63
6 案例分析	67
6.1 案例分析实验设计	67
6.1.1 CVE 案例选择标准	67
6.1.2 实验环境与复现流程	67
6.2 案例一: CVE-2022-24768 内部权限问题	69
6.2.1 漏洞详解与复现	69
6.2.2 策略部署	71
6.2.3 三因素分析	72
6.3 案例二: CVE-2025-2241 (Hive ClusterProvision 敏感信息泄露)	72
6.3.1 漏洞详解与复现	72
6.3.2 策略部署	74
6.3.3 三因素分析	74
6.4 案例三: CVE-2023-30512 (CubeFS CSI 过度权限配置)	75
6.4.1 漏洞详解与复现	75
6.4.2 策略部署	77
6.4.3 三因素分析	78
6.5 案例四: CVE-2020-4062 Conjur OSS 缺乏网络隔离策略	78
6.5.1 漏洞详解与复现	78
6.5.2 策略部署	80
6.5.3 三因素分析	80
6.6 案例五: CVE-2022-24877 (Flux kustomize-controller 路径穿 越)	81
6.6.1 漏洞详解与复现	81
6.6.2 策略部署	82
6.6.3 三因素分析	82
6.7 案例六: CVE-2022-29171 (Sourcegraph gitserver 远程代码执 行)	82
6.7.1 漏洞详解与复现	82
6.7.2 策略部署	83
6.7.3 三因素分析	84

6.8 案例七：CVE-2023-22482（Argo CD OIDC aud 未校验导致越权访问）	84
6.8.1 漏洞详解与复现	84
6.8.2 策略部署	86
6.8.3 三因素分析	86
6.9 小节	86
7 结论与展望	87
7.1 总结	87
7.2 展望	88
附录 A 云原生权限提升漏洞分类与 CWE 对应关系	99
A.1 KubeGuard 权限提升检测规则汇总	99
附录 B CVE202224768 复现步骤与策略清单	101
B.1 漏洞复现详细步骤	101
B.2 策略代码清单	103
B.2.1 Kubernetes RBAC：收敛 AppProject 的写权限	103
B.2.2 Argo CD RBAC：默认只读 + 高风险能力隔离	104
B.2.3 准入控制：Gatekeeper（推荐）	105
B.2.4 准入控制：Kyverno（替代方案）	108
附录 C CVE-2025-2241 复现步骤与策略清单	111
C.1 漏洞复现详细步骤	111
C.2 策略代码清单	112
C.2.1 1) 立刻收敛 ClusterProvision 的读取权限（RBAC）	112
C.2.2 2) 对读取行为启用审计并联动告警	113
C.2.3 3) 缩短暴露窗口：供应完成后清理 ClusterProvision	114
C.2.4 4) vCenter 凭据轮换与强化（止血）	115
C.2.5 5) “敏感字段守护”检测（Gatekeeper/Kyverno，建议审计模式）	116
附录 D CVE-2023-30512 复现步骤与策略清单	121
D.1 漏洞复现详细步骤	121
D.2 策略代码清单	122
D.2.1 1) RBAC 最小化：移除 ClusterRole 中的 secrets 权限	122
D.2.2 2) ServiceAccount 硬化（按需评估）	123
D.2.3 3) 准入防回归（OPA Gatekeeper）：禁止创建包含 secrets 读取动词的 ClusterRole	123
D.2.4 4) 审计与检测：对 secrets 枚举行为进行可观测化	125
D.2.5 5) 网络收敛（可选）：限制 CSI Pod 出站访问面	125

附录 E CVE-2020-4062 复现步骤与策略清单	127
E.1 漏洞复现详细步骤	127
E.2 策略代码清单	128
E.2.1 1) 缓解措施: NetworkPolicy, 默认拒绝并仅放通 Conjur Server 访问 Postgres 5432	128
E.2.2 2) 命名空间隔离与 RBAC 收敛	129
附录 F CVE-2022-24877 复现步骤与策略清单	131
F.1 漏洞复现详细步骤	131
F.2 策略代码清单 (优先级方案)	132
F.2.1 1) 工作区缓解: 在 CI/CD 中校验 kustomization.yaml 路径 策略	132
F.2.2 2) 多租户变更治理与运行时硬化 (纵深防御)	133
附录 G CVE-2022-29171 复现步骤与策略清单	135
G.1 漏洞复现详细步骤	135
G.2 策略清单	136
G.2.1 1) 快速缓解: 禁用或移除 Gitolite code host (若业务允许) ..	136
G.2.2 2) 网络隔离: 限制 gitserver 的可达面 (默认拒绝并按需 放通)	136
G.2.3 3) 权限分离与暴露面收敛: 限制管理面与观测面的高权限 ...	138
附录 H CVE-2023-22482 复现步骤与策略清单	139
H.1 漏洞复现详细步骤	139
H.2 策略清单	140
H.2.1 1) 身份提供方治理: 独立客户端与唯一受众	140
H.2.2 2) 入口补偿控制: 前置强制校验 aud	141
H.2.3 3) 权限收敛: RBAC 最小化与项目边界约束	143
致谢	147
作者简历及在学研究成果	149
独创性说明	151
关于论文使用授权的说明	153
学位论文数据集	155

1 引言

本章节主要介绍课题的研究背景与意义、研究内容与成果，以及论文的组织结构。

1.1 背景与意义

容器技术借助内核级资源隔离与标准化镜像分发，改变了微服务应用的构建与交付方式 [1]。Kubernetes 作为主流容器编排平台，为复杂业务的部署与弹性扩展提供了基础支撑 [2, 3]。云原生计算基金会 CNCF 2025 年度调查报告 [4] 显示，生产企业 Kubernetes 采用率已达 98%，66% 的组织已在 Kubernetes 上运行 AI 工作负载。随着业务规模增长，Kubernetes 集群往往需要集成大量第三方开源组件，这些组件通过基于角色的访问控制（RBAC）机制 [5] 获取集群资源访问权限。然而，第三方应用在增强平台能力的同时也带来了安全隐患——多租户场景下，过度权限配置 [6]、权限管理缺陷、敏感信息泄露 [7]、网络隔离缺失等因素都可能引发权限提升风险。此类漏洞威胁性高、修复周期不确定，且成因与利用路径各异，给企业安全防控带来持续压力。

围绕云原生安全治理，学术界和工业界已积累了一定的工具链与方法体系。静态层面，针对镜像依赖、Kubernetes 清单及 IaC 模板的漏洞与误配置扫描被广泛用于风险前置发现 [8, 9]；运行时层面，基于系统调用或 eBPF 事件的观测审计能够辅助异常检测与攻击取证 [10]；防护层面，OPA/Gatekeeper [11]、Kyverno [12] 等准入控制器可拦截不合规资源进入集群，网络策略则通过最小连通约束抑制横向移动 [13, 14]。然而上述手段在面对权限提升漏洞时仍有如下不足：

- (1) **处置精准度不足**。多数方法依赖通用静态规则或单一方向检测，无法结合特定 CVE 的利用前置条件与攻击链路给出有针对性的处置方案 [8, 15]。
- (2) **防控策略难以复用**。已有研究大多围绕某一类权限提升问题展开 [6, 16, 17]，但不同漏洞的应用上下文、利用前提和影响范围差异较大，所得策略往往只适用于特定场景，跨场景迁移困难。
- (3) **漏洞安全响应能力薄弱**。策略编写高度依赖安全专家的个人经验，面对复杂异构的生产环境，难以形成可规模化、可快速响应的工程流程。

大语言模型的推理与代码生成能力为自动化生成防控策略提供了新的可

能，但将其应用于漏洞防控场景仍需克服以下研究挑战：

- (1) **补丁窗口期内的快速响应。**权限提升漏洞从披露到官方补丁发布之间往往存在较长的空白期。本文对 2020 至 2025 年间 55 个云原生权限提升漏洞样本的统计表明，30.4% 的漏洞在披露后长时间处于无补丁状态（如图 1-1 所示）。补丁的发布、验证与分发本身需要时间，而生产环境的组件依赖复杂、兼容性要求严格，进一步拖延了升级进度。因此补丁在生产环境中就绪前如何快速生成可用的临时防控措施，是需要优先解决的问题。

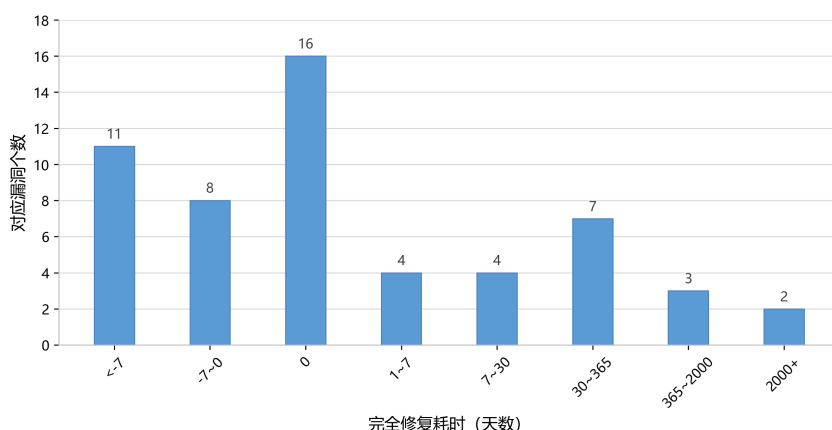


图 1-1: 云原生环境下权限提升漏洞的修复时间分布

Fig. 1-1: Distribution of remediation time for privilege escalation vulnerabilities in cloud-native environments

- (2) **多类型漏洞的领域知识融合。**权限提升漏洞涵盖 RBAC 滥用、容器逃逸、API 越权等多种攻击类型，各自的成因、利用前提和波及范围差别很大。要让大模型生成有针对性的策略，需要先将分散在代码仓库、漏洞公告、安全社区等渠道中的异构上下文信息加以收集与结构化组织，形成可被检索调用的领域知识体系。
- (3) **大模型生成的不确定性控制。**大语言模型在将漏洞语义转化为可执行策略代码时，存在产生幻觉、逻辑矛盾或偏离实际运行环境的风险。如果缺乏有效的验证与校正环节，生成的策略可能无法正确阻断攻击甚至干扰正常业务。

1.2 研究内容与成果

基于上述现实需求，本文聚焦于补丁窗口期内云原生权限提升漏洞的动态防控。按照控制手段的作用方式，可将相关措施概括为监控策略与防护策略两个互补层次。其中，防护策略侧重在漏洞利用发生前阻断关键条件并收

敛攻击面；监控策略则侧重运行时异常行为的持续观测与取证。

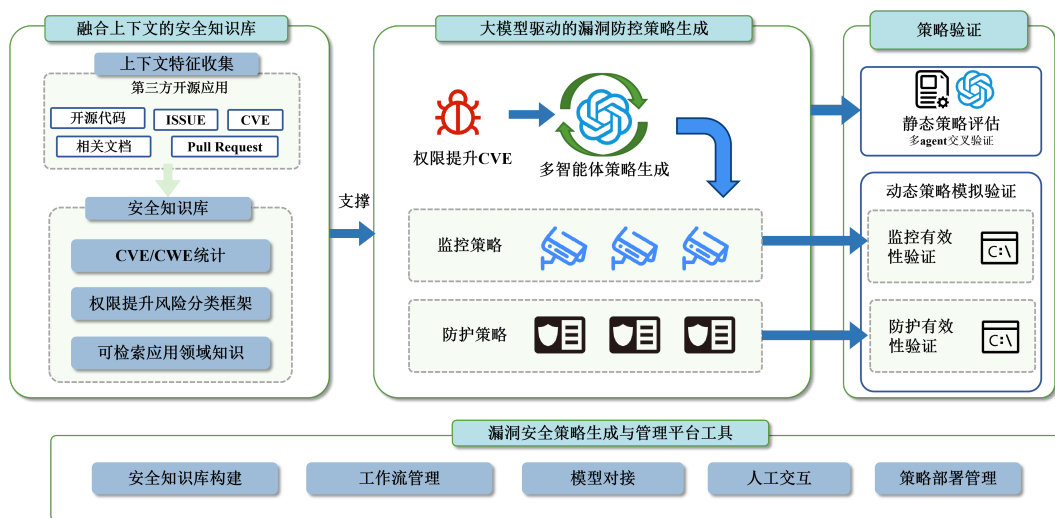


图 1-2: 基于多智能体的策略化漏洞防控研究总体框架

Fig. 1-2: Overall multi-agent framework for strategy-based vulnerability prevention and control based on large language models

如图 1-2所示，本文提出的智能防控框架以权限提升类 CVE 为输入，依托大语言模型（LLM）与多智能体协同，结合检索增强（RAG）、思维链（CoT）与反馈优化（Feedback）等技术，围绕安全知识支撑、策略生成、策略验证及平台工具建设，形成从漏洞披露到输出结构化 JSON 临时防控策略的自动化流程。具体研究内容如下：

（1）融合上下文的安全知识库。面向第三方开源应用场景，从代码仓库、议题（Issue）/合并请求（Pull Request）、CVE 公告等多源渠道汇聚漏洞证据，并构建权限提升漏洞的分类框架与领域知识库，为策略生成提供结构化证据链。

（2）大模型驱动的漏洞防控策略生成。依托多智能体协同机制，经由证据检索、威胁分析、策略草案生成、评审与迭代修订等阶段，将漏洞攻击链转化为可在 Kubernetes 平台快速落地的监控与防护策略。

（3）策略验证。对研究方法开展静态评估与动态验证：静态阶段提出多维度多主体交叉评审方法，对防控策略进行静态质量评估；动态阶段在实验环境中进行案例分析，基于漏洞复现与策略部署对策略效果进行动态评估。

（4）漏洞安全策略生成与管理平台工具。开发了涵盖知识库构建、工作流管理、模型对接、人工交互与策略部署的平台化工具，支持本研究方法实践落地。

1.3 论文组织结构

全文结构如下：第2章介绍相关概念、关键技术与国内外研究现状；第3章给出面向云原生权限提升漏洞的智能防控技术体系；第4章描述漏洞防控系统的设计与实现；第5章呈现实验设置与结果分析；第6章给出案例分析；第7章总结全文并展望后续工作。

2 背景介绍

本章节主要介绍相关概念、技术，以及与 Kubernetes 权限提升漏洞相关的国内外研究现状。

2.1 相关概念与技术

2.1.1 容器

服务应用的部署经历从物理机部署、到虚拟机部署、再到容器化部署的演进过程（见图 2-1）。虚拟机部署基于 KVM 等虚拟化技术，实现了完整操作系统模拟与资源隔离与复用，但存在虚拟化开销较大，启动时间较长，镜像体积庞大等问题，影响了应用的交付效率和资源利用率。与传统的物理机和虚拟机相比，容器通过操作系统内核级的资源隔离，实现了更高的资源利用率和更快的启动速度。容器以**镜像**为交付单元，将部署应用及其依赖环境以镜像形式打包。容器技术原理依赖三类内核技术：

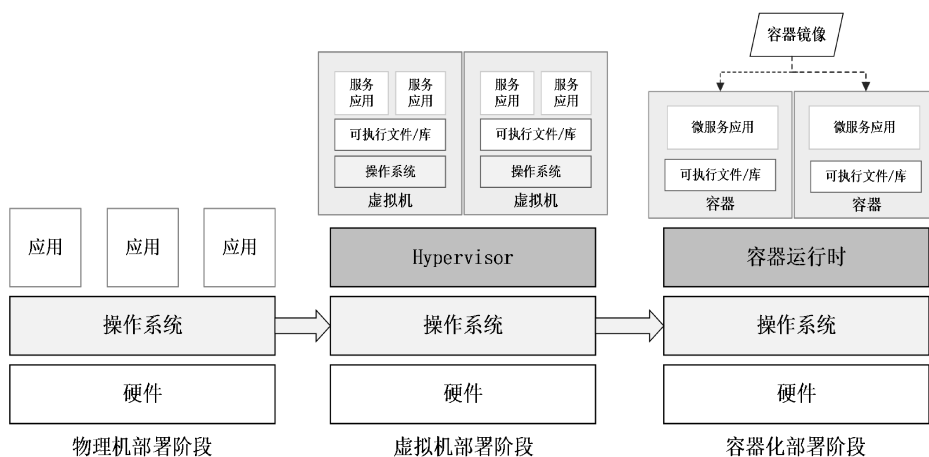


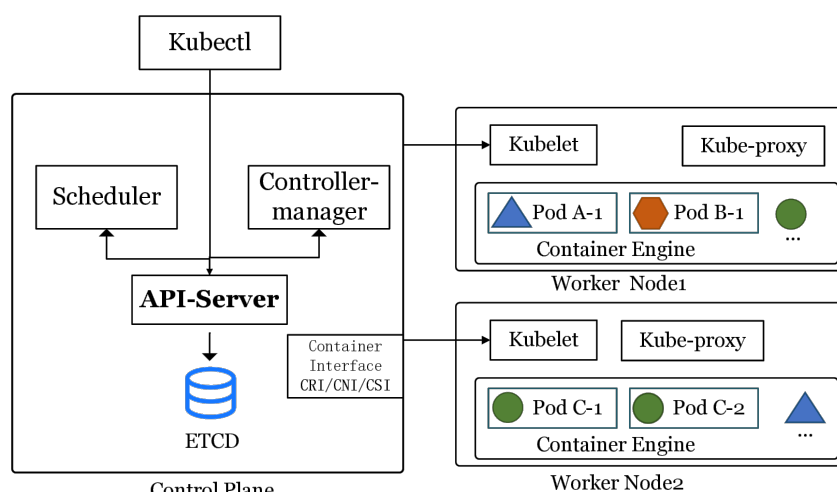
图 2-1: 服务部署形式演进

Fig. 2-1: Evolution of service deployment paradigms

- **命名空间（Namespace）隔离**：通过隔离进程、网络、挂载点等，使容器内运行的程序有独立系统抽象资源；
- **控制组（Cgroup）限制**：负责对容器施加 CPU、内存等硬件资源使用的强制规划，提供物理资源隔离；
- **只读分层与隔离挂载**：利用 OverlayFS 等联合文件系统 [18]，通过复用底层只读镜像并叠加独立的顶层读写区域，实现容器文件系统隔离。

2.1.2 容器集群系统 Kubernetes

Kubernetes 是面向容器群集负载的分布式编排系统，简称 K8S。源自于谷歌内部的虚拟化编排系统 Borg[19] 并在 2014 年正式对外开源。其核心思想是以**声明式 API** 描述期望状态，再由控制平面持续驱动集群内容器向期望状态收敛 [2]。图2-2为 Kubernetes 的体系结构示意图，一个 Kubernetes 集群



Architecture of the Kubernetes

图 2-2: Kubernetes 体系结构示意

Fig. 2-2: Overview of Kubernetes architecture

由**控制平面**与多个**工作节点**构成。控制平面负责提供控制接口、调度与集群控制逻辑，其中：**API Server** 是所有资源访问与状态变更的统一入口，基于键值数据库 **ETCD** 承担认证、授权、准入与资源对象持久化等关键职责；**调度器**根据资源请求、亲和/反亲和等约束为工作负载选择节点；**控制器**负责提供控制与调节不同资源。工作节点侧，**kubelet** 负责与 **API Server** 通信并维护调度到本节点的容器运行，**容器运行时**负责拉取镜像、创建容器并管理生命周期，**kube-proxy** 负责服务转发与网络规则的实现。

Kubernetes 通过一系列持久化**资源对象**表示集群对多种资源的编排与管理。例如，**Pod** 是最小调度单元，包含一个或多个共享网络与存储的容器；**Deployment/StatefulSet/DaemonSet** 分别面向无状态、有状态与节点守护型工作负载提供期望副本与更新策略；**Service** 与 **Endpoint/EndpointSlice** 提供稳定的服务发现与负载均衡抽象；**Ingress** 面向南北向流量提供入口控制；**ConfigMap/Secret** 用于配置与敏感信息的声明式注入；**Volume/PV/PVC** 则提供存储生命周期与绑定关系的抽象。

2.1.3 Kubernetes 权限模型

基于角色的访问控制（RBAC）是 Kubernetes 授权机制的核心 [2]。图 2-3 左侧展示了权限元模型的形式化表达，该权限模型类似“主谓宾”结构，即界定何种实体被允许对何种目标资源执行特定操作，其中：

- **服务账户（ServiceAccount）**即模型中的“主语”节点，代表发起请求的身份载体。在 Kubernetes 工作负载中每个 Pod 都有自己的服务账户。
- **权限角色（Role/ClusterRole）**即模型中的“谓-宾”约束。该组件声明了在特定作用域内，针对各类资源（resources）所允许执行的操作动词（verbs）集合。
- **角色绑定（RoleBinding/ClusterRoleBinding）**作为关联映射机制，将服务账户与权限角色绑定以完成授权。

图 2-3 右侧展示了一个典型的 RBAC 授权流程示例：首先创建一个名为 kada 的 ServiceAccount；随后定义名为 basic-operation 的 Role，在其 rules 字段中授予对 pods 与 secrets 的 get、list、create 权限；最后通过 RoleBinding 将该 Role 绑定至 kada 账户。这种配置在接入第三方应用时很常见，但如果防范不当配置了过大的权限，将引发权限提升漏洞的风险。

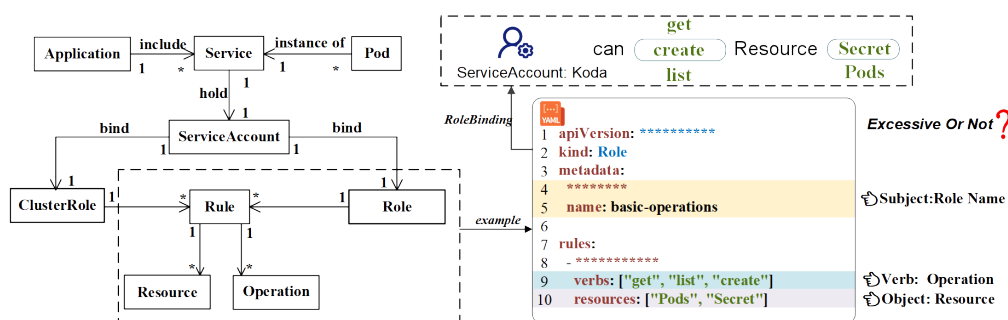


图 2-3: Kubernetes RBAC 权限配置 YAML 文件示意 [7]

Fig. 2-3: Example of Kubernetes RBAC permission configuration in YAML[7]

2.1.4 Kubernetes 权限提升漏洞

权限提升指攻击者在仅具备受限权限或初始执行能力的情况下，借助漏洞或错误配置，将自身权限提升到更高等级的过程 [20]。在 Kubernetes 场景下，权限提升既可能表现为授权范围的扩大或横向获取，也可能表现为节点侧系统权限提升，从而对更大范围的资源与工作负载产生影响。

在云原生环境中，权限提升的危害往往具有更强的放大效应。一方面，单

个节点通常承载多个 Pod，节点侧权限提升成功后，攻击者可能对同节点上的多个工作负载实施窃取敏感信息、篡改运行环境或注入恶意组件等行为。另一方面，Kubernetes 的资源对象与运维动作高度集中且自动化，控制平面授权不当同样会放大攻击后果。结合图 2-3 中的示例，若主体被授予针对 Secret 的 get/list 权限，攻击者即可读取服务账户令牌、数据库口令或 API 密钥等敏感信息；若被授予针对 Pod 的 create 权限，则可能进一步创建携带恶意镜像、特权配置或宿主机挂载的工作负载，作为横向移动、凭据窃取乃至后续节点侧利用的跳板。

图2-4 以 CVE-2024-33522 为例展示了典型的节点侧权限提升漏洞。该漏洞源于 Calico CNI 安装程序的 SUID 位配置不当。若攻击者借助弱口令或服务暴露等前置手段获取了节点的本地访问权限，便可注入恶意二进制文件，并诱导安装程序以高权限执行。越权成功后，攻击者能直接接管容器运行时数据、窃取各类高优凭据并干扰网络组件，其危害将突破单一工作负载的隔离边界，迅速蔓延至整个集群。

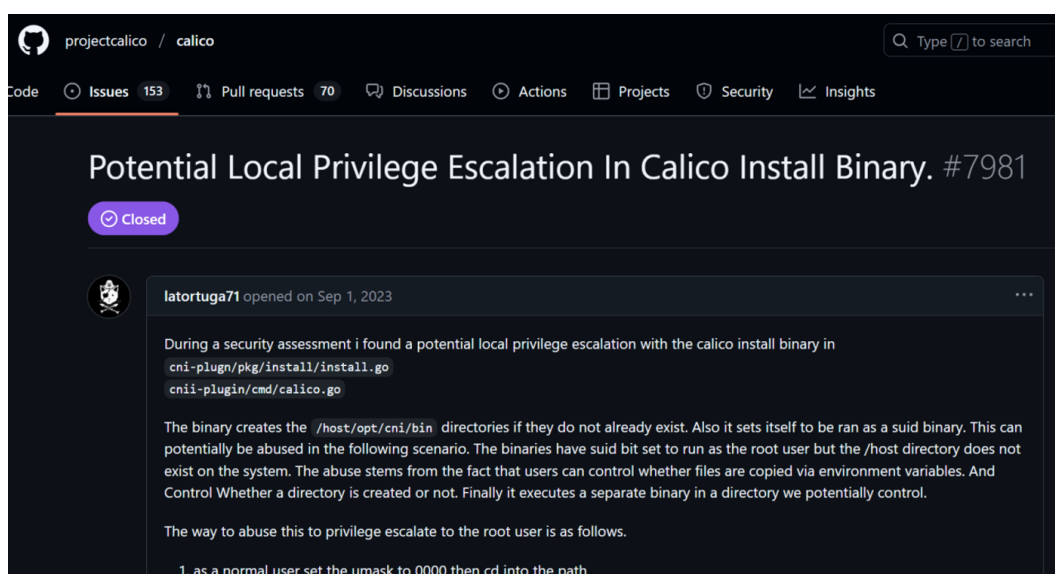


图 2-4: CVE-2024-33522（Calico）节点侧权限提升漏洞报告 ISSUE

Fig. 2-4: Issue report of node-side privilege escalation vulnerability CVE-2024-33522 in Calico

2.2 云原生安全工具

云原生安全工具为 Kubernetes 集群提供全生命周期的安全风险监管机制。相关安全工具可基于规则化 DSL 代码对容器镜像、静态配置、集群系统行为、容器运行时等维度进行安全检查。下文将介绍几种广泛使用的云原生

安全工具。

2.2.1 监控工具：Falco

Falco[10] 是云原生环境下广泛使用的开源容器运行时威胁检测引擎，其主要作用是为集群工作负载提供实时的异常行为监控与告警能力。在实现原理上，Falco 的机制包含内核态数据采集与规则引擎分析两个核心维度：它利用 eBPF 技术 [21] 在内核层以极低侵入性的方式动态捕获各类系统调用事件，并将其转换为富含云原生上下文的事件流，随后交由基于领域特定语言构建的声明式规则引擎进行模式匹配，从而在不阻断正常业务的前提下识别并捕获风险行为。Falco 同时也支持 K8s 业务监控，如 K8s 事件、K8s 资源对象、K8s API 调用等。

例如图2-5展示了监测“在容器内启动交互式 shell 进程”的规则。这种白盒化的检测方式具备极强的可解释性，能将告警直接关联至云原生实体，缩短安全定位路径。但该机制也存在一定的局限性：一方面，静态规则高度依赖专家知识的持续更新，对未知与变种攻击的泛化能力较弱；另一方面，其观测视野主要局限于系统调用层，难以有效感知应用层协议滥用或纯内存型的攻击。

```
1 - rule: Terminal Shell in Container
2   desc: Detects the spawning of a shell process inside a container.
3   condition: >
4     evt.type = execve and
5     container.id != host and
6     proc.name in (bash, sh, zsh)
7   output: "Notice: Shell spawned (user=%user.name container_id=%container.
8     id container_name=%container.name cmd=%proc.cmdline)"
9   priority: WARNING
10
```

图 2-5: Falco 运行时监控与规则示意

Fig. 2-5: Illustration of Falco runtime monitoring and rules

2.2.2 防护工具：准入控制工具

准入控制是 Kubernetes 中典型的事前防护机制。在 API Server 将资源持久化之前执行规则化校验，拦截特权容器、危险挂载、敏感权限增加等高风险配置变更。主流方案中，OPA/Gatekeeper 依托 Rego 语言，适合表达复杂组织策略；Kyverno 则采用 YAML 描述规则，更贴近运维习惯，落地成本更低。在补丁窗口期，这种规则化准入控制能够快速阻拦漏洞相关敏感资源操

作。如图 2-6 所示，该 Kyverno 策略可以拒绝未携带 owner 标签的 Pod 创建请求。

```
demo > ky.yaml
1  apiVersion: kyverno.io/v1
2  kind: ClusterPolicy
3  metadata:
4    name: audit-env-label
5  spec:
6    validationFailureAction: audit # <--- 关键点: 这里是 audit (审计), 而不是 enforce (强制)
7    background: true # 对集群里已经存在的 Pod 也进行扫描
8    rules:
9      - name: check-env-label
10     match:
11       resources:
12         kinds: [Pod]
13     validate:
14       message: "建议添加 env 标签以区分环境。"
15       pattern:
16         metadata:
17           labels:
18             env: "?*" # "?*" 意思是有这个 key 且 value 不为空
```

图 2-6: Kyverno 策略示例：强制要求 Pod 包含 owner 标签

Fig. 2-6: Example of a Kyverno policy enforcing owner labels on Pods

2.2.3 网络防护工具：NetworkPolicy

网络隔离用于降低横向移动与数据外传风险，是云原生环境中与身份权限控制互补的关键手段。Kubernetes 的 NetworkPolicy 为声明式防护策略，在多租户或微服务体系中可将控制域落到命名空间与工作负载级别，减少单点被攻破后对整个集群的扩散影响。常见的 Kubernetes 网络插件（如 Calico、Cilium、Weave Net 等）均支持 NetworkPolicy 的实现。

如图 2-7 所示，该策略定义了一个典型的微服务网络隔离场景：针对带有 role: db 标签的数据库工作负载，仅允许来自 role: frontend 前端服务的流量通过 TCP 5432 端口访问。这种基于标签选择器（Label Selector）的白名单机制有效地构建了微服务隔离边界，确保即使集群内其他容器试图横向移动访问数据库，也会在网络层被直接阻断。

2.2.4 静态扫描与配置审计工具

与运行时检测相对，静态扫描更偏向事前发现与持续治理。静态扫描在镜像构建、交付与部署前，通过自动化检查提前识别已知漏洞、弱配置与合规风险，避免高风险对象进入集群。

第一类是**镜像与依赖漏洞扫描**。该类工具通常解析镜像分层文件系统与软件包清单，基于公开漏洞库对已知 CVE 进行匹配，并输出风险等级、受影响版本与修复建议。代表性工具包括 Trivy[22]、Grype/Anchore[23] 等；其中，

```
YAML

apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: db-allow-frontend-only
spec:
  # 对应描述: “针对带有 role: db 标签的数据库工作负载”
  podSelector:
    matchLabels:
      role: db

  # 声明策略类型为入站控制
  policyTypes:
    - Ingress

  ingress:
    # 对应描述: “仅允许来自 role: frontend 前端服务的流量”
    - from:
        - podSelector:
            matchLabels:
              role: frontend

    # 对应描述: “通过 TCP 5432 端口访问”
    ports:
      - protocol: TCP
        port: 5432
```

图 2-7: NetworkPolicy 示例：限制数据库仅允许前端服务访问

Fig. 2-7: Example of a NetworkPolicy allowing only frontend services to access the database

Syft[24] 等工具可生成软件物料清单 (SBOM)，用于提升漏洞告警的可追溯性与供应链治理能力。

第二类是 **Kubernetes 清单与基础设施即代码 (IaC) 扫描**。这类工具面向 YAML、Helm、Terraform 等配置，以规则化方式识别特权容器、宿主机路径挂载、过宽权限、明文密钥及资源限制缺失等风险，可用于代码评审、部署前基线核查与准入前置门禁。典型工具包括面向工作负载清单的 kube-linter[25]、Kubescape[26]、kube-score[27]、KubeSec[28]、Polaris[29]，以及面向 IaC 模板的 Checkov[30]；针对权限审计的工具包括 KubiScan[31]、Krane[32]、RBAC Police[33]。此类方法易于规模化落地，但也可能因业务差异产生误报，因此需要进行人工二次审查。表2-1对 Kubernetes 常见安全扫描与审计工具进行分类整理。

2.3 国内外研究现状

本节从系统权限安全问题与 Kubernetes 安全问题两个层面梳理国内外相关研究进展，并介绍本课题组已有的研究工作，以阐明本研究的提出基础及其创新价值。

表 2-1: Kubernetes 常见安全扫描与审计工具整理

Tab. 2-1: Classification of common Kubernetes security scanning and auditing tools

类别	代表性工具	主要用途
镜像/依赖漏洞扫描	Trivy、Grype、Clair、Snyk	发现镜像与依赖漏洞，用于 CI 门禁、制品准入与修复建议。
SBOM 生成与供应链	Syft、CycloneDX、cosign	生成 SBOM 与签名校验，用于供应链追溯和完整性校验。
配置与 IaC 安全扫描	Kubescape、kube-score、kube-linter、Polaris、KubeSec、datree、Checkov、KICS、tfsec	扫描 YAML/Helm/IaC 弱配置与基线违规，用于评审和部署前核查。
RBAC/权限审计	KubiScan、Krane、RBAC Police、kubeadit、rakkess、kubectl-who-can	审计角色、绑定和账户权限，用于过宽授权识别与权限查询。

2.3.1 系统权限安全问题

围绕系统权限治理，既有研究已逐步形成涵盖设计原则、访问控制模型、风险检测以及后果追踪的系统性研究脉络。在基础设计原则层面，Saltzer 与 Schroeder[34] 于 1975 年指出，**最小权限原则**是实现特权隔离的重要基础。基于这一原则，Ferraiolo 等 [5] 提出基于角色的访问控制模型（RBAC），通过以角色组织多类权限并将其映射至不同主体，缓解了规模化授权中的管理难题。然而，**过度授权**现象普遍存在，并进一步扩大了系统攻击面 [35]。针对这一问题，Molloy 等 [36] 以及 Sanders 与 Yue[37] 分别提出基于日志与算法模型逆向生成应用最小权限集的策略，为数据驱动的权限管理研究奠定了基础。

随着系统架构组件化演进，基础权限机制的局限性进一步催生了复杂的跨域提权与攻击形态。在典型移动操作系统 Android 中，访问控制的粗粒度特性带来了更为严重的权限提升与隐蔽滥用风险。Davi 等人 [38] 较早揭示了**混淆代理**攻击模型，即低权限应用可通过伪造合法请求，诱导高权限组件执行敏感操作。Hutchinson 等 [39] 将其概括为三个条件：请求敏感权限、开放导出组件以及存在通向敏感 API 的执行路径。随着攻防对抗持续升级，提权漏洞的利用方式也由单一组件劫持 [40]，演变为多重调用链上的跨应用共谋 [41]，甚至扩展至同一设备内由第三方 SDK 发起的共谋攻击 [42]。

为系统性抑制提权风险，研究者围绕静态架构、动态监控及深层数据流设计了多维治理机制。在静态层面，Au 等 [43] 提出 PScout，通过分析系统源码自动提取底层权限检查图谱；后续研究进一步结合最小执行权限要求，引入强制访问控制（MAC）机制 [40, 44]。为应对不断演化的零日攻击，动态防御研究结合进程代数建模 [45]、深度学习以及系统级探针 [46, 47]，实现运行期拦截与预警。当授权边界被突破后，则可进一步借助数据流监控开展后果溯源，例如相关研究已从面向 Web 的污点分析 [48]，拓展至 Android 系统级动态隐私追踪方法，如 TaintDroid[49] 和 Taintman[50]。

针对安卓生态架构越权行为的分析研究，亦为宿主内嵌小程序生态的权限安全研究提供了参考。Zhang 等 [51] 提取了通用权限控制模型，并通过审计大规模 API 揭示了跨运行态环境下的 6 类典型越权漏洞；Wang 等 [52] 则聚焦数据权限滥用问题，定义了 5 类违规申请模式，并设计静态分析自动化引擎开展规模化检测，同时指出现有平台权限管理在底层设计上仍存在缺陷。

2.3.2 Kubernetes 安全问题

围绕 Kubernetes 安全问题，现有研究主要沿配置风险识别、模板与 IaC 安全分析、架构与权限链风险挖掘、网络与运行时防护四条主线展开。总体而言，研究视角已由清单合规检查逐步扩展至跨组件交互、授权路径以及底层隔离机制，表明 Kubernetes 安全风险具有显著的多层耦合特征。

首先，安全误配置是 Kubernetes 场景中最常见且最早受到关注的风险来源。Islam Shamim 等 [53] 系统梳理了 Kubernetes 安全实践，为理解配置风险及其治理重点提供了总体框架；Shamim[54] 进一步指出，部署清单偏离最佳实践可能直接演化为可利用的提权路径。Curtis 与 Eisty[55] 基于 Stack Overflow 数据开展的实证分析同样表明，容器基线与配置安全始终是开发侧持续关注的核心问题。在此基础上，Rahman 等 [9] 归纳了开源清单中的 11 类典型安全误配置并进行了大规模实证研究；然而静态工具往往存在漏报与误报。对此，Shamim 等 [8] 基于开源与企业清单评估了 CIS 推荐安全配置的合规现状，发现从业人员对安全基线存在显著的认知差异，且现有 SAST 工具在检测支持上存在明显局限；随后，他们进一步结合工业界部署开展经验研究，强调了应用 DAST 动态分析来弥补配置基线及运行态核验的客观必要性 [15]。静态分析通过大规模实证期望从配置项识别向真实风险发展，也说明运行时动态核验是弥合静态扫描局限的重要措施。

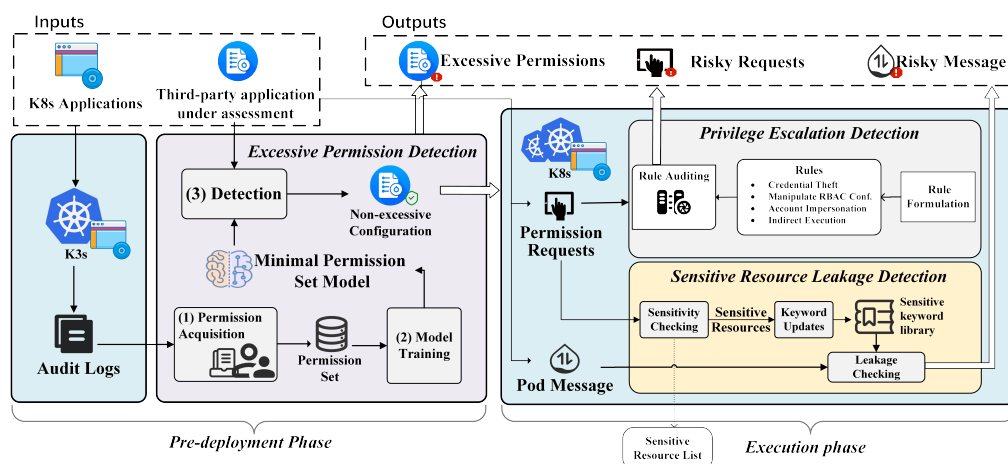
随着基础设施即代码 (IaC) 与模板化交付的普及, 研究对象进一步由单一清单扩展至模板依赖及跨平台配置链路。针对 Helm Charts 所带来的依赖传播与供应链风险, Zerouali 等 [56] 和 Blaise 与 Rebecchi[57] 分别从生态规模与依存关系角度提供了分析基础; Minna 等 [58] 进一步评估了大语言模型在修复 IaC 配置异味方面的效果与边界。为缓解多平台安全语义碎片化问题, Ul Haque 等 [59] 提出知识图谱框架 KGSecConfig, 对安全配置概念进行统一抽取与建模。这类研究表明, Kubernetes 安全治理正由局部规则检查转向面向模板、依赖关系与安全语义的自动化分析。

在此基础上, 相关研究进一步深入到体系结构缺陷与过度授权所引发的链式风险。Dell' Immagine 等 [60] 提出面向微服务场景的 Security Smells 模型, 并借助 KubeHound 对横向边界及潜在风险进行验证, 说明安全问题可能嵌入系统交互结构之中。Yang 等 [6] 通过分析 CNCF 集群插件指出, 控制平面对第三方组件授予过高权限, 可能成为工作负载接管的重要入口; Gu 等 [16] 提出 EPScan, 从目标代码端识别超出声明预期的高危特权滥用行为, 提升了对隐蔽越权与 0-day 组件风险的发现能力。与前述误配置研究相比, 这一方向更加关注风险的形成机制及其沿授权链路的放大过程。

除控制平面与授权逻辑之外, 底层网络与运行时隔离同样是 Kubernetes 安全防护的重要环节。Minna 等 [13] 指出, 单纯依赖身份隔离并不足以完全阻断连通性缺陷。Budigiri 等 [14] 论证了基于 eBPF 的低开销流量风险监控机制, Kim 等 [61] 则分析了不同 CNI 框架在协议约束深度与并发性能之间的权衡。在缓解机制方面, Pecka 等 [17] 梳理了 DevOps 链路中的高危利用路径, Cesarano 与 Natella[62] 提出 KubeFence, 以约束 Operator 与 API 通信中的非必要参数暴露; Zhu 与 Gehrmann[63] 以及 Le 等 [64] 则分别从 AppArmor 配置下发和系统调用感知调度角度强化节点侧防护。总体来看, 相关研究已形成从流水线控制到运行时隔离的多层缓解思路, 但对于权限提升漏洞在暴露窗口期的快速协同防控, 仍缺乏统一方法。

2.3.3 本课题组研究基础

在上述研究基础上, 课题组围绕 Kubernetes 权限风险检测开展了系统研究, 已形成从风险建模、原型实现到实验验证的较为系统的前期基础。其中, 代表性成果为 KubeGuard 框架。该框架面向过度权限、权限提升和敏感信息泄露三类风险, 分别设计了最小权限集构建、规则化审计以及敏感词监测机



在方法设计上，KubeGuard 一方面利用 NLP 技术识别应用所需的最小权限集，以判定权限配置是否存在过度授权；另一方面通过动态规则审计检测多类权限提升风险。具体而言，该工作将权限提升场景归纳为凭证窃取、账户冒用、RBAC 操纵和间接执行四类，并针对各类场景提出相应的检测思路。考虑到规则细节较为繁杂，相关风险描述与检测规则在正文中不再展开，具体内容见附录表A-2。此外，该框架还通过监测 Pod 间消息传输并结合关键词匹配，对涉及敏感资源的内容进行告警，从而覆盖敏感信息泄露风险。

综上，现有工作仍主要停留于权限层面的规则化检测，其核心依据在于 RBAC 配置与 API 访问行为之间的匹配关系。该方法虽然能够较为系统地覆盖多种权限提升模式，但在真实生产环境中仍存在三方面局限。第一，仅依据权限配置与操作行为的对应关系，难以准确区分正常运维行为与恶意提权行为，因此误报风险较高。第二，该方法主要面向控制平面与资源访问层，尚未深入节点侧系统层面的权限提升漏洞，对容器逃逸、SUID 配置不当及内核漏洞等底层利用缺乏针对性的检测能力。第三，基于预定义规则的检测方式对

未知漏洞与新型攻击手法的泛化能力有限。上述不足表明，围绕 Kubernetes 权限提升风险，仍有必要进一步研究面向真实利用链与底层运行态的检测与防护方法。

2.4 小结

通过本章的介绍，本章主要介绍了相关概念、技术以及国内外研究现状的介绍，为后续章节的研究方法设计与实验验证奠定了理论基础和研究背景。

3 面向云原生权限提升漏洞的智能防控技术

本章系统阐述面向云原生权限提升漏洞的智能防控方法，依次给出形式化定义与总体方法流程，并围绕知识库构建、多智能体策略生成与质量评估三项核心技术展开具体设计。

3.1 形式化定义

3.1.1 基本对象

为刻画面向漏洞的安全策略生成问题，本文将方法中的核心对象抽象为漏洞实例、安全上下文与安全策略三类实体。记候选策略全集为 Π 。关于策略质量评价所涉及的维度与记号，将在下一小节中进一步给出。

定义 1（漏洞实例）。 设 $\mathcal{V}$ 为漏洞实例空间。任一漏洞实例表示为

$$v = \langle \text{id}, d, m \rangle \in \mathcal{V}$$

其中， id 为漏洞标识符（如 CVE 编号）， d 为漏洞描述文本， m 为与漏洞相关的元数据，用于刻画其严重性、受影响组件及版本范围等属性。

定义 2（安全上下文）。 给定漏洞实例 v ，其安全上下文 $c \in \mathcal{C}$ 定义为带来源标识的有限证据集合：

$$c = \{ \langle t_i, \text{src}_i \rangle \mid i \in [n_c] \}$$

其中， t_i 表示证据内容， src_i 表示来源标识（如文档路径、公告链接或代码位置）。该定义强调上下文不仅提供与漏洞相关的背景事实，而且保留事实来源，以支撑后续策略生成与结果复核。

定义 3（安全策略）。 针对给定漏洞实例 v 的完整安全策略表示为三元组

$$\pi = \langle \text{type}, \text{method}, \text{action} \rangle$$

其中， $\text{type} \in \{\text{mon}, \text{prot}\}$ 表示策略属于监控型还是防护型； $\text{method} = (m_1, \dots, m_k)$ 表示控制机理序列， $\text{action} = (a_1, \dots, a_k)$ 表示与之对应的执行表达序列。二者满足一一对应关系，即每个 m_i 均对应一个可部署的 a_i ，从而共同构成完整的防控措施单元。

3.1.2 评价与选择

由于不同漏洞样本在业务环境、证据完备程度与候选策略规模上存在差异，直接采用绝对分值容易引入尺度漂移与比较偏置。与此同时，监控策略与防护策略在优化目标上并不完全同质，前者更强调观测、告警与取证能力，后者更强调阻断、收敛与约束效果。因此，本文在评价阶段首先按策略类型划分候选集合，再在各自子集中实施基于排序的相对评价。记类型集合为 $\mathcal{T} = \{\text{mon}, \text{prot}\}$ ，对任意 $\tau \in \mathcal{T}$ ，定义

$$\Pi_v^\tau = \Pi_v \cap \Pi_\tau$$

表示漏洞实例 v 在类型 τ 下的同质候选子集。下文若无特别说明， Π_v 均指代某一固定类型下的同质候选集合。记评价维度集合为 $\Delta = \{E, I, D\}$ ，分别对应有有效性、无干扰性与可部署性。

定义 4 (三维质量向量)。策略质量由映射 $q: \Pi \rightarrow [0, 1]^3$ 给出：

$$q(\pi) = (E(\pi), I(\pi), D(\pi))$$

其中， E 衡量策略对主要攻击路径与关键风险面的覆盖程度， I 衡量策略对正常业务的扰动程度及误报风险， D 衡量策略在现有环境中的落地难度与执行清晰度。

三维质量向量确定了各候选的评价基准；在每个维度上对候选集合排序、并归一化为可比得分的具体算子，将在 3.5.3 节聚合模型中统一引入。

3.2 总体方法流程

综合前述定义，本文方法可概括为由知识组织、候选生成与质量评估构成的三阶段映射链。给定漏洞实例 v ，首先围绕其所属应用、公开公告、修复提交与相关讨论构建可追溯知识库，并据此形成安全上下文 c ；随后在上下文约束下生成候选策略集合，并按策略类型划分为监控候选子集与防护候选子集；最后在同类型候选内部进行多主体排序评价与聚合，分别得到监控策略与防护策略的推荐排序。具体设计如下：

- **开源应用漏洞安全知识库构建：**围绕漏洞相关仓库、公告、补丁与讨论信息建立带来源标识的证据集合 c ，并构造可检索的向量索引，为后续生成提供事实支撑；
- **基于多智能体协同的策略生成与优化：**由安全知识智能体、漏洞分析智

能体、策略生成智能体、策略评审智能体与策略优化智能体分工协作，生成满足三元组结构 $\pi = \langle \text{type}, \text{method}, \text{action} \rangle$ 的候选策略，并通过评审反馈完成定向修正；

- **策略评估与质量控制：**以三维质量向量 $q(\pi)$ 为基础，对候选策略进行维度排序评价，并通过多源排名聚合获得可审计的共识结论。

上述流程的核心在于同时约束生成阶段的针对性与评价阶段的可复核性。前者通过证据约束与结构化中间结果抑制泛化输出，后者通过匿名化、多评审与排序聚合降低单一主体偏差，从而在有效性、稳定性与可部署性之间形成相对均衡的交付结果。

3.3 开源应用漏洞安全知识库构建

本节对应总体流程中的知识组织阶段，围绕漏洞实例 v 经四个依次递进的步骤构建可追溯知识库 $\mathcal{K}_v$ ，并通过检索形成安全上下文 c ，为后续策略生成提供带来源标识的证据支撑。整体上，该过程包括四个阶段：第一阶段从 CNCF Landscape 出发，经目标应用仓库映射与多源漏洞检索汇聚形成 CVE 候选池；第二阶段通过关键词、严重度与人工复核三重过滤，得到筛选后漏洞样本集；第三阶段完成 CWE 标注与两级成因分类，输出带标签漏洞样本集；第四阶段对多源材料进行规范化、差异化分块与向量化，形成可检索的向量化知识库。

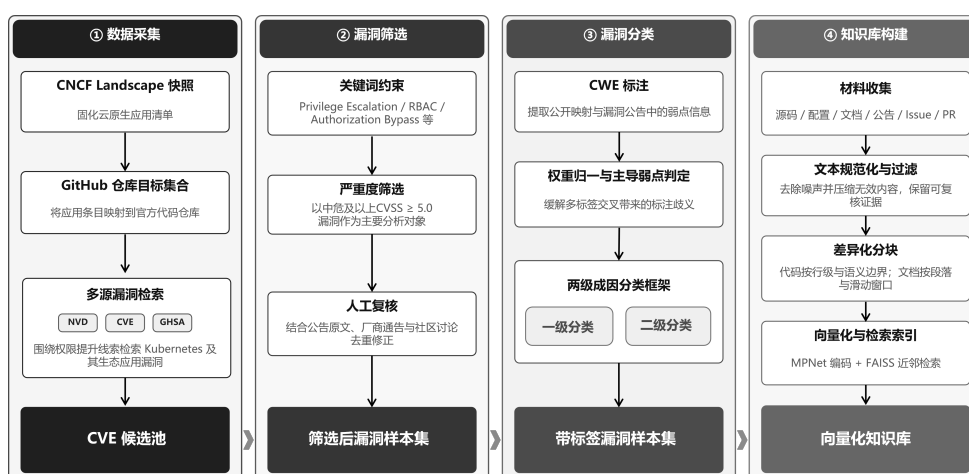


图 3-1: 云原生权限提升漏洞样本数据集概览

Fig. 3-1: Overview of the cloud-native privilege escalation vulnerability sample dataset

图 3-1 展示了本文收集并整理的漏洞样本数据集，包括来源分布、时间范围、CVSS 评分区间与漏洞类型构成，为后续的漏洞筛选、成因分类与知

识库构建提供样本基础。

3.3.1 数据采集

以 Kubernetes 及其生态开源应用为研究对象, 首先从 CNCF Landscape 获取云原生应用清单, 并对清单进行快照固化以保证采集过程可复现。随后将清单条目名称映射到其官方代码仓库, 形成后续漏洞检索与证据收集的目标集合。

在此基础上, 综合使用 NIST NVD¹、MITRE CVE 官方库²与安全公告平台 GitHub Security Advisory³三个来源, 围绕权限提升线索对 Kubernetes 及其生态应用进行多源漏洞检索, 获得初步 CVE 候选池。

3.3.2 漏洞筛选

针对 CVE 候选池, 通过三重过滤策略得到筛选后的高质量漏洞样本集:

- **关键词约束:** 在 CVE 描述与参考链接中匹配 Privilege Escalation、Authorization Bypass、Improper Access Control、RBAC、ServiceAccount、ClusterRole、Container Escape/Sandbox Escape/Breakout 等关键词;
- **严重性阈值:** 采用 CVSS v3.x 评分, 并以 ≥ 5.0 的中危及以上漏洞为主要对象;
- **人工复核:** 对候选条目结合公告原文、厂商通告与社区讨论进行复核, 剔除重复或与权限提升无关的记录, 并补全攻击前提、影响面与可用缓解信息。

最终保留的结构化字段包括 CVE 编号、漏洞摘要、披露时间、受影响组件与版本范围、修复版本、CVSS 评分、参考链接, 以及与修复相关的补丁提交记录等。上述字段与漏洞实例 $v = \langle id, d, m \rangle$ 直接对应: CVE 编号即标识符 id , 漏洞摘要构成描述 d , 其余属性 (严重性评分、受影响版本、参考链接等) 共同构成元数据 m 。

为保证数据构建过程可复现, 本文对同一批次样本采用统一快照方式完成采集, 并在构建配置中记录应用清单快照、检索时间窗口、纳入与剔除准则以及去重规则。对于来自不同数据源但指向同一漏洞事实的记录, 以 CVE 编号为主键进行合并; 对于无独立编号但经公告与补丁信息确认指向同一漏

¹NIST NVD (National Vulnerability Database): <https://nvd.nist.gov/>

²MITRE CVE: <https://cve.mitre.org/>

³GitHub Security Advisory: <https://github.com/advisories>

洞事件的记录，视为重复项而不重复计数。

3.3.3 漏洞分类

漏洞样本的分类方式直接影响后续策略生成的针对性。为此，本文构建了面向策略生成场景的两级成因分类框架，旨在为模型提供相对稳定且可解释的风险语义线索，从而约束推理路径、辅助关键控制点与实施路径的选择，提升输出策略的可部署性、可验证性与副作用可控性。

首先，本文依据公开映射与公告信息提取每个 CVE 的 CWE（Common Weakness Enumeration）标签，并统计不同 CWE 的出现频次。CWE 是 MITRE 维护的软件弱点枚举体系，可用于将具体 CVE 归并到更抽象的弱点类型。

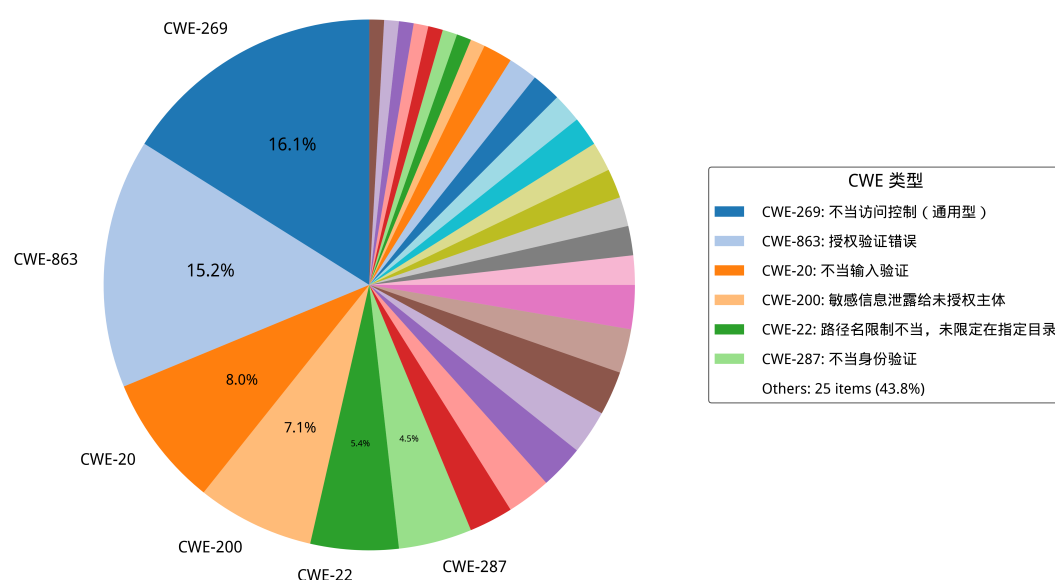


图 3-2: Kubernetes 权限提升 CVE 对应的 CWE 类型占比
Fig. 3-2: Distribution of CWE categories corresponding to Kubernetes privilege escalation CVEs

CWE 体系在组织形态上并非严格正交的分类结构：同一漏洞可能对应多个弱点条目，且不同条目在抽象层次上跨越具体机制与高层概念，使得类别之间存在交叉与重叠。在本文收集的 55 个权限提升漏洞中，36% 的 CVE（20 个样本）同时关联两个及以上 CWE 标签。为客观反映 CWE 分布，统计时对每个 CVE 赋予单位权重 1.0，多标签情况下各 CWE 均分该权重。统计结果显示，访问控制类弱点（如 CWE-863 授权不当、CWE-269 权限管理不当）与输入处理类弱点（如 CWE-20 输入验证不当）在多个样本中共现，反

映出漏洞成因的复合特征。上述非正交特性导致单一 CWE 标签难以为防控策略选择提供唯一且稳定的语义约束，因此需构建面向防控动作的成因分类框架。

为解决“根因标签不稳定/不互斥”对策略生成的影响，本文进一步提出面向云原生权限提升漏洞的两级成因分类框架，并将每个漏洞映射为（一级类，二级类）标签：

- **一级分类**刻画成因大类（如安全逻辑缺陷、配置缺陷、代码注入、敏感信息泄露等），用于提供高层风险语义与控制思路提示；
- **二级分类**进一步细化为更贴近防控动作选择的类型（例如授权验证不完善、默认不安全配置等），用于将漏洞分析结果对齐到可操作的控制点（RBAC 最小权限、准入控制、网络隔离、运行时策略与审计等）。

在策略生成阶段，分类结果作为提示词中的结构化约束，有助于模型将“漏洞语义 → 关键控制点 → 可执行动作序列”对齐，减少泛化建议与无关信息，提高策略的可部署性与针对性。完整的一级/二级分类与 CWE 对应关系见附录表 A-1。

为降低多标签条件下的标注歧义，本文对每个样本仅赋予一个主导的一级类与二级类标签。判定时遵循“直接导致权限提升的主导成因优先”原则：若某一弱点直接造成权限边界突破，则优先将其作为主标签；其余伴随弱点保留在 CWE 映射表中作为辅助语义信息。对于边界不清或存在多种可解释路径的样本，则结合漏洞公告、修复补丁语义与攻击路径描述进行复核后裁决。

以下对各一级分类分别展开说明。

配置缺陷指应用或基础设施的配置不当导致的安全漏洞，主要包括两种情形：

- **过度权限配置**：Kubernetes RBAC 配置不当、Role/ClusterRole 规则范围过宽或绑定关系不合理等，使低权限主体获得超出其职责的资源访问与高危操作能力。该类问题在第三方应用接入场景中尤为常见，并已被证明可被用于接管集群关键资源 [6, 35]。
- **网络隔离不足**：容器、Pod 或服务间缺乏网络隔离，导致攻击者可以横向移动获取其他权限。

安全逻辑缺陷指应用代码中的安全机制设计或实现不当，涵盖以下情形：

- **访问控制缺陷**：应用的权限检查不完全或存在逻辑漏洞，允许绕过权限

表 3-1: 云原生权限提升漏洞分类框架（CWE 对应详见附录表 A-1）
Tab. 3-1: Classification framework for cloud-native privilege escalation vulnerabilities
(see Appendix Table A-1 for the CWE mapping)

一级分类	二级分类	示例 CVE
配置缺陷	冗余权限	CVE-2023-30512
	网络隔离不足	CVE-2020-4062
安全逻辑缺陷	访问控制缺陷	CVE-2022-24768
	身份验证绕过	CVE-2023-22482
	输入验证不足	CVE-2023-3955
	授权验证不完善	CVE-2022-21701
敏感信息泄露	API 端口暴露	CVE-2022-1902
	日志泄露	CVE-2019-10165
	配置文件泄露	CVE-2019-3869
代码注入	命令注入	CVE-2021-41254
	路径遍历	CVE-2022-24877

验证；

- 身份验证绕过：认证机制设计缺陷，攻击者可以冒充他人身份或跳过认证步骤；
- 输入验证不足：对请求参数、资源标识或边界条件的校验不充分，导致攻击者借由异常输入扩大权限边界或触发越权路径；
- 授权验证不完善：权限检查在某些操作路径或边界条件下缺失。

敏感信息泄露指重要的身份、密钥或权限信息意外暴露，常见形式包括：

- API 端口暴露：管理接口、内部 API 或调试端口未受保护暴露在网络上；
- 日志泄露：日志文件包含敏感信息（如 token、密钥、命令历史）；
- 配置文件泄露：配置文件包含硬编码的凭证或密钥。

代码注入指应用对用户输入的处理不当，使攻击者得以注入恶意代码，典型子类包括：

- 命令注入：应用直接执行用户提供或可控的命令，攻击者可以执行任意系统命令；
- 路径遍历：应用文件访问检查不完整，攻击者可以访问系统其他部分的文件。

3.3.4 知识库构建

知识库构建阶段的目标是将与漏洞 v 相关的多源材料组织为带来源标注的证据集合，并建立支持按需取证的检索能力。围绕 v 构建知识库 $\mathcal{K}_v$ 后，在策略生成阶段通过语义检索从中提取相关证据片段，按需组装为安全上下文 c ，为后续分析提供可追溯的事实依据。

具体而言，对每个漏洞关联的开源应用仓库，收集代码、配置与文档等可读文本；同时纳入与漏洞相关的安全公告、议题（Issue）/合并请求（Pull Request）讨论与修复提交信息。对文本进行最小化规范，并过滤超长或二进制内容，以降低噪声与资源开销。

在完成材料收集后，为兼顾证据粒度与可复核性，本文对不同材料采用差异化分块：对代码/配置按行切分并尽量对齐函数或语义边界；对文档按段落与窗口切分并保留适当重叠，以减少跨块语义断裂。

在此基础上，为实现语义层面的证据召回，对每个文本块构造向量表示，并建立近邻检索索引。实验实现中，采用 sentence-transformers 框架中的 MPNet 句子编码器 all-mpnet-base-v2 完成文本表示 [65, 66]，并基于 FAISS[67] 构建相似度索引。为保证可追溯与可复现，索引与元数据进行同步持久化，内容包括向量索引文件、块级元数据，以及构建配置。块级元数据记录来源路径或链接、块 ID、块类型与范围等信息；构建配置记录分块参数与检索阈值等关键设置。

上述流程得到的向量知识库为后续策略生成提供了“可追溯证据”的供给能力：模型在推导控制措施时可以检索到与漏洞相关的关键事实与上下文片段，并通过来源标识完成快速定位与复核。

3.4 基于多智能体协同的策略生成与优化

策略生成阶段以漏洞实例 v 与安全上下文 c 为输入，生成候选策略集合 $\Pi_v \subseteq \Pi$ (图3-3)，其中每个候选 $\pi \in \Pi_v$ 均表示为三元组 $\pi = \langle \text{type}, \text{method}, \text{action} \rangle$ 。按策略类型划分， $\Pi_v^{\text{mon}} = \Pi_v \cap \Pi_{\text{mon}}$ 为监控策略候选子集， $\Pi_v^{\text{prot}} = \Pi_v \cap \Pi_{\text{prot}}$ 为防护策略候选子集。该流程综合使用检索增强（RAG）、思维链推理（CoT）与反馈优化（Feedback）三类机制，由五类智能体分工完成：安全知识智能体整理证据；漏洞分析智能体给出结构化分析；策略生成智能体产出候选（包括监控策略与防护策略）；策略评审智能体与策略优化智能体基于评审反馈

修订，以降低误报与运维噪声并提升可部署性。

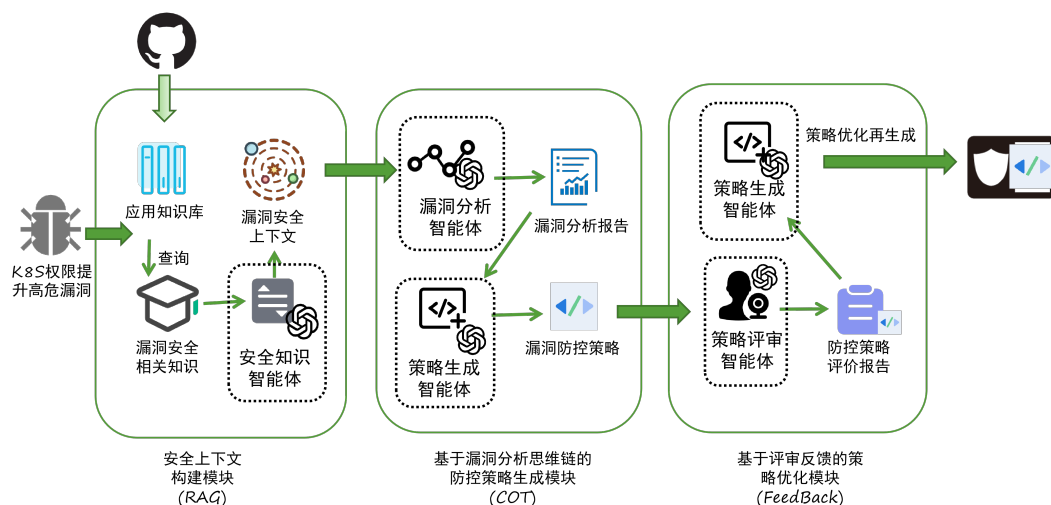


图 3-3: 多智能体协同的安全策略生成框架

Fig. 3-3: Multi-agent collaborative framework for security strategy generation

3.4.1 安全上下文构建

安全上下文构建的核心要求是保证证据的可追溯性。证据集合记为 $c = \{ \langle t_i, \text{src}_i \rangle \mid i \in [n_c] \}$ 。知识库阶段为每个片段记录来源元数据（如路径、URL、版本与位置）以便回溯；生成阶段再从知识库与安全材料中检索候选证据，并由安全知识智能体完成去重与提炼，输出触发条件、受影响组件、权限边界与可控点，同时保留对应的 src_i 供复核。

为保证不同实验轮次中证据整理过程具有一致的输入输出边界，本文将该智能体的指令约束抽象为“角色-输入-任务-输出”四元模式，其在全流程中的统一定义见表 3-2。其中，安全知识智能体的核心职责是将离散证据整理为带来源标识的结构化摘要，而非直接给出策略建议。

安全知识智能体提示词简化示例

你是云原生安全知识整理助手。

输入

漏洞实例 v 与候选证据集合 c

任务

提炼受影响组件、触发条件、权限边界、攻击前提与缓解线索，保留每条结论对应的来源标识。

输出

返回结构化证据摘要，不直接生成防控策略。

3.4.2 基于漏洞分析思维链的策略生成

本节将漏洞分析、技术选型与策略生成纳入统一的结构化约束，要求各环节输出可复核的中间结论。与仅给出自然语言建议的方式不同，中间分析结果与最终策略均映射到预定义结构模式，以减少信息缺失与表述歧义。

首先，**漏洞分析智能体**基于漏洞摘要与安全上下文 c 形成结构化分析结果，覆盖漏洞类型、攻击路径、影响范围与风险类别，并与前述成因分类体系保持一致。随后，**策略生成智能体**在结构化指令约束与分步推理引导下，以分析结论为条件完成控制手段选择与动作序列生成：结合平台可用能力，围绕身份与权限收敛、网络隔离、运行时约束与准入控制等手段确定实施路径，并将其映射为最终策略对象；当证据不足以支持唯一结论时，则显式保留待补充信息与必要假设。为兼顾方法主线的清晰性与实验过程的可复现性，本文保留简化后的提示词示例，并以表 3-2 概括各智能体的统一约束。

策略生成阶段需要在统一的数据模式下将分析结论映射为可部署的策略对象，因此除任务目标外，还需明确输出字段与约束边界。表 3-2 中漏洞分析与策略生成两类智能体的约束共同保证了这一映射过程的稳定性。

漏洞分析智能体提示词简化示例

你是 Kubernetes 漏洞分析助手。

输入

漏洞实例 v 与安全上下文 c

任务

给出漏洞类型、攻击路径、影响范围与成因分类，
并指出导致权限提升的关键环节。

输出

返回结构化分析结果；证据不足处显式标注 `assumptions`。

表 3-2: 多智能体指令约束摘要
Tab. 3-2: Summary of instruction constraints for the multi-agent workflow

智能体	输入	核心任务	输出约束
安全知识智能体	漏洞实例与候选证据片段	抽取受影响组件、触发条件、攻击前提、权限边界与缓解线索	输出带来源标识的结构化证据摘要
漏洞分析智能体	漏洞实例与安全上下文	形成漏洞类型、攻击路径、影响范围与风险类别的结构化分析	输出满足预定义模式的分析结果
策略生成智能体	中间分析结论、控制手段选择结果与安全上下文	生成监控或防护策略, 并保持字段完整、一致且可验证	输出包含 <code>type</code> 、 <code>method</code> 与 <code>action</code> 的结构化策略对象
策略评审智能体	漏洞实例、候选策略集合与补充约束	从有效性、干扰风险与验证需求等方面形成评审意见	输出包含总体结论与逐策略点评的结构化评审结果
策略优化智能体	漏洞实例、候选策略集合与评审结论	吸收有效控制、剔除高噪声建议, 并补齐前置条件与回滚边界	输出优化后策略及相对原候选的改进说明
评价智能体	匿名化与随机化后的同类型候选策略集合	在 E 、 I 、 D 三个维度上给出严格排序	输出满足预定义模式的维度排序结果及必要判据

策略生成智能体提示词简化示例

你是云原生安全策略生成助手。

输入

中间分析结论、可用控制手段与安全上下文 c

任务

围绕监控或防护目标生成策略 $\pi =$

$\rightarrow \langle \mathrm{type}, \mathrm{method}, \mathrm{action} \rangle$

$\rightarrow$,

优先给出可部署、可验证、可回滚的措施。

输出

返回结构化策略对象; 若信息不足, 显式说明前置假设。

为保证与前述形式化定义保持一致, 策略对象统一采用 `type`、`method` 与 `action` 三字段模式。其中, `type` 用于区分监控型与防护型策略, `method` 用于描述控制机理, `action` 用于承载与之对应的可执行表达 (如规则片段、配

置约束或执行命令)。当一项策略包含多条控制措施时, `method` 与 `action` 均表示为等长序列, 并通过位置对应关系保持语义一致。

为保证候选策略具有可比性与交付价值, 本文进一步要求其至少满足以下条件: 覆盖明确的防控目标, 包含可验证的实施步骤, 并对潜在副作用与回滚边界作出清晰说明。

候选策略集合 Π_v 通过多候选并行生成方式获得, 即围绕同一漏洞构造多个结构化策略对象, 以探索不同控制思路与实施路径。为避免异质目标混合比较, 生成结果在进入评价阶段前按策略类型划分为 Π_v^{mon} 与 Π_v^{prot} , 并分别进入后续评价与排序环节。

在进入交付与评估阶段之前, 系统还需对生成结果进行规范化整理, 包括统一字段格式、补齐必要步骤、清理冗余信息, 并在证据不足处显式标记假设条件与前置依赖。

3.4.3 基于评审反馈的策略优化

在候选策略生成之后, 系统进入评审、反馈与再生成的迭代优化环节, 以降低误报与运维噪声, 提升可部署性与一致性。系统并行生成多份候选形成 Π_v , 由**策略评审智能体**给出结构化评审(有效点、风险、验证要点与缺失信息), 再由**策略优化智能体**合并可行措施、剔除不可行建议, 并补齐前置条件与回滚方案。

为保证评审阶段具有一致的比较口径, 本文将评审与优化两个环节的指令约束统一纳入表 3-2 所示模式, 并保留简化后的提示词示例, 以展示其输入边界、判断依据与输出结构。

策略优化阶段的目标是将多份候选与评审反馈压缩为单一、可交付的最终策略, 因此其关键不在于模板表述本身, 而在于前序评审信息的吸收边界、修订依据与回滚约束是否得到显式保留。

策略评审智能体提示词简化示例

你是云原生安全策略评审助手。

输入

漏洞实例 v 、候选策略集合 Π_v 及补充约束

任务

从有效性、无干扰性与验证需求三个方面比较候选，指出优势、风险与缺失信息。

输出

返回结构化评审结果，包含总体结论与逐策略点评。

策略优化智能体提示词简化示例

你是云原生安全策略优化助手。

输入

漏洞实例 v 、候选策略集合 $\setminus P_{i_v}$ 与评审结论

任务

保留有效控制，删除高噪声或不可行措施，
补齐前置条件、验证步骤与回滚边界。

输出

返回优化后的最终策略及改进说明。

为提高结构化输出的一致性，本文进一步引入外部校验器，对策略中的规则片段与配置表达进行语法与模式一致性检查，从而将部分可部署性风险前移到生成阶段。系统实现中，该校验器可通过模型上下文协议（MCP, Model Context Protocol）等接口接入，但该实现细节并不改变方法本身的定义。校验重点包括语法正确性、最小格式约束以及与 `type`、`method`、`action` 三字段模式的一致性。

当校验未通过时，外部校验器返回结构化错误摘要，并将其作为反馈信号回注至生成过程，触发对相关片段的局部修正。该修正过程仅作用于出错片段及其必要上下文，以避免对已通过校验的内容产生无关扰动；若在预设迭代轮次内仍无法满足校验要求，则将该候选标记为待人工复核或待补充信息，并在后续质量评价中降低其优先级。

3.5 策略评估与质量控制

该阶段对应总体流程中的质量评价与聚合过程 $\mathcal{A}$ ，面向同类型候选策略子集 Π_v^r ，依据三维质量向量 $q(\pi)$ ，以及由秩函数 r_δ 、归一化得分 $s_\delta^{(h)}$ 与共识得分 $\hat{S}_\delta(\pi)$ 构成的排序与得分框架，输出各维度上的排序结果。评价整体遵循“结构化输入 $\rightarrow$ 规则化校验 $\rightarrow$ 证据支撑结论”的评价路径，以提高结论的可验证性与可复核性。由于监控策略与防护策略的优化目标并不完全同质，评价过程首先按类型对候选池进行划分，再在各自子集内部完成排序与聚合。相较于绝对分值评定，基于排序的评价方式更适合大语言模型参与的评审场景：绝对评分要求模型在跨样本条件下维持稳定且可解释的评分尺度，而已有研究表明，大语言模型在评分任务中往往依赖启发式判断，难以形成一致的内部尺度函数 [68]；同时，绝对评分更易受到文本长度、表达风格与候选顺序等非语义因素影响，从而产生系统性偏置 [69, 70]。相较之下，排序任务将评价问题约化为局部比较，更有利于降低尺度漂移与偏置累积，其结果也通常更接近人类评审的一致性判断 [71–73]。基于此，本文采用按维度排序并进一步聚合的评价框架，而不直接依赖绝对分值输出。

3.5.1 评价算子

为兼顾自动化效率与专家约束，本文将评价算子划分为两类：**大语言模型支持的评价算子**与**专家人工评价算子**。两类算子在评价维度与输出形式上保持一致，均以三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ 为依据，输出各维度上的相对排序；差异主要体现在评价主体及其使用证据的方式上：前者在结构化指令与证据上下文支持下形成排序结论，后者则结合组织约束、运维经验与现场条件进行复核与修正。

其中，大语言模型支持的评价算子以同类型候选策略子集 Π_v^r 为输入，在固定的评价准则与输出结构规范约束下，对候选策略在有效性 E 、无干扰性 I 与可部署性 D 三个维度进行比较并给出排序。为降低模型的提示偏置与先验印象影响，评价前对候选策略进行**随机化与匿名化**处理：

- **随机化**：对候选策略列表进行随机打乱，避免顺序效应；
- **匿名化**：移除或遮蔽与生成来源相关的元信息（如模型名称、模型配置标识、生成时间等），仅保留内容本身，并为每条候选分配不可语义化的匿名编号（如 $S_1, S_2, \dots$ ）。

为保持可追溯性，系统同时记录匿名编号与原始候选的映射表，供最终复核与实验统计使用。

与之对应，专家人工评价算子由安全专家基于同一评价维度与检查清单，对候选策略进行复核：重点验证关键前置条件是否成立、控制点是否覆盖主要攻击路径、策略副作用与回滚边界是否明确，以及配置片段是否符合组织内基线与现场约束。专家评价可以对大语言模型排序结果进行解释性标注与必要校正，并在存在高风险不确定性时给出需补充信息或需灰度验证的结论。

为使评价算子在不同实验批次中保持一致的判断口径，本文将评价代理的指令约束同样统一归入表 3-2，并在正文中保留简化后的评价提示词示例。评价环节的关键在于固定输入匿名化方式、评价维度与输出模式。

评价智能体提示词简化示例

你是云原生安全评价助手。

输入

匿名化、随机化后的同类型候选策略集合 $\{Pi_v^i \setminus \tau\}$

任务

分别在有效性 E 、无干扰性 I 与可部署性 D 三个维度上给出从优到劣的严格排序，并说明主要判据。

输出

返回结构化排名结果与必要备注。

3.5.2 多智能体评估体系

依据三维质量向量 $q(\pi)$ ，本文将候选策略 $\pi \in \Pi$ 的质量表示为 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ ，并在每个维度 $\delta \in \Delta$ 上要求对给定的同类型候选集合 Π_v^τ 输出严格排序，其中排序算子由秩函数 r_δ 表示。因此，多智能体评估体系的目标可表述为：对任一固定类型 $\tau \in \mathcal{T}$ 下的候选集合 Π_v^τ ，引入多个评价主体 $\mathcal{H} = \{h_1, \dots, h_m\}$ ，由每个 $h \in \mathcal{H}$ 独立实现维度排名算子 $R_\delta^{(h)}(\Pi_v^\tau)$ ，并输出三维严格排序 $\{R_E^{(h)}, R_I^{(h)}, R_D^{(h)}\}$ ，作为后续聚合与选择的基础信号。

为降低单一模型偏差与不稳定性对结论的影响，本文采用多评审的同口径可审计组织方式：由多个不同来源或不同配置的基于大语言模型（LLM）

的评价算子对同一输入进行**独立评审**，再将各评审结果聚合为总体排序。该设计遵循两条原则：其一，**多样性**，通过引入不同能力侧重的评审主体覆盖更多评价视角（例如对有效性证据链的敏感度、对业务干扰的谨慎程度、对可部署细节的严格性）；其二，**独立性**，各评价算子在同一评价准则与输出结构规范约束下独立输出，避免评审主体之间接触对方结论而引入一致性偏置。

具体实现上，系统对通过规则化校验的候选集合 Π_v^τ 执行匿名化与随机化处理后，将相同输入分发给各 $h \in \mathcal{H}$ 。在输出层面，每个 h 仅需在每个维度上给出一个全序排序（即由 $r_\delta^{(h)}$ 表示的严格排序），并可在必要时返回简短的判别依据与风险提示。该设计避免了直接要求不同评审主体共享同一套绝对评分尺度，从而更利于跨样本复现与统计分析。为增强鲁棒性与可复核性，多智能体评估体系引入如下控制机制：

- **一致性约束**：评价输出必须严格遵循既定的结构规范（仅输出三维严格排序与简短备注），并显式禁止使用候选来源信息推断质量（输入已匿名化）。
- **评审权重（等权）**：本文不区分不同评审主体（不同大语言模型或专家复核）的可靠性差异，默认采用等权设置：对 $m = |\mathcal{H}|$ 个评审主体，取 $w_h = \frac{1}{m}$ ，并满足 $w_h \geq 0$ 且 $\sum_{h \in \mathcal{H}} w_h = 1$ 。该设置更简洁、可复现，也避免引入额外的权重估计过程与主观性。
- **分歧触发复核**：当不同评审在某策略/某维度上分歧显著时，将该策略标记为高不确定性，优先进入专家复核或补充信息流程，避免仅凭单一评审结论做交付决策。

在上述体系下，评价算子的输出不再是不可解释的单点分数，而是可追踪的多源排序信号；下一小节将给出将多智能体三维排序聚合为总体质量评价与总排名的数学模型。

3.5.3 多源排名聚合模型

令 Π_v^τ 表示通过规则化校验的同类型候选集合， $|\Pi_v^\tau| = n_v$ ；令 $\mathcal{H} = \{h_1, \dots, h_m\}$ 表示参与评审的评价算子集合（可为多个评审大语言模型，必要时可包含专家复核）。每个 $h \in \mathcal{H}$ 在维度 $\delta \in \Delta$ 上输出一个严格排序：

$$R_\delta^{(h)}(\Pi_v^\tau) \rightarrow (\pi_{(1)}^{\delta,h}, \pi_{(2)}^{\delta,h}, \dots, \pi_{(n_v)}^{\delta,h})$$

其中 $\pi_{(1)}^{\delta, h}$ 表示在评审 h 的维度 δ 上最优候选。排序在匿名编号 (S1、S2、...) 上进行，最终通过映射表恢复到原策略用于审计与交付。

定义 5 (秩函数)。在维度 $\delta \in \Delta$ 下，给定候选集合 Π_v^τ ($|\Pi_v^\tau| = n_v$) 上的严格排序，秩函数 $r_\delta : \Pi_v^\tau \rightarrow \{1, \dots, n_v\}$ 将每个候选映射为其排序位次：

$$r_\delta(\pi) = 1 + |\{\pi' \in \Pi_v^\tau \mid \pi' \succ_\delta \pi\}|$$

其中 $\pi' \succ_\delta \pi$ 表示 π' 在维度 δ 上严格优于 π 。在严格排序下 r_δ 为——映射，最优候选获秩 1，最劣候选获秩 n_v 。

为便于计算，将单评审情形下的秩函数 r_δ 推广至多评审场景，对每个评审主体 h 定义 $r_\delta^{(h)} : \Pi_v^\tau \rightarrow \{1, \dots, n_v\}$ ：

$$r_\delta^{(h)}(\pi) = 1 + |\{\pi' \in \Pi_v^\tau \mid \pi' \succ_\delta^{(h)} \pi\}|$$

其中 $\pi' \succ_\delta^{(h)} \pi$ 表示在 h 的维度 δ 排序中 π' 严格优于 π 。在严格排序约束下，每个候选获得唯一秩，且 $r_\delta^{(h)}$ 为——映射。

上述映射不引入新的偏好假设：它仅将排序的“相对位置”编码为整数，从而使排序结果可以直接进入后续的加权 Borda 聚合与分歧度量计算。

定义 6 (归一化聚合得分)。设各评价算子等权 $w_h = \frac{1}{m}$ ($\sum_{h \in \mathcal{H}} w_h = 1$)。候选 π 在评审 h 、维度 δ 下的归一化秩得分为

$$s_\delta^{(h)}(\pi) = \frac{n_v - r_\delta^{(h)}(\pi)}{n_v - 1} \in [0, 1]$$

候选 π 在维度 δ 的共识得分为 $\hat{S}_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot s_\delta^{(h)}(\pi) \in [0, 1]$ 。

在获得各评审排序后，对 $m = |\mathcal{H}|$ 个评审主体的排名进行加权平均，取等权 $w_h = \frac{1}{m}$ ，满足 $w_h \geq 0$ 且 $\sum_{h \in \mathcal{H}} w_h = 1$ 。对每个 $h \in \mathcal{H}$ ，将其在维度 δ 上对候选 π 的排名转换为归一化得分：

$$s_\delta^{(h)}(\pi) = \frac{n_v - r_\delta^{(h)}(\pi)}{n_v - 1}$$

随后对所有评审主体的维度得分进行加权平均，得到维度 δ 的共识得分 $\hat{S}_\delta(\pi) \in [0, 1]$ ：

$$\hat{S}_\delta(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot s_\delta^{(h)}(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot \frac{n_v - r_\delta^{(h)}(\pi)}{n_v - 1}$$

基于共识得分可得到维度 δ 的共识排序 $\bar{R}_\delta$ ：按 $\hat{S}_\delta(\pi)$ 从大到小进行排序。为保证输出为严格排序，若出现数值相等，则按预先约定的确定性规则打破平分，例如依次比较各评审主体的原始排名并以匿名编号作为最终判别。

为刻画“评审一致性/不确定性”，可进一步定义分歧度量，例如秩的加

权方差：

$$U_{\delta}(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot (r_{\delta}^{(h)}(\pi) - \bar{r}_{\delta}(\pi))^2, \quad \bar{r}_{\delta}(\pi) = \sum_{h \in \mathcal{H}} w_h \cdot r_{\delta}^{(h)}(\pi)$$

当 $U_{\delta}(\pi)$ 较大时，表示该策略在该维度上存在明显分歧，需在后续决策中优先人工复核。

在获得三维共识质量后，进一步给出总体质量函数 $Q : \Pi_v^{\tau} \rightarrow [0, 1]$ ：

$$Q(\pi) = \alpha_E \hat{S}_E(\pi) + \alpha_I \hat{S}_I(\pi) + \alpha_D \hat{S}_D(\pi), \quad \alpha_E + \alpha_I + \alpha_D = 1$$

其中 $\alpha_E, \alpha_I, \alpha_D$ 是三个维度的权重，用于将“有效性/无干扰性/可部署性”的偏好映射到最终排序。该权重可按业务场景配置：例如在应急与临时缓解场景下可提高 α_E, α_D ；在生产稳定性优先场景下可提高 α_I 。

在本文的安全防控语境下，若无额外业务约束，默认偏好应体现“有效性优先”：即 $\alpha_E > \alpha_I > \alpha_D$ 。例如可取 $\alpha_E = 0.5, \alpha_I = 0.3, \alpha_D = 0.2$ 作为默认基线（具体数值可结合组织的风险偏好与运维能力进行微调）。

为避免“权重选择导致结论偶然变化”，实验中可对 α 做简单敏感性分析（例如在合理范围内扰动 α 并观察总排名是否稳定），将不稳定样本优先交由专家复核。对任一固定类型 τ ，最终总排名 $\bar{R}^{\tau}$ 由 $Q(\pi)$ 从大到小排序得到；为保证输出为严格排序，若出现 $Q(\pi)$ 相等，则按 $(\hat{S}_E, \hat{S}_I, \hat{S}_D)$ 的词典序进行细化，并在仍无法区分时以匿名编号作为最终判别。由此，可分别得到监控型与防护型候选的可审计交付结论。

3.6 小结

本章围绕云原生权限提升漏洞的防控需求，给出了从知识组织、候选生成到多评审聚合的完整方法链条。首先，对漏洞实例、安全上下文、策略对象与评价过程进行了统一形式化；其次，构建了面向开源应用漏洞的可追溯知识库，并基于多智能体协同完成策略生成、评审与优化；最后，通过三维评价与多源排名聚合模型实现了对候选策略的可审计选择。上述设计为下一章的实验验证与效果分析奠定了方法基础。

4 面向云原生权限提升漏洞的智能化防控系统设计与实现

基于前述方法论，本章给出 KPEAgent 原型系统的设计与实现。该系统面向云原生权限提升漏洞防控任务，将第三章提出的检索增强生成（Retrieval-Augmented Generation, RAG）、思维链推理（Chain-of-Thought, CoT）与人机协同评审机制集成为统一的交互式工作流，以支撑从漏洞知识检索、候选策略生成到专家评审与策略部署的完整处置过程。下文从需求分析、总体设计与详细设计三个层面展开论述。

4.1 需求分析

4.1.1 用户角色与用例分析

按照职责划分，系统主要面向两类协同用户：**安全工程师与专家评审员**（见图 4-1）。其中，安全工程师承担系统的主要操作任务，负责漏洞条目的检索、分析会话创建、候选策略生成以及部署执行等工作；专家评审员主要参与质量控制与决策确认环节，对候选策略进行多维度评审、排序与最终确认。专家反馈既服务于当前漏洞处置过程，也可沉淀为偏好数据，用于后续模型行为校准与知识库迭代。图中采用 UML 用例图的标准表示方法：椭圆表示系统用例，参与者以 «Actor» 构造型标注；«include» 关系表示“策略生成”必然包含检索增强生成（RAG）与思维链推理（CoT）两个子过程；«extend» 关系表示“人工修订与优化”和“策略部署”分别是对“策略生成”和“发布最终决策”的条件性扩展。

4.1.2 功能需求

结合上述角色职责，系统须覆盖以下功能：

- (1) **知识库管理**：导入、检索与维护 CVE 漏洞条目，为后续分析积累结构化语料。
- (2) **智能策略生成**：集成检索增强生成（RAG）与思维链推理（CoT），并结合反馈优化机制，输出面向特定漏洞的候选策略。
- (3) **反馈优化**：接收专家评审反馈并驱动模型迭代，逐步提升候选策略质量。
- (4) **专家评审与排序**：在可视化界面上对比多份候选策略，按有效性、可部署

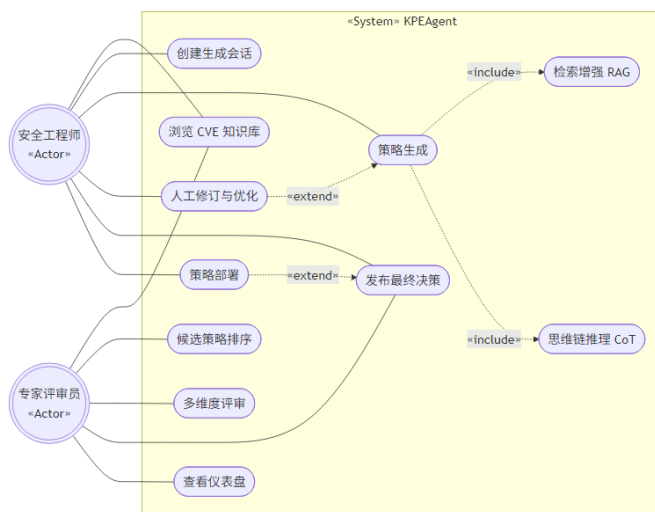


图 4-1: 系统用户角色与核心用例 (UML 用例图)

Fig. 4-1: System user roles and core use cases (UML use case diagram)

性、无干扰性等维度排序。

- (5) **策略部署管理**: 将通过评审的候选策略下发至目标 Kubernetes 集群或远程主机。

4.2 总体设计

4.2.1 系统架构设计

系统采用分层架构设计，自顶向下划分为四个逻辑层级（图 4-2）。各组件以 UML «component» 构造型标注，以突出不同层之间的职责边界与调用关系：

- (1) **界面层**: 基于 Vue3 构建单页应用 (SPA)，采用 Pinia 管理应用级状态，并集成 ECharts 可视化组件，负责 CVE 详情展示、策略交互与评审结果呈现等前端功能。
- (2) **服务层**: 采用 Flask 实现 REST API 网关，负责请求路由、访问控制与接口编排；同时通过任务调度器 (Scheduler) 管理耗时较长的检索与推理任务，并协调 CVE 管理、策略生成、评审控制与部署管理等业务模块。
- (3) **智能引擎层**: 封装系统的核心智能分析能力。该层由 KPEAgent 引擎统一调度检索增强生成模块、思维链推理模块与反馈优化器，实现从漏洞描述到候选策略的生成与迭代优化。
- (4) **数据层**: 负责数据持久化与检索支撑，包括存放 CVE 元数据及结构化策略结果的文件存储，以及支持语义检索的向量索引库。

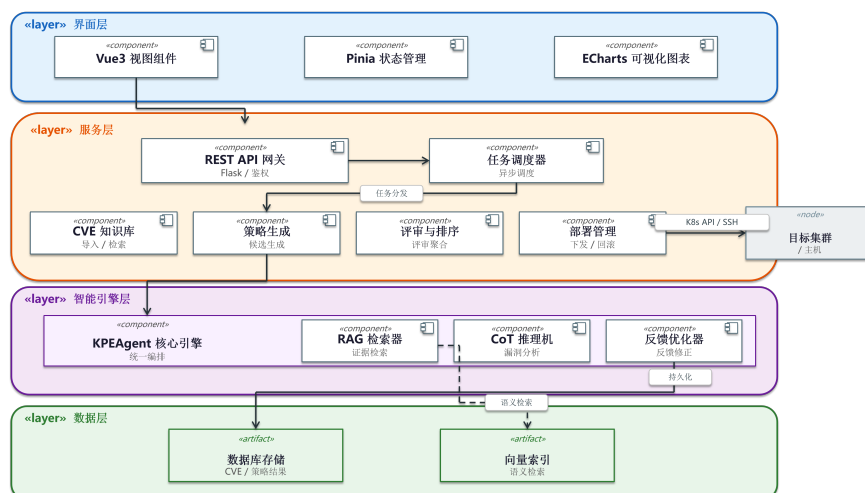


图 4-2: 系统总体架构与分层设计 (UML 组件图)

Fig. 4-2: Overall system architecture and layered design (UML component diagram)

4.2.2 数据模型设计

系统的数据持久化方案围绕以下四类核心实体构建（详见图 4-3 UML 类图）。图中按照 UML 标准标注了可见性修饰符（- 表示私有属性，+ 表示公有操作）、多重性约束及关联方向：

- (1) **CVE 实体**: 作为数据源头, 包含 `cve_id`、`summary`、`cvss_score` 及 `severity` 等描述漏洞严重程度的基础属性, 并以枚举类型 `status` 标记当前处置状态（未处理 / 分析中 / 已部署）。每个 CVE 与后续生成的会话呈“一对多”发散关系。
- (2) **会话实体 (Session)**: 表示一次完整的分析会话, 记录所选模型 `model_name`、提示词模板 `prompt_template` 及会话状态。会话与策略呈“一对多”关系, 支持同一会话下的多版本迭代。
- (3) **策略实体 (Strategy)**: 记录单次生成的完整策略结果, 包括策略内容 `content`、思维链快照 `reasoning_chain`、RAG 召回片段 `rag_snippets` 以及版本号 `version`, 从而支持对生成过程的复现、版本对比与增量追踪。
- (4) **排序实体 (Ranking)**: 存储专家的决策结果, 其核心属性包括专家评审员标识、决策时间戳、策略排序结果以及多维评分信息。该实体同时关联会话与策略, 用于刻画专家对特定候选策略集的即时评价。

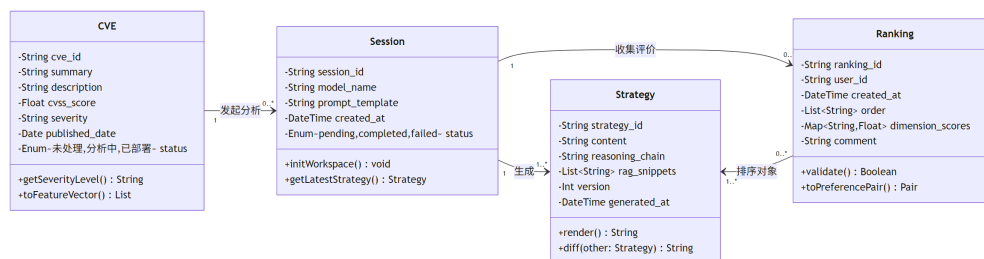


图 4-3: 系统核心实体类图 (UML 类图)

Fig. 4-3: Core entity class diagram (UML class diagram)

4.2.3 数据流转与处置流程设计

为保障关键决策过程的可追溯性，系统构建了如图 4-4 所示的闭环数据流模型。流程始于安全工程师导入 CVE 知识库 (Data Store D1)，漏洞条目经特征提取后进入策略生成管道；推理引擎 (Process P2) 结合检索结果生成候选策略，并写入策略库 (D2)；随后系统进入专家评审环节 (P3)，专家评审员的评分与排序结果进一步驱动策略优化与最终选型 (P4)；最终，排序记录、评审意见与决策依据统一归档至审计日志 (D3)，从而形成可复核的处置证据链。

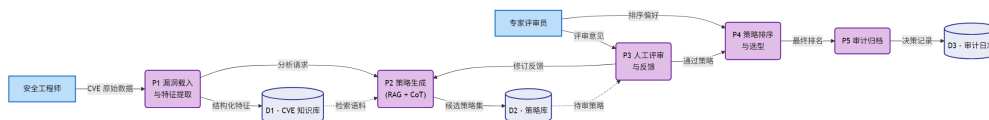


图 4-4: 防控闭环的数据流与审计点

Fig. 4-4: Data flow and audit points in the prevention-and-control loop

在数据流模型的基础上，系统进一步采用 UML 状态图 (图 4-5) 对 CVE 处置生命周期进行建模，以刻画漏洞从导入到部署完成的状态演化过程。该生命周期包含“未处理”、“分析中”、“待评审”、“已批准”、“部署中”与“已部署”六个顶层状态，其中“分析中”与“待评审”为复合状态，分别用于描述检索-推理-生成的迭代过程，以及多维评分与排序决策的评审过程。状态迁移条件，如“提交评审”、“驳回”与“部署失败重试”，均与系统业务规则一一对应，从而保证流程表达的完整性与可追溯性。

4.2.4 交互时序设计

策略生成过程通常涉及多轮检索与大模型推理操作，单次任务执行时间相对较长。为提升交互稳定性并降低前端阻塞风险，系统设计了异步交互时序 (见图 4-6 和图 4-7)。当安全工程师发起生成请求后，后端首先返回任务

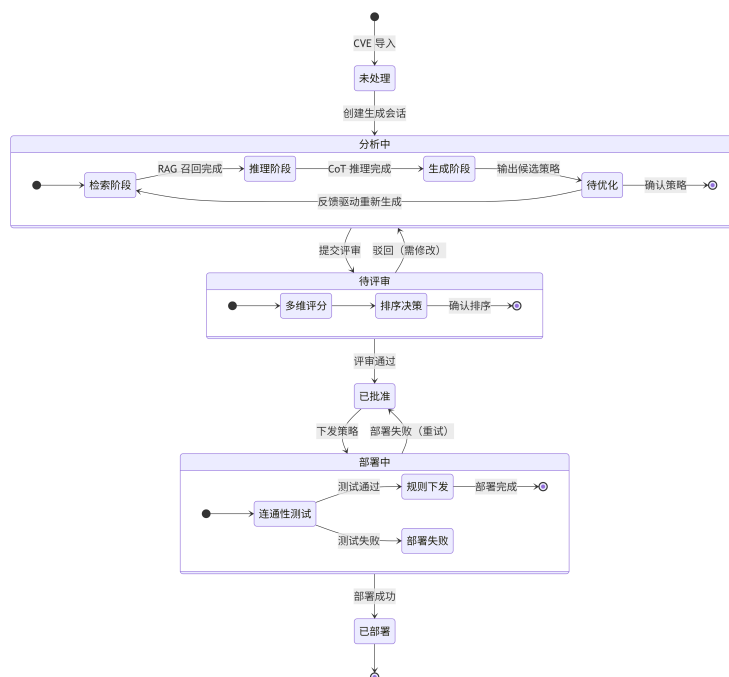


图 4-5: CVE 漏洞处置生命周期状态图 (UML 状态图)

Fig. 4-5: CVE vulnerability lifecycle state diagram (UML state diagram)

标识,并在后台依次执行检索、推理、反馈优化与结果提交等步骤;前端则通过轮询或 WebSocket 机制获取任务状态更新。UML 时序图将完整流程划分为四个交互片段:图 4-6 展示会话创建与策略生成两个阶段,其中进一步给出了 RAG 检索器访问向量索引、CoT 推理机执行推理链的消息交互;图 4-7 展示反馈优化与评审排序提交两个阶段,其中反馈优化以 opt 可选片段表示。该设计有助于增强长时任务执行过程的可观测性,并为人工干预预留接口。

4.3 详细设计与实现

本节按功能模块对系统的交互流程与实现方式进行说明。各模块的设计遵循前述总体架构中的层间职责划分与数据模型约束,并围绕 CVE 处置生命周期中的关键状态迁移展开。下文依次阐述知识库管理、策略生成与优化、专家评审与排序以及策略部署管理四个模块的详细设计与界面实现。

4.3.1 CVE 知识库管理

知识库管理模块对应服务层的 CVE 知识库组件,承担漏洞条目的汇聚、检索与状态维护功能,其操作结果直接影响 CVE 实体的 status 字段及后续会话的创建。如图 4-8 所示,系统主界面采用列表视图展示已录入的 CVE 条目,核心字段包括 CVE 编号、漏洞摘要、CVSS 风险评分以及当前处置状态。

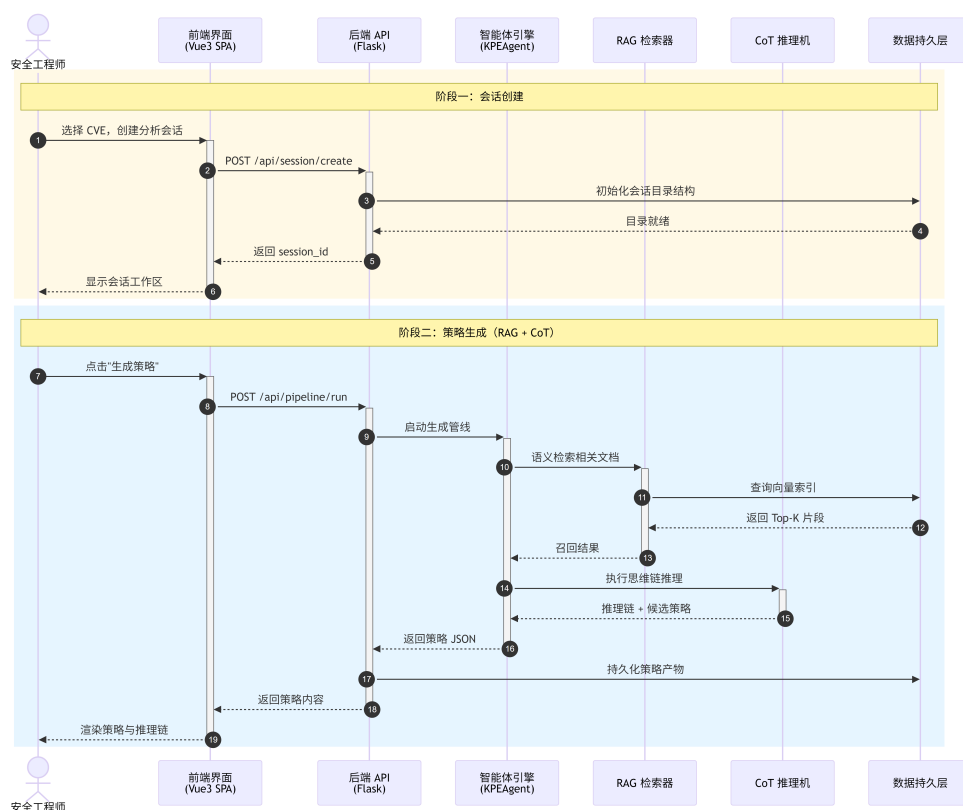


图 4-6: 会话创建与策略生成的交互时序 (UML 时序图)

Fig. 4-6: Interaction sequence of session creation and strategy generation (UML sequence diagram)

界面上方提供关键词检索与状态筛选功能，支持安全工程师依据漏洞编号、漏洞类型或关键描述快速定位目标记录；选择具体条目后，系统可进一步进入详情视图并发起相应的策略生成会话。

4.3.2 策略生成与优化

策略生成与优化模块对应智能引擎层的 KPEAgent 核心引擎及其子模块 (RAG 检索器、CoT 推理机与反馈优化器)，在服务层由策略生成组件与任务调度器协同驱动。该模块以会话 (Session) 为基本组织单元，维护单个漏洞分析任务的上下文与执行状态，从而保证不同漏洞之间的分析过程相互隔离。

如图 4-9 所示，会话初始化界面左侧给出历史会话列表，记录既有分析任务的漏洞对象与时间信息；右侧主工作区通过阶段条标识当前所处环节，即启动 (Start)、检索 (Retrieval)、推理 (Reasoning) 与生成 (Generation)。在初始化阶段，安全工程师可选择大语言模型后端及提示词模板，为后续推理过程设定运行配置。

检索阶段 (Retrieval) 负责构建策略生成所需的外部知识上下文。如图 4-10 所示，系统首先从 CVE 描述中抽取关键语义特征，并据此在本地知识库

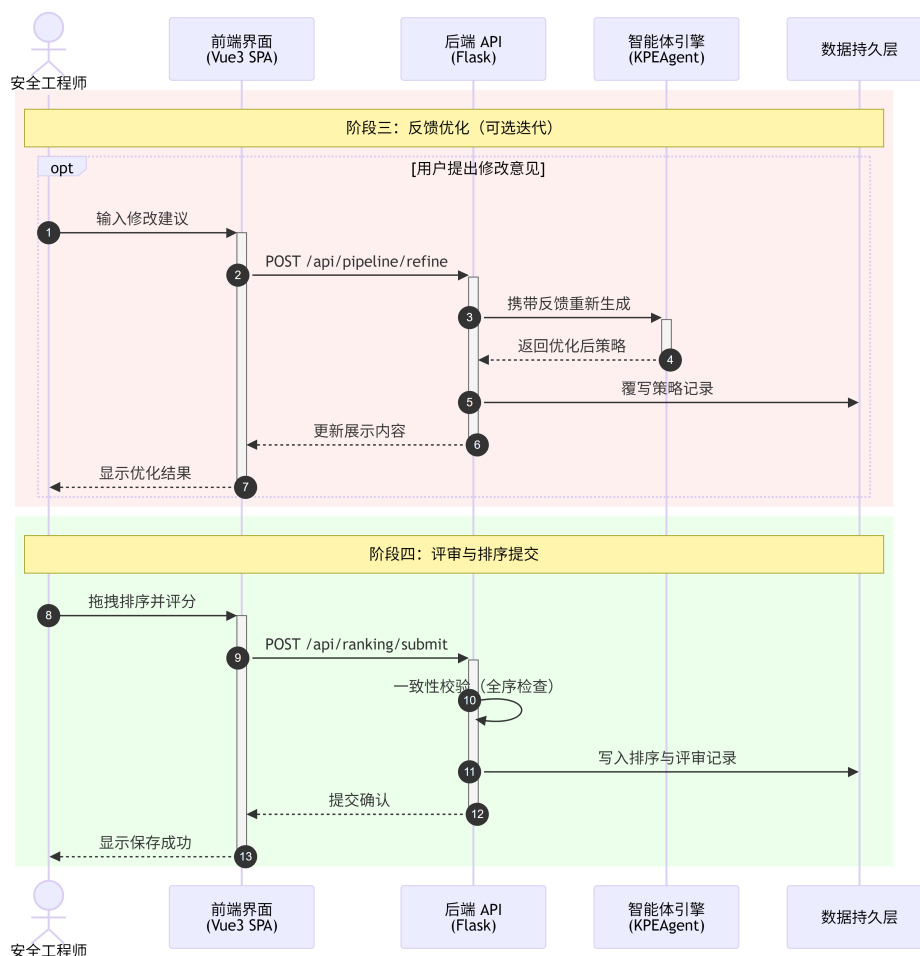


图 4-7: 反馈优化与评审排序的交互时序（UML 时序图）

Fig. 4-7: Interaction sequence of feedback optimization and review ranking (UML sequence diagram)

1

中召回语义相关的 Top-K 文档片段。界面以卡片方式呈现召回结果，同时显示来源文档与相关度得分，便于安全工程师对检索质量进行人工核验。经筛选后的有效片段将作为补充语境注入后续提示词，从而提高生成结果与漏洞场景的匹配程度。

思维链推理（CoT）阶段承担漏洞机理分析与防控逻辑展开任务。如图 4-11 所示，左侧提供结构化推理模板，右侧展示模型生成的中间推理步骤，例如攻击路径分解、权限边界识别以及 Kubernetes 权限模型映射等内容。进一步地，点击具体步骤可进入详情视图（图 4-12），系统以“背景识别”、“因果分析”和“结论推导”等字段组织推理结果。该设计有助于提高生成过程的可解释性，并便于人工核验关键推理链条。

在完成检索与推理后，系统进入候选策略生成阶段。如图 4-13 所示，生成结果预览界面将候选策略对应的防护规则（如 NetworkPolicy 或 OPA Rego 脚本）以结构化代码视图呈现，并采用语法高亮方式展示。界面同时提供复

Agent UI	Data Table	Raw Results	Multi Results	Strategy Ranking	策略仪表盘	多Agent生成工作流	策略部署	部署目标设置			
Raw Results											
	CVE Date	CVEID	CVSS v3.x Score	Published Date	Severity	Summary	Third party Applications	analysis	fix date, risk_level, 备注		
	2017-07-17T13:18:17Z	CVE-2017-100056	9.8	2017/3/21	CRITICAL	Kubernetes version 1.5.0-1.5.4 is vulnerable to a privilege escalation in the PodSecurityPolicy admission plugin resulting in the ability to make use of any existing PodSecurityPolicy object.	Kubernetes	1. “漏洞类型分类或成因”：多步漏洞链，具体源于PodSecurityPolicy (PSP) 准入插件的权限验证逻辑不完善导致。攻击者可绕过策略限制，将已授权的进程实例权限提升，即使自身未被授权关联该策略。2. “潜在危害场景分析”：攻击者通过修改宿主机用户（如普通用户或root用户）创建或修改文件时，利用宿主机访问权限绕过限制，从而获取宿主机root的PSP（如允许特权容器、访问宿主主机系统等），从而在集群内执行任意操作。例如，通过指定本进程的PSP进行特权容器，安装设备端高权限，获取宿主主机节点控制权，进而通过设备端或宿主机端提权。3. “修复策略建议”：“立即升级Kubernetes”：部署安全策略版本（如1.5.5或更高，需确认具体补丁版本）。“临时使用PSP”：若无可用版本，可通过API Server的“enable-admission-plugins”参数移除PodSecurityPolicy（可能降低安全，需评估风险）。“严格策略限制PSP”：确保所有节点遵循最小权限原则，避免冗杂或过度宽松的权限。限制“allowedUnsafeSysctl”、“privileged”等高危配置。“启用RBAC管理”：结合RBAC限制用户权限和API Server的权限验证，但需与主机系统权限验证协同。4. “攻击链分析”：“攻击”：CVE5.9对宿主机危害极大，且Kubernetes作为核心基础设施，一旦被利用可能导致集群完全沦陷，建立后门等。攻击者向多租户或容器API Server的环境。	2024.3.7	低危	K8S
	2018-05-17T02:29:02Z	CVE-2018-0268	10	2018 May 16	CRITICAL	A vulnerability in the container management subsystem of Cisco Digital Network Architecture (DNA) Center could allow an unauthenticated, remote attacker to bypass authentication and gain elevated privileges. This vulnerability is due to an insecure default configuration of the Kubernetes container management subsystem within DNA Center. An attacker who has the ability to access the Kubernetes service port could execute commands with elevated privileges within provisioned containers. A successful exploit could result in a complete compromise of affected containers. This vulnerability affects Cisco DNA Center Software Releases 1.1.3 and prior. Cisco Bug ID: CSCv47253.	Cisco DNA	1. “漏洞类型分类或成因”：“安全策略绕过”，具体源于Kubernetes容器管理子系统在Cisco DNA Center中配置了“不安全默认策略”，默认策略可能未充分考虑必要的认证和授权逻辑，导致攻击者无需身份验证即可访问关键资源。2. “潜在危害场景分析”：攻击者通过直接访问并修改Kubernetes服务端口（8081或8443），利用默认策略的认证失败，篡改与Kubernetes API交互，部署恶意容器或修改现有容器配置。通过修改容器执行任务命令（如向宿主系统其他容器，如数据库、部署持久化后门），最终可能导致整个DNA Center系统被控制，破坏网络管理系统的稳定性和可用性。3. “修复策略建议”：“立即升级软件”：更新Cisco官方发布的修复版本（1.1.3.2之后的版本）。“网络隔离”：限制Kubernetes服务端口的访问范围，仅允许信任的IP地址访问该端口。4. “最小化漏洞影响”：限制网络访问，限制角色和访问权限，TLS证书和证书，部署安全策略。5. “安全审计”：检查日志记录是否异常活动或未经授权修改，监控Kubernetes API日志。6. “最小权限原则”：确保容器运行所需的最小权限，避免使用root（privileged）模式。4. “攻击链分析”：通过CVE5.10分析漏洞具有最高严重性，攻击者可通过无认证即可完全控制受影响的系统，且Cisco DNA Center作为网络核心管理平台，一旦被攻破将导致灾难性安全后果（如配置篡改、数据泄露、服务中断），需立即修复。	2018年4月5日	低危	网络漏洞
	2018-05-18T13:26:00Z	CVE-2018-100400	8.8	2018 May 18, 2018, 2:29:00 PM -0400	HIGH	Kubernetes CRI-O version prior to 1.9 contains a Privilege Context Switching Error (CVE-2018-100400) vulnerability in the handling of ambient capabilities that can result in containers running with elevated privileges, allowing users abilities they should not have. This attack appears to be exploitable via container execution. This vulnerability appears to have been fixed in 1.9.	Kubernetes	1. “漏洞类型分类或成因”：“安全策略绕过”（具体为容器管理漏洞），源于容器运行时CRI-O在容器环境中ambient capabilities（环境能力）处理逻辑存在缺陷，导致容器进程继承了本应受限的权限能力（如CAP_SYS_ADMIN等），从而造成安全策略失效。2. “潜在危害场景分析”：“容器特权化”：攻击者无需通过root权限，利用本漏洞可绕过ambient capabilities，使容器进程获得与主机root级别的操作权限（如修改文件系统、修改内核参数）。“持久化后门”：通过容器逃逸提权，攻击者可以安装后门程序或木马，实现长期远程控制。攻击链分析：攻击者可能进一步通过容器逃逸（如利用Pod Security Policies或Pod Security Admission (PSA) 漏洞）通过Kubernetes Pod Security Policies或Pod Security Admission (PSA) 漏洞实现“ambient”配置，从而获取必要的min capabilities（如“drop”、“ALL”），从而“提升权限”，使用FalconTrace等工具监控容器内异常特权操作（如“CAP_SYS_ADMIN”的使用）。3. “最小化漏洞影响”：通过容器隔离和限制，攻击者可以限制容器逃逸和主机访问，避免造成严重安全后果。执行代码或脚本等。4. “策略生成与优化”：若攻击者控制Kubernetes节点上的网络资源，可能进一步通过容器逃逸（如利用API Server），导致集群被完全接管。5. “策略部署与验证”：通过Kubernetes Pod Security Policies或Pod Security Admission (PSA) 策略限制“ambient”配置，从而限制必要的min capabilities（如“drop”、“ALL”），从而“提升权限”，使用FalconTrace等工具监控容器内异常特权操作（如“CAP_SYS_ADMIN”的使用）。6. “最小化漏洞影响”：通过容器隔离和限制，攻击者可以限制容器逃逸和主机访问，避免造成严重安全后果。	6 May 18	低危	K8S

图 4-8: CVE 知识库查看与管理界面

Fig. 4-8: Interface for CVE knowledge base browsing and management

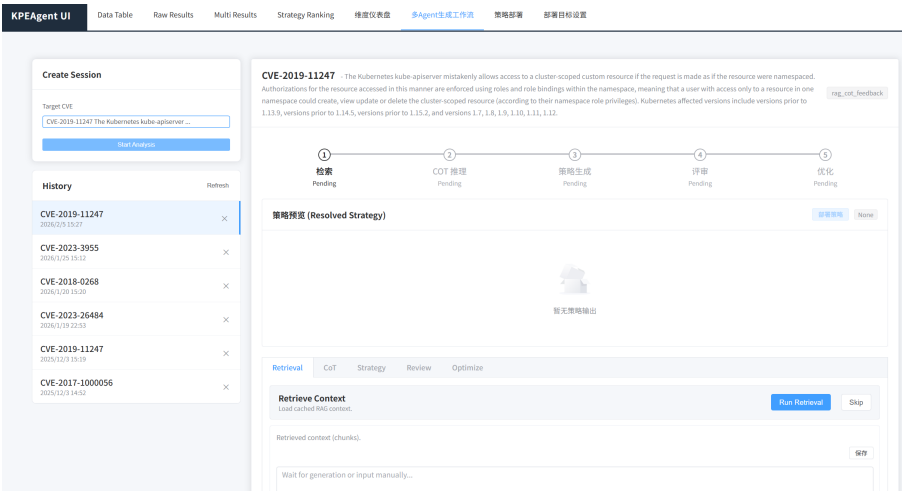


图 4-9: 策略生成会话启动界面

Fig. 4-9: Startup interface of a strategy generation session

制与导出等操作接口，并给出针对策略作用机理的简要说明，便于安全工程师在部署前对规则内容进行静态审查与合理性确认。

为适应复杂漏洞场景下的迭代修正需求，系统引入了“评审-优化”闭环机制。当专家评审员提出修改意见后，安全工程师可在优化面板发起再生成请求。如图 4-14 所示，界面以并列方式展示原始版本与修订版本，并对变更位置进行高亮标识。通过该机制，系统能够在保留既有生成轨迹的基础上逐步提升候选策略质量。

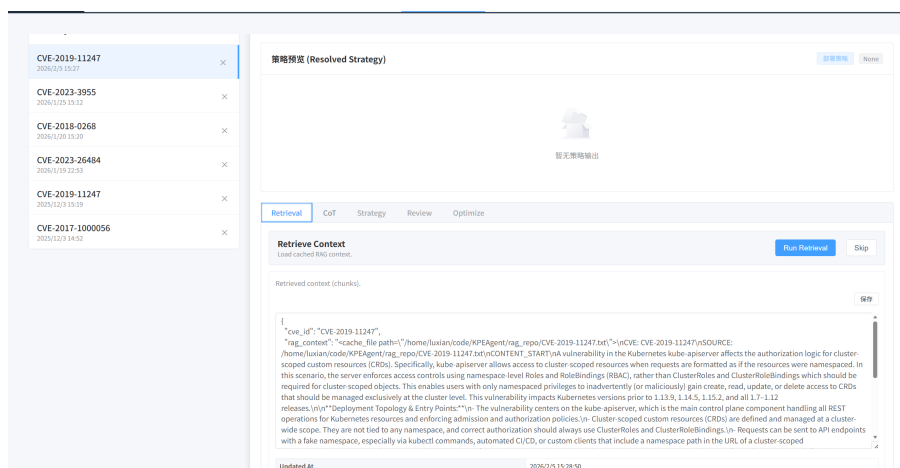


图 4-10: 检索增强生成（RAG）阶段的检索结果展示

Fig. 4-10: Display of retrieval results in the retrieval-augmented generation stage

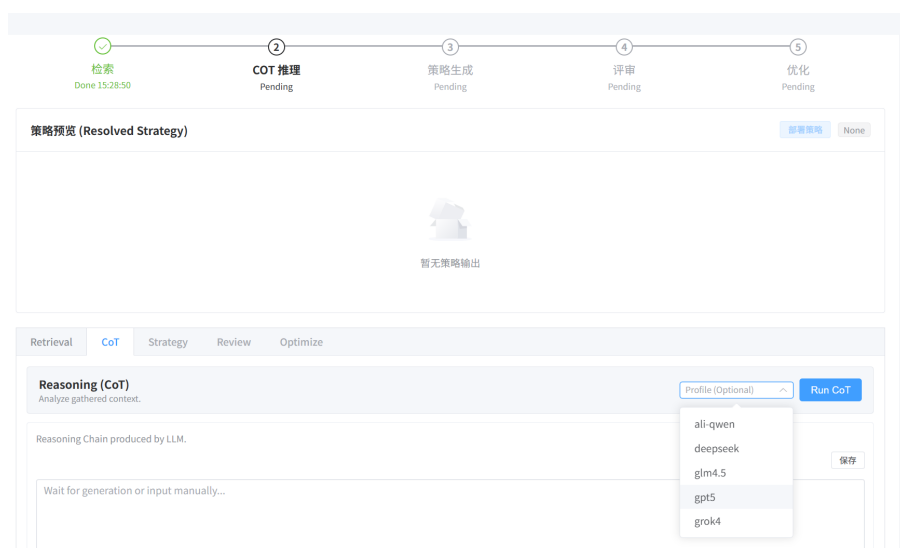


图 4-11: 思维链推理（CoT）展示界面

Fig. 4-11: Interface for displaying the chain-of-thought reasoning process

4.3.3 专家评审与排序

专家评审与排序模块对应服务层的评审与排序组件，其输入为策略生成模块输出的候选策略集合，输出为满足排序实体（Ranking）结构约束的偏好数据。该模块的设计目标是将专家知识有效引入策略选择过程，并为后续模型对齐提供高质量偏好信号。为此，系统设计了“评审”与“排序”两阶段流程。

评审阶段侧重采集细粒度定性反馈。如图 4-15 所示，结构化评审详情页要求专家评审员围绕语法正确性、业务逻辑完备度等指标对候选策略进行逐项核查，并允许输入自由文本意见。此类非结构化反馈可作为提示词修订与策略优化的重要依据。

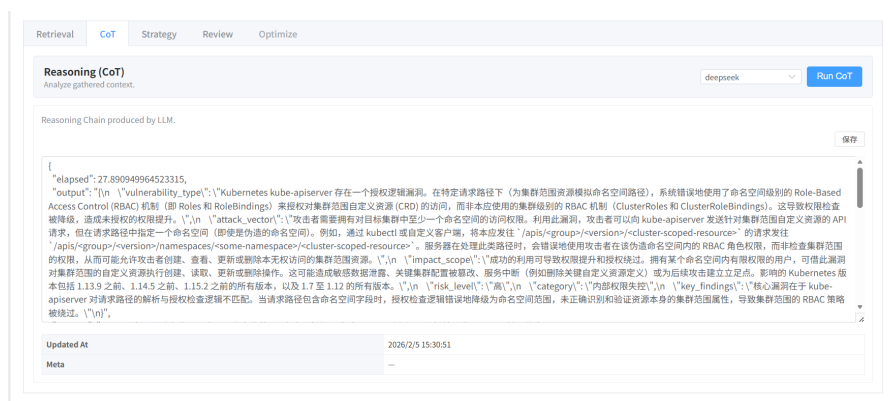


图 4-12: 思维链推理（CoT）详情

Fig. 4-12: Details of chain-of-thought reasoning

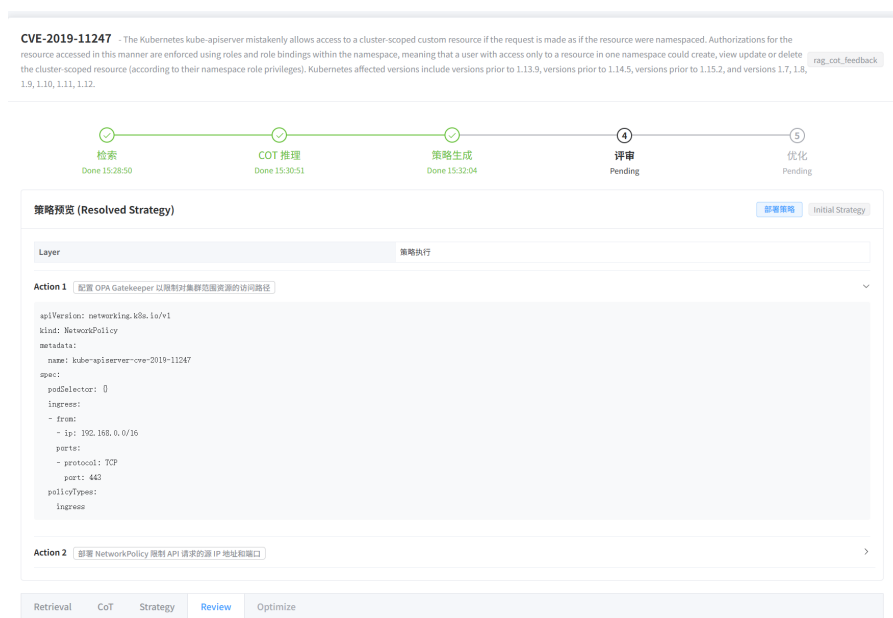


图 4-13: 策略生成初步输出

Fig. 4-13: Initial output of strategy generation

排序阶段侧重多候选方案之间的相对比较与一致性判别。如图 4-16 所示，拖拽式排序主界面左侧列出同一 CVE 对应的多个候选策略，右侧设置有效性、可部署性与无干扰性等维度评分区，并提供必要的排序提示。从交互机制看，拖拽排序属于相对判断范式，与绝对打分相比更有利于降低评分尺度漂移，提高不同专家评审员之间的结果一致性。同时，维度评分区为排序结果提供显式解释依据。在数据采集层面，系统以排序结果作为主信号，并以维度得分与文本备注作为辅助信号，共同构成偏好样本的结构化表示，便于后续开展一致性分析与模型对齐训练。

为支持细粒度比较，系统允许专家评审员展开单个候选策略的详细信息。如图 4-17 所示，专家评审员可在该界面对不同方案的规则逻辑、关键参数与实现细节进行逐项核对，重点关注以下三个方面：（1）策略对攻击路径与权

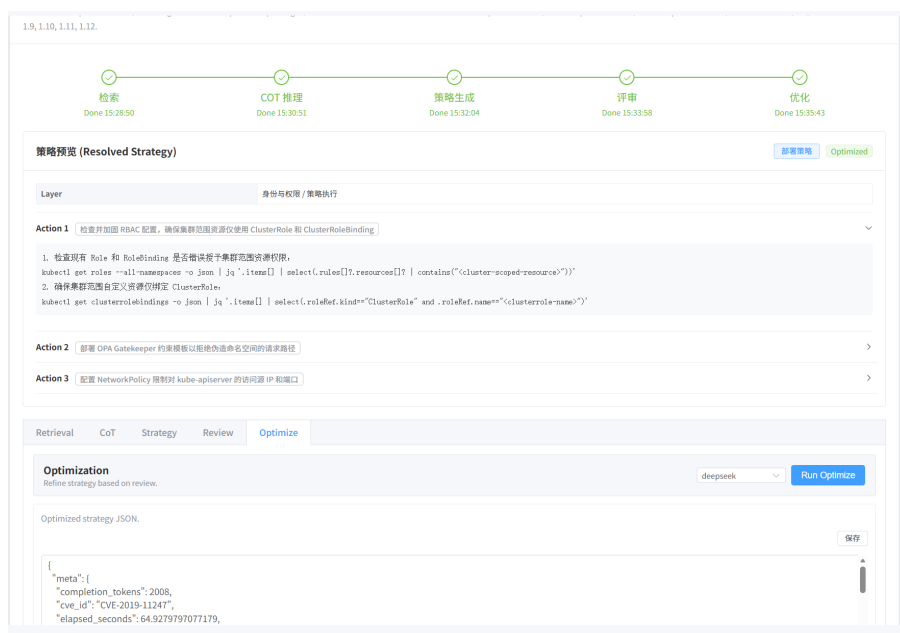


图 4-14: 策略优化后的最终输出

Fig. 4-14: Final output after strategy optimization



图 4-15: 评审阶段的结构化输出示例

Fig. 4-15: Example of structured output in the review stage

限边界的覆盖范围；（2）规则前置条件与约束是否充分且不过度收紧；（3）执行代价与潜在业务影响是否可接受。该设计有助于减少因信息分散造成的判断偏差，并提高排序决策的稳定性。

最终确认的排序结果通过提交界面（图 4-18）写入数据库，并与维度得分、时间戳及专家评审员标识一并存档，形成可用于后续分析与模型对齐的偏好数据集。为保证数据质量，系统在提交前执行一致性校验，例如检查排序结果是否构成全序、是否存在缺失条目，并在持久化过程中绑定候选策略版本与会话上下文，以降低后续策略迭代造成的语义漂移风险。

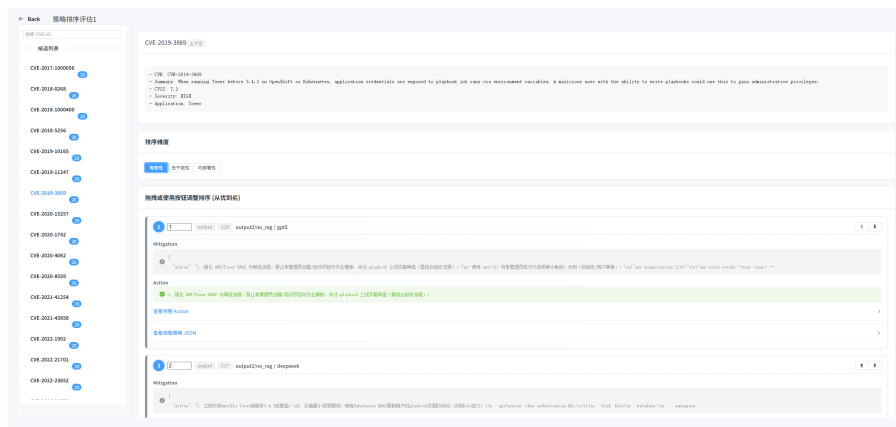


图 4-16: 策略排序主界面

Fig. 4-16: Main interface for strategy ranking

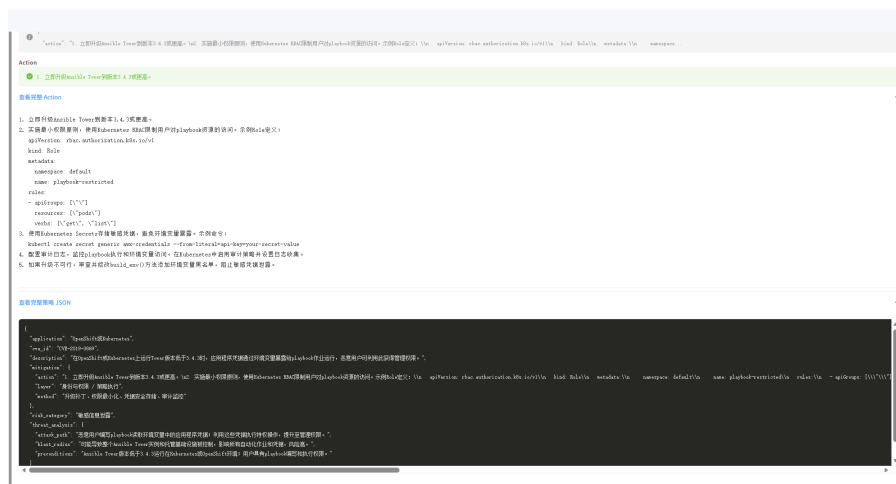


图 4-17: 策略详情查看界面

Fig. 4-17: Interface for viewing strategy details

4.3.4 策略部署管理

策略部署管理模块对应服务层的部署管理组件与外部目标节点的交互，负责将通过评审的候选策略安全下发至目标环境，并在 CVE 处置生命周期中完成从“已批准”到“已部署”的状态迁移。如图 4-19 所示，部署目标配置界面针对 Kubernetes 集群和传统主机环境分别提供差异化配置方式。对于 Kubernetes 集群，系统通过 Python Kubernetes SDK 与 API Server 建立连接，安全工程师需配置集群地址、访问令牌与命名空间；对于传统主机环境，系统支持基于 SSH 的部署方式，需要输入主机地址、端口与认证密钥。正式部署前，系统提供连通性测试功能，用于验证配置有效性及通道可用性。

部署完成后，系统通过部署管理面板统一展示跨集群、跨主机的策略实例。如图 4-20 所示，界面记录策略名称、目标对象、部署时间及当前状态等信息，并提供回滚与日志查看操作，以支持部署结果核查、异常诊断与后续

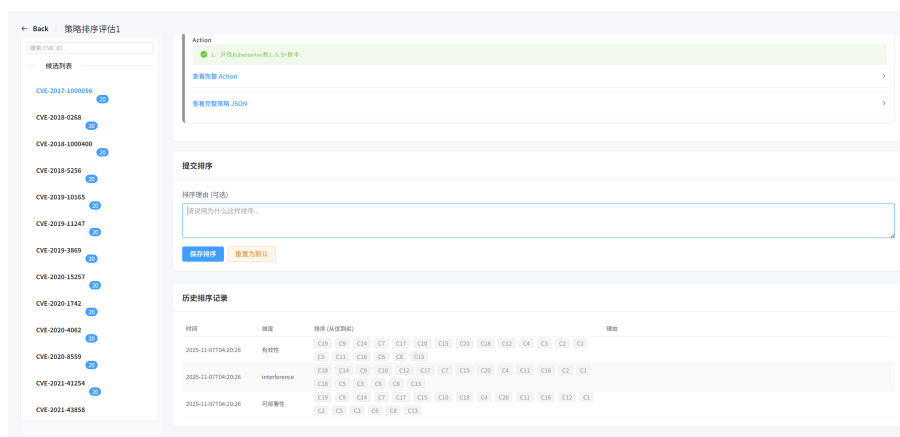


图 4-18: 排序结果提交界面

Fig. 4-18: Interface for submitting ranking results

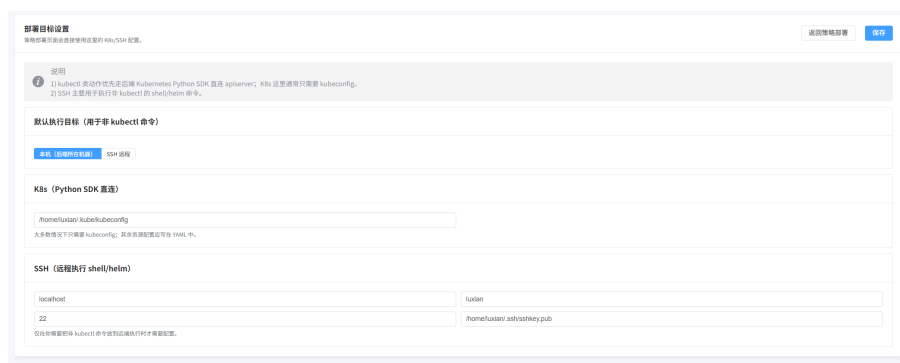


图 4-19: 部署目标配置界面

Fig. 4-19: Interface for configuring deployment targets

运维管理。

4.4 小结

本章围绕 KPEAgent 原型系统的设计与实现展开论述。首先从用户角色与功能需求出发完成需求分析；随后在总体设计阶段给出分层架构、数据模型、数据流转与处置流程以及交互时序四方面的设计方案；最后在详细设计阶段按知识库管理、策略生成与优化、专家评审与排序以及策略部署管理四个模块，结合系统运行界面说明了各模块的交互流程与实现方式。上述设计为下一章的实验验证与效果分析提供了平台支撑。

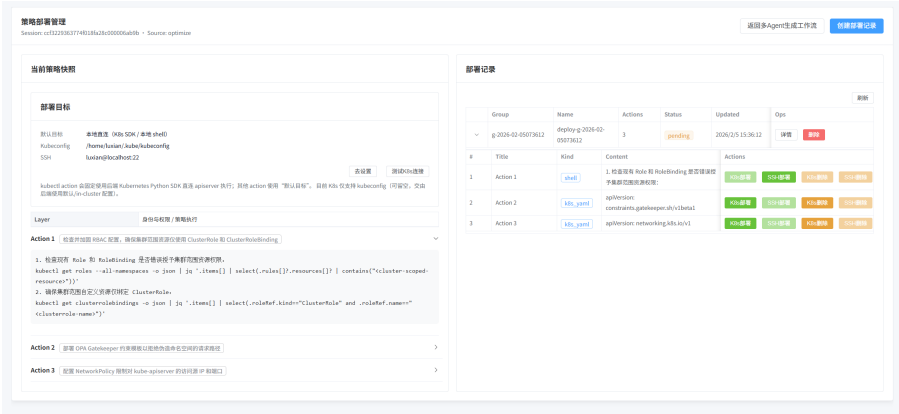


Fig. 4-20: Interface for strategy deployment management

5 实验研究

5.1 研究问题

本章实验旨在验证所提方法在权限提升漏洞实时防控场景下的有效性与适用性，具体围绕以下研究问题展开：

- RQ1:** 三个核心模块在三维质量指标上的贡献度与跨模型稳健性如何？
- RQ2:** 不同大语言模型在策略生成任务中呈现怎样的能力异质性？
- RQ3:** 基于大语言模型的自动化评审方法是否具备足够的内部一致性与外部有效性？
- RQ4:** 相较于现有静态分析工具，本文方法能否提供更具针对性的漏洞处置方案？
- RQ5:** 不同模型与方法配置在资源消耗上呈现怎样的成本-效益权衡特征？

5.2 实验设置

实验以 55 个权限提升漏洞实例为对象，在统一的提示词与输出结构规范下对五个大语言模型（LLM）生成候选策略集合 Π_v 。表5-1 给出本实验所采用的五个大语言模型的基本信息。为验证关键模块的贡献，设置三项消融：移除检索增强（w/o rag）、移除思维链推理（w/o cot）、移除基于评审反馈的反馈优化（w/o feedback），其余流程保持一致。

表 5-1: 实验采用的大语言模型
Tab. 5-1: Large language models used in experiments

模型代号	开发机构	发布日期	参数规模
Qwen3-Max	阿里云	2025 年 9 月	1000B+
DeepSeek-V3.1	深度求索	2025 年 9 月	670B
GPT-5	OpenAI	2025 年 6 月	未公开
Grok 4	xAI	2025 年 7 月	未公开
GLM-4.5	智谱 AI	2025 年 7 月	355B

评审阶段将上述五个模型作为评审模型，对 Π_v 进行排名式评价。评价指标采用三维质量向量 $q(\pi) = (E(\pi), I(\pi), D(\pi))$ ，分别对应有效性、无干扰性与可部署性。为缓解大模型评分能力缺陷 [68]，并降低不同评审模型评分

尺度差异带来的影响，本文以相对排序作为基础评价信号。随后依据第三章给出的等权聚合模型，将各评审模型的维度排序映射为归一化得分并计算共识得分，再据此形成最终的共识排序，以增强结果的稳健性与可解释性。

为评估令牌消耗及调用成本，本文在生成与评审阶段记录每次调用的输入与输出令牌数，并按 CVE 样本聚合统计；对于包含反馈优化的配置，则累计其多轮调用的令牌消耗。

为与传统静态安全工具比较，本文选取四种代表性工具或文献作为基线。在得到风险合规报告后，再人工评估其对策略生成的实际帮助，包括能否定位与漏洞路径相关的高风险配置，以及是否给出可直接落地的修正建议。

表 5-2: 静态扫描工具
Tab. 5-2: Static scanning tools

工具	来源
kube-linter	https://github.com/stackrox/kube-linter
Kubescape	https://github.com/kubescape/kubescape
KubeSec	https://kubesecc.io/
SLI-Kube	Rahman 等 [9]

5.3 RQ1: 消融实验与 workflow 模块贡献

本节首先分析三个模块各自带来的差异，随后考察这些差异在不同生成模型下是否仍然成立。比较对象包括完整 workflow `all` 与三种消融配置 `w/o rag`、`w/o cot`、`w/o feedback`，考察维度为有效性 E 、无干扰性 I 与可部署性 D 。

图5-1给出了四种方法配置在三个维度上的整体排序，表5-3列出了对应得分。55 个权限提升漏洞样本先由五个生成模型产生候选策略，再交由五个评审模型独立排序，最后经等权聚合得到各维度的共识结果。完整 workflow 在三个维度上均占优，因此下文先讨论各模块带来的变化，再考察这种优势是否受具体模型影响。

表5-3最直接地显示出反馈优化的作用。移除该模块后，有效性、无干扰性和可部署性分别降至 0.121、0.269 和 0.288，其中有效性从 0.581 降到 0.121，下降幅度最大。单次生成往往难以同时兼顾漏洞覆盖、配置约束和落地细节，而评审反馈提供了持续纠偏的过程，使策略能够在多轮修正中逐步逼近可用

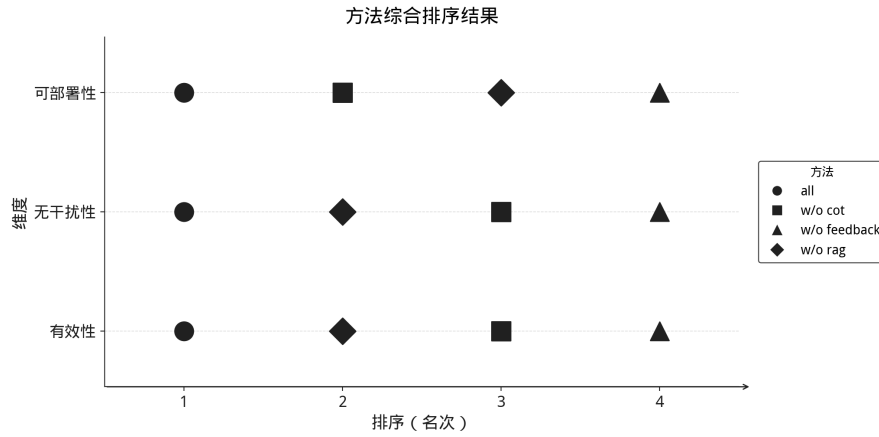


图 5-1: 方法配置在三个维度上的排名对比

Fig. 5-1: Comparison of method configurations across three dimensions

表 5-3: 消融实验方法配置得分

Tab. 5-3: Average scores in ablation study

方法配置	有效性 (E)	无干扰性 (I)	可部署性 (D)
all	0.581	0.431	0.403
w/o rag	0.476	0.370	0.321
w/o cot	0.234	0.340	0.398
w/o feedback	0.121	0.269	0.288

状态。

检索增强更多影响策略与具体漏洞场景的贴合度。w/o rag 在有效性维度上的得分为 0.476，虽然低于完整工作流，但仍明显高于 w/o cot 的 0.234 和 w/o feedback 的 0.121。这说明漏洞特征、历史处置案例以及 Kubernetes 安全实践被引入上下文后，模型更容易围绕真实攻击面组织策略，而不是停留在一般性的加固建议上。

思维链推理的作用则没有前两者那样单向。去除 CoT 后，有效性明显下降，但可部署性仍为 $D = 0.398$ ，与完整工作流的 $D = 0.403$ 相差不大。换言之，分步推理确实有助于把漏洞成因、利用路径和处置逻辑梳理得更完整，但这类更充分的分析有时也会带来更重的配置表达和更复杂的依赖关系。对于强调处置彻底性的场景，保留 CoT 更有价值；如果目标是尽快下发可执行策略，则适度简化推理过程未必是坏事。

综合以上结果，三个模块的作用并不处在同一层面。反馈优化决定策略能否在迭代中逐步收敛，检索增强主要影响策略与漏洞场景的贴合度，思维链推理则更多改变分析深度与部署复杂度之间的平衡。在资源受限的情况下，

若只能保留部分模块，反馈优化和检索增强应优先保留。

5.3.1 workflow跨模型稳健性验证

前述比较基于聚合后的整体结果。进一步在单个生成模型层面考察完整工作流的优势是否仍然存在。图5-2给出不同生成模型在有效性维度上的共识排序。从图中可见，a11 在五个生成模型条件下大多保持最优或次优，说明前述结论并非由个别模型驱动。不同模型对模块的敏感性仍有差别：GPT-5 与 GLM-4.5 在移除反馈后下降更明显，DeepSeek 与 Qwen3-Max 则对检索增强更敏感。这表明模块作用的方向总体一致，但各模型对知识补充和反馈修正的依赖程度并不完全相同。

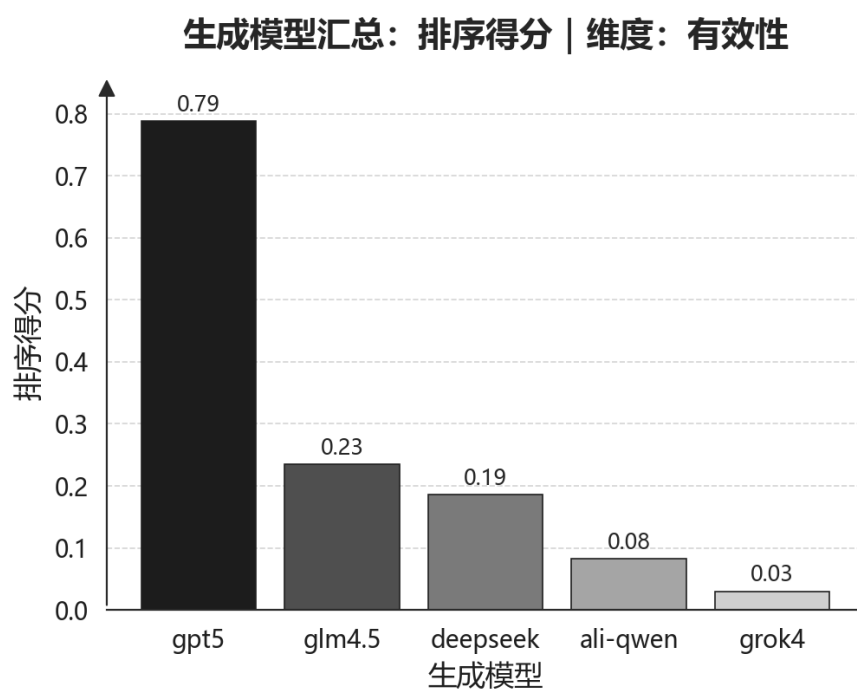


图 5-2: 生成模型有效性维度评分

Fig. 5-2: Scores of generation models on the effectiveness dimension

5.4 RQ2: 生成模型能力评估

RQ2 将讨论对象由 workflow 模块转向生成模型本身。下文分别比较质量得分和输出稳定性，再结合最佳方法配置讨论不同模型各自的取向。

5.4.1 三维质量指标的能力分化

图5-3展示了五个生成模型在有效性 E 、无干扰性 I 与可部署性 D 三个维度上的排名分布。

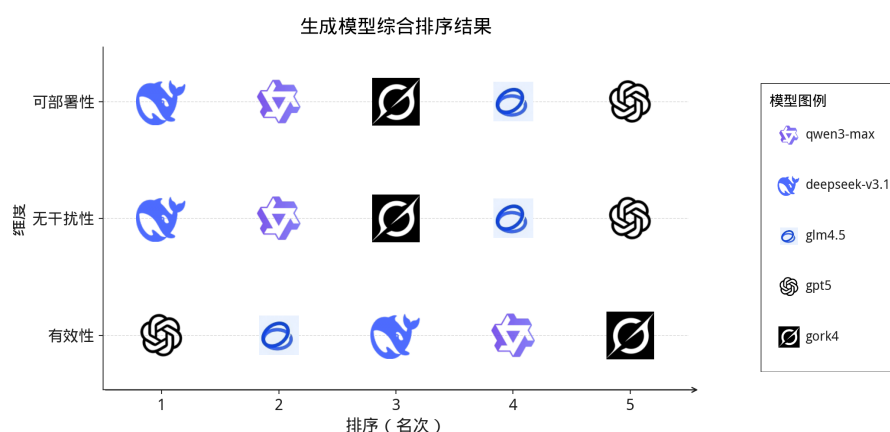


图 5-3: 生成模型在三个维度上的排名对比

Fig. 5-3: Comparison of generation model rankings across three dimensions

表5-4给出了五个生成模型在三个维度上的详细得分。

表 5-4: 生成模型三维质量得分

Tab. 5-4: Quality scores of generation models

生成模型	有效性 (E)	无干扰性 (I)	可部署性 (D)
GPT-5	0.814	0.175	0.000
GLM-4.5	0.327	0.324	0.336
DeepSeek	0.284	0.497	0.605
Qwen3-Max	0.193	0.425	0.477
Grok 4	0.147	0.342	0.346

表5-4显示，五个生成模型的得分没有沿同一方向变化，而是分散在不同目标之间。GPT-5 在有效性维度上明显领先，得分达到 $E = 0.814$ ，但无干扰性和可部署性分别只有 0.175 和 0.000，说明它更容易给出约束较强、处置力度较大的方案。DeepSeek 则呈现出相反的倾向，其无干扰性和可部署性分别达到 $I = 0.497$ 和 $D = 0.605$ ，均为五个模型中最高，但在有效性上并不占优。这意味着它更倾向于生成改动范围较小、部署代价较低的策略。

其余几个模型位于这两种倾向之间。Qwen3-Max 在有效性上的得分不高，但在无干扰性和可部署性上维持了相对较好的水平；GLM-4.5 三个维度

的分布较为接近，没有出现某一维度特别突出的情况；Grok 4 的三项得分都不算高，但分布相对平缓。依据这组结果，很难对五个模型作简单的线性排序。更合理的解释是，不同模型对应了不同的策略风格：有的更强调封堵强度，有的更强调部署成本和业务扰动。

5.4.2 输出稳定性评估

为评估不同生成模型在策略生成过程中的输出稳定性，本文统计了各模型在生成阶段触发重试的次数。重试机制主要针对三类失败情形：结构化输出中的 JSON 结构错误，通常因字段缺失、括号不匹配或编码问题导致；生成内容的语法错误，包括 YAML 语法错误、逻辑矛盾或不可解析的配置片段；以及内容违规，即触发模型供应商的安全策略或内容审核机制。为便于跨模型对比，本文将计数按固定分母 504 归一化为失误率。表5-5 给出五个生成模型的失误率统计。

表 5-5: 生成模型输出稳定性统计
Tab. 5-5: Output stability statistics of generation models

错误类型	DeepSeek 3.1	GLM -4.5	GPT -5	Grok 4	Qwen 3
JSON 格式错误	0.99%	4.56%	0.00%	0.99%	2.58%
语法错误	0.60%	1.98%	0.00%	0.00%	0.99%
内容违规	0.00%	1.98%	0.00%	0.00%	0.00%
合计	1.59%	8.53%	0.00%	0.99%	3.57%

从重试统计看，不同模型在结构化输出约束下的稳定性差异较为明显。GPT-5 在三类错误上均未触发重试，Grok 4 和 DeepSeek 的总失误率也较低，分别为 0.99% 和 1.59%。相比之下，GLM-4.5 的总失误率达到 8.53%，其中 JSON 格式错误占比最高；Qwen3-Max 的总失误率为 3.57%，主要问题同样集中在结构化输出。这说明在固定输出格式的要求下，后两者更容易出现需要重试或后处理的情况。

此外，只有 GLM-4.5 出现了内容违规，失误率为 1.98%。这意味着它在安全相关语境下更容易触发供应商侧的限制。结合质量得分与重试统计，GPT-5 的优势主要体现在有效性和生成稳定性，DeepSeek 则在可部署性、无干扰性和稳定性之间保持了较好的平衡；Grok 4 的质量得分不高，但重试成本较低；

GLM-4.5 与 Qwen3-Max 则需要预留更多后处理和重试空间。因此，这部分结果更适合用于区分模型的适用场景，而不宜压缩成单一排序。

5.4.3 生成模型能力评估小结

综合三维得分、重试情况和最佳方法配置，GPT-5 和 DeepSeek 代表了两种最鲜明的生成倾向。前者在有效性上优势明显，后者则在无干扰性和可部署性上更占优势，其余模型提供了不同程度的折中。

再结合表5-6可以看到，完整工作流在多数模型和维度上都能取得最优或次优结果，这与 RQ1 的消融结论基本一致。不过，这一结论并非没有例外。以 DeepSeek 为例，其在 w/o cot 配置下的可部署性达到 0.749，高于完整工作流的 0.678。这说明 CoT 并不必然对所有模型都带来同方向收益；至少对部分模型而言，更简化的推理过程有时反而更有利于生成易部署的策略。

表 5-6: 生成模型最佳方法配置
Tab. 5-6: Best method configurations per model

生成模型	有效性最佳	无干扰性最佳	可部署性最佳
GPT-5	all (0.945)	w/o cot (0.369)	w/o cot (0.256)
GLM-4.5	all (0.670)	all (0.522)	all (0.504)
DeepSeek	all (0.647)	w/o rag (0.634)	w/o cot (0.749)
Qwen3-Max	all (0.609)	all (0.629)	all (0.656)
Grok 4	all (0.434)	all (0.589)	all (0.597)

因此，五个生成模型提供的是不同风格的候选策略，而不是一条单一的强弱序列。若更看重漏洞封堵的充分性，GPT-5 的结果更值得参考；若更看重部署代价和业务扰动，DeepSeek 更合适。Qwen3-Max、GLM-4.5 和 Grok 4 则可以作为补充候选，提供不同侧重的备选策略。

5.5 RQ3: 评审方法可信度评估

前两节分别讨论了工作流差异和模型差异，这些比较都建立在一个前提之上，即评审结果本身具有足够可靠性。RQ3 即围绕这一前提展开验证。先考察五个评审模型之间的一致性，再把自动评审与人工专家评审对照，判断聚合后的结果是否具有足够可信度。

5.5.1 不同大语言模型的评审一致性验证

首先考察评审模型之间能否给出接近的排序。不同模型若在同一批候选策略上给出相近判断，聚合结果就更可信；反之，后续统计结论的解释空间也会随之增大。

在固定评价维度下，设共有 n 个候选项与 m 个大语言模型评审模型，候选项索引 $i = 1, \dots, n$ ，评审模型索引 $j = 1, \dots, m$ 。令 r_{ij} 表示评审模型 j 对候选项 i 给出的名次，名次越小表示越优。

Kendall's W 协调系数用于衡量多评审模型整体一致性，取值范围为 $[0, 1]$ ， $W = 1$ 表示完全一致：

$$W = \frac{12 S}{m^2 (n^3 - n)} \quad (5-1)$$

其中

$$S = \sum_{i=1}^n (R_i - \bar{R})^2, \quad R_i = \sum_{j=1}^m r_{ij}, \quad \bar{R} = \frac{1}{n} \sum_{i=1}^n R_i. \quad (5-2)$$

Spearman 等级相关系数用于比较任意两个评审模型排序趋势的一致性。对评审模型 j 与 k ，其排名向量分别记为 $\mathbf{r}_j$ 与 $\mathbf{r}_k$ ，则

$$\rho_{jk} = \text{corr}(\mathbf{r}_j, \mathbf{r}_k). \quad (5-3)$$

当两个评审模型排序趋势一致时 ρ_{jk} 接近 1，当趋势相反时 ρ_{jk} 接近 -1 。

本文在有效性 E 、无干扰性 I 和可部署性 D 三个维度上计算 Kendall's W ，并统计评审模型之间的 Spearman 相关系数。表5-7给出量化结果，图5-4到图5-5b展示分布细节。

表 5-7: 不同维度下大语言模型评审一致性统计

Tab. 5-7: Consistency statistics of large-language-model judges across dimensions

维度	Kendall's W	Spearman 相关系数			
		均值	中位数	最小值	最大值
有效性	0.955	0.944	0.944	0.904	0.991
无干扰性	0.846	0.808	0.839	0.585	0.932
可部署性	0.687	0.609	0.689	0.238	0.947

三项维度的一致性从高到低依次为有效性、无干扰性和可部署性。

表5-7显示，有效性维度的一致性最高， $W = 0.955$ ，Spearman 均值为 0.944。这说明不同评审模型对漏洞是否被有效封堵、攻击路径是否被覆盖，

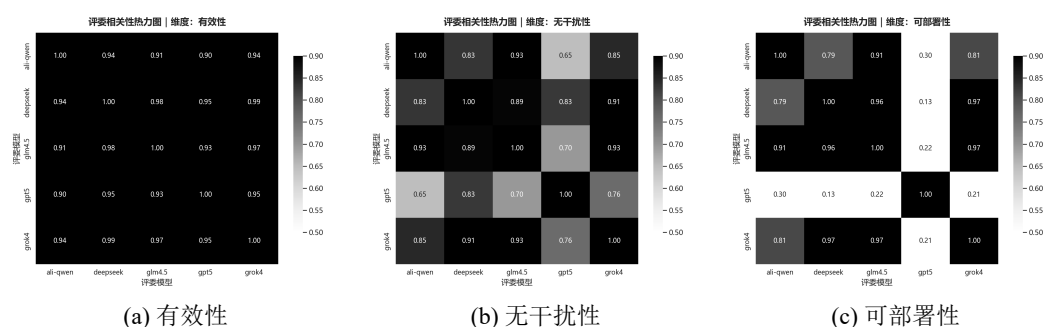


图 5-4: 不同维度下大语言模型评审两两相关性热力图 (Spearman)

Fig. 5-4: Pairwise correlation heatmaps of large-language-model judges across different dimensions (Spearman)

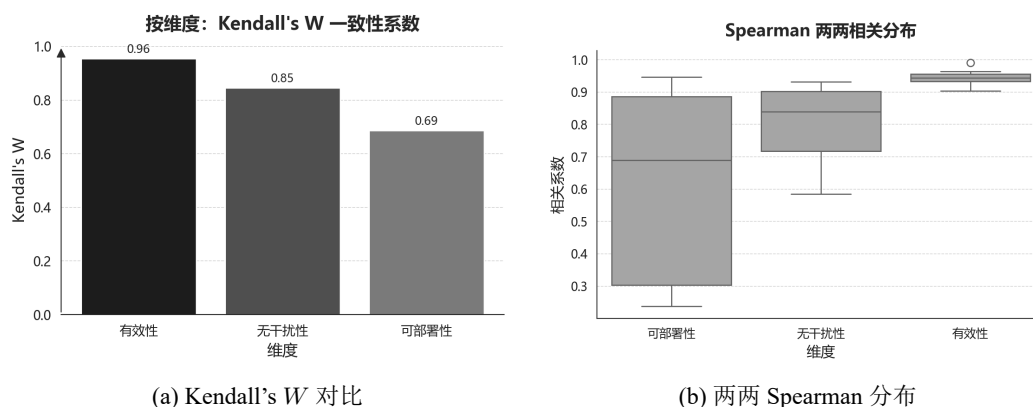


图 5-5: 大语言模型评审一致性统计

Fig. 5-5: Consistency statistics of large-language-model judges

往往会给出相近的排序。无干扰性居中， $W = 0.846$ ，Spearman 均值为 0.808。可部署性最低， $W = 0.687$ ，Spearman 均值为 0.609。这一维度需要同时考虑工程依赖、部署成本和运维复杂度，不同评审模型更容易在权衡标准上出现偏差。

因此，多评审结果虽然在三个维度上强弱有别，但等权聚合后的排序并未出现明显失稳。

5.5.2 跨维度评审差异分析

在一致性统计的基础上，还需结合图形结果观察这种差异的具体分布。本小节结合图5-7与图5-8，分析不同维度的分歧主要落在何处。

图5-7中，可部署性上的分歧最明显。有的评审模型更看重配置是否简洁，有的更在意依赖是否完备，因此该维度更容易拉开评审者之间的距离。相比之下，有效性上的排序接近一致，说明在漏洞覆盖和处置充分性上，各评审模型的判断标准更接近。无干扰性介于两者之间。也正因为如此，聚合策略

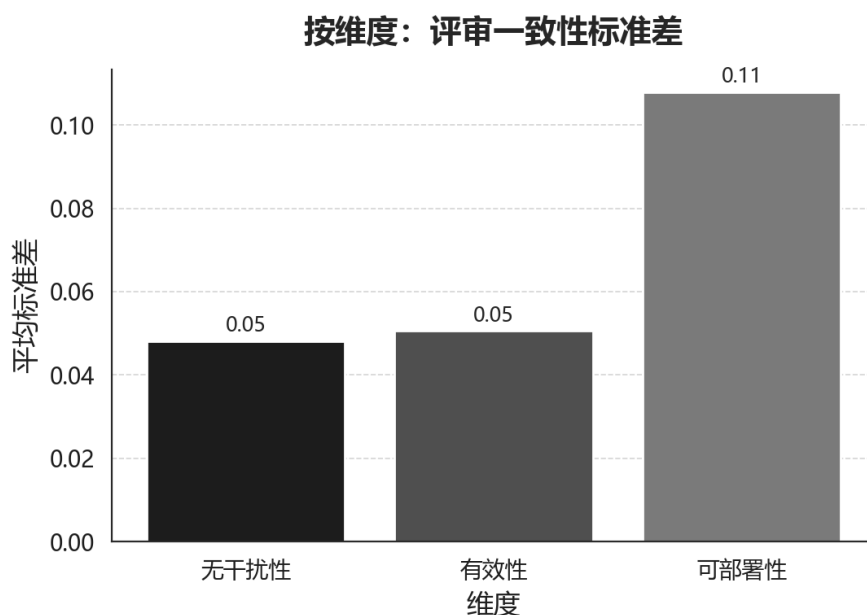


图 5-6: 不同维度下大语言模型评审分歧程度统计

Fig. 5-6: Statistics of disagreement among large-language-model judges across different dimensions

在可部署性维度上的意义更大，它能够削弱单个评审模型的偏好。

图5-8则反映了生成模型本身的取向。GPT-5 在有效性上领先，但可部署性最低；DeepSeek 在可部署性和无干扰性上更强；Qwen3-Max 与 GLM-4.5 的三项得分分布相对平稳。把这组结果与评审一致性放在一起看，可以发现有效性和可部署性之间存在较明显的拉扯关系。为了提高封堵强度，策略往往会变得更复杂；而结构越简洁，越容易部署，也越可能减少对业务的额外影响。

5.5.3 与人工评审的对比较验证

仅凭内部一致性尚不足以证明评审结果可靠，还需要检验自动评审能否接近人工判断。为此，本小节将聚合后的自动评审结果与人工专家评审进行对照。人工评审由三位具有五年以上 Kubernetes 安全运维经验的专家完成。评审对象为随机抽取的 20 个 CVE 样本，覆盖不同漏洞类型和危害等级。每位专家分别对五个生成模型在四种方法配置下生成的策略排序，再对三位专家的结果做等权聚合，作为人工评审基准。

给定 N 个待评审样本，第 i 个样本对应 K 个大语言模型评审者的打分向量 $\mathbf{x}_i = (x_{i,1}, \dots, x_{i,K})^\top$ ，以及人工评审分数 y_i 。目标是求得一组权重，使自动评审的加权结果尽量接近人工评审。实验比较两种做法。

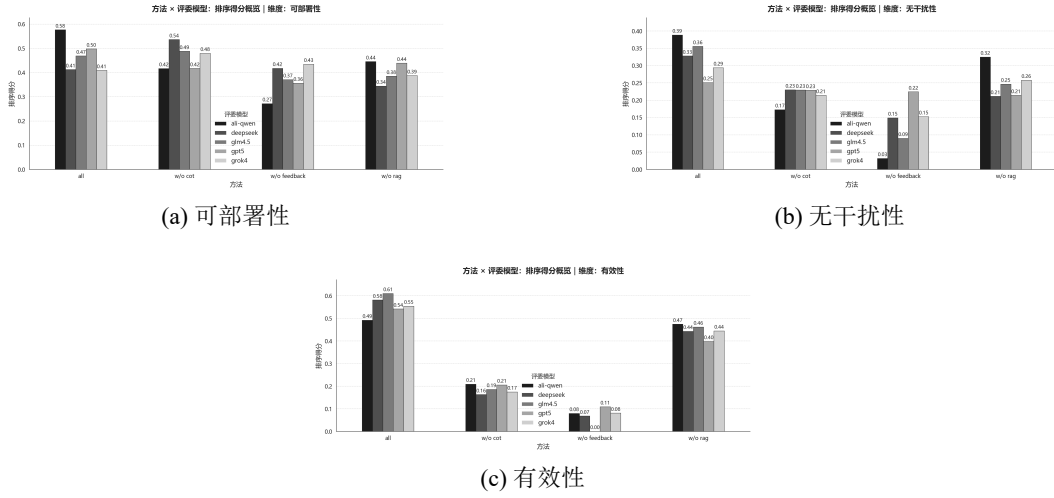


图 5-7: 评审模型单维度评分对比

Fig. 5-7: Comparison of single-dimension scores among judge models

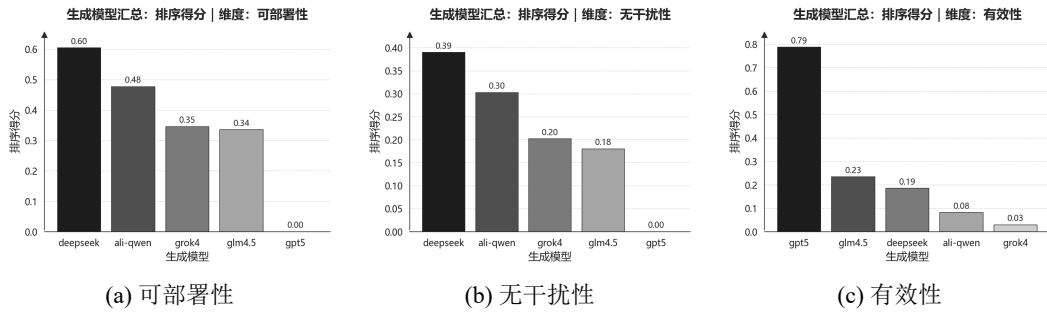


图 5-8: 生成模型三维质量评分对比

Fig. 5-8: Comparison of three-dimensional quality scores among generation models

方案 1 采用凸组合权重，直接加权平均： $\hat{y}_i = \sum_{j=1}^K w_j x_{i,j}$ ，约束 $w_j \geq 0$ 且 $\sum_j w_j = 1$ 。通过最小化 MSE 求解：

$$\min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N \left(\sum_{j=1}^K w_j x_{i,j} - y_i \right)^2 \quad (5-4)$$

方案 2 在加权平均之后再线性变换：

$$\hat{y}_i = a \left(\sum_{j=1}^K w_j x_{i,j} \right) + b \quad (5-5)$$

权重 $\mathbf{w}$ 保持相同约束，参数 a 和 b 由最小二乘拟合。这样既可以调整分数尺度，也可以修正整体偏移。三个评审维度分别拟合独立权重。评审者打分从排名转为分数时采用组内归一化，公式为 $\text{score} = (N_{\max} - \text{avg_rank}) / (N_{\max} - 1)$ 。其中 $N_{\max}$ 表示该组候选数上限。泛化能力采用留一法和 5 折交叉验证评估。表5-8给出了两种方案的结果。

表5-8显示，加入线性校准后，5 折验证误差由 0.0373 降到 0.0331，降幅

表 5-8: 两种权重方案的交叉验证结果
Tab. 5-8: Cross-validation results of two weighting schemes

方案/维度	训练 MSE	LOO MSE	5-fold MSE
凸组合权重			
有效性	0.0344	0.0496	0.0458
无干扰性	0.0344	0.0324	0.0329
可部署性	0.0344	0.0341	0.0332
总体	0.0344	0.0387	0.0373
带校准权重			
有效性	0.0263	0.0371	0.0335
无干扰性	0.0263	0.0285	0.0256
可部署性	0.0263	0.0378	0.0402
总体	0.0263	0.0345	0.0331
改善幅度	23.6%	10.9%	11.3%

约为 11%。训练误差下降更明显，说明线性校准确实吸收了一部分尺度偏差；但训练集与验证集的降幅并不完全同步，也提示了过拟合风险。

分维度来看,有效性的改善最明显,5 折验证误差从 0.0458 降到 0.0335。无干扰性也有明显下降。可部署性则没有受益,误差反而从 0.0332 升到 0.0402。一个直接原因是,这一维度上的评审分数本来就比较集中,继续增加校准参数后,更容易把噪声也一并拟合进去。表5-9给出了用全量数据学习得到的权重。

表 5-9: 学习到的评审者权重分布
Tab. 5-9: Learned weight distribution of large-language-model judges

评审者	有效性	无干扰性	可部署性
qwen3-max	0.42	0.38	0.20
deepseek	0.18	0.15	0.20
glm4.5	0.15	0.22	0.20
gpt5	0.20	0.18	0.20
grok4	0.05	0.07	0.20

从权重分布看,有效性和无干扰性两个维度里, Qwen3-Max 的权重最高, 分别为 0.42 和 0.38。这说明在这两个维度上, 它的评审结果与人工判断最接近。GPT-5 和 DeepSeek 在有效性上的权重也不低。可部署性则呈现完全均匀的分布, 说明在这一维度上, 很难仅凭现有评审信号把某个模型明显区分出

来。

图5-9把人工评审结果进一步可视化。按生成模型看，GPT-5 在有效性上仍然最高，DeepSeek 在可部署性上仍然最好；按方法配置看，完整 workflow 在三个维度上依旧领先。这与前文自动评审得到的总体趋势是一致的。差别主要出现在可部署性上，人工评审拉开的差距更明显，说明人工评审者对部署难度的变化更敏感。

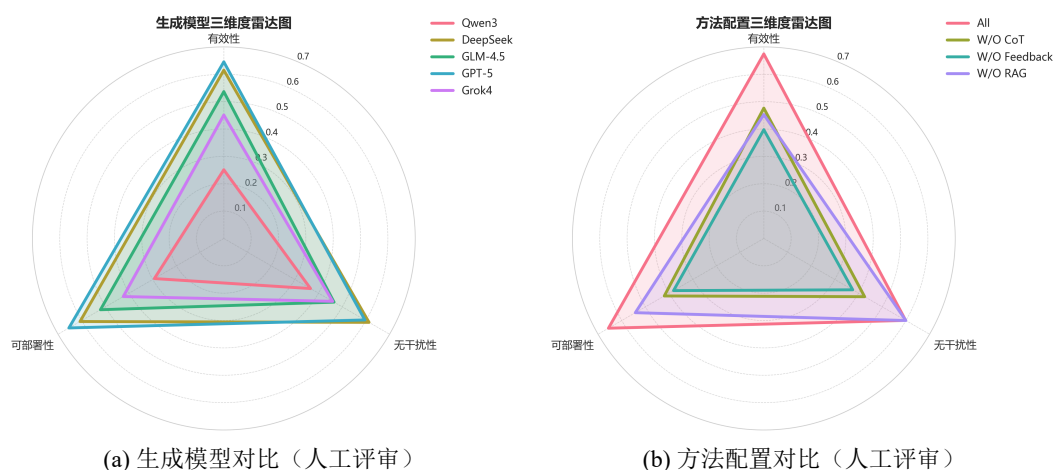


图 5-9: 人工评审的三维度雷达图对比

Fig. 5-9: Comparison of three-dimensional radar charts based on human evaluation

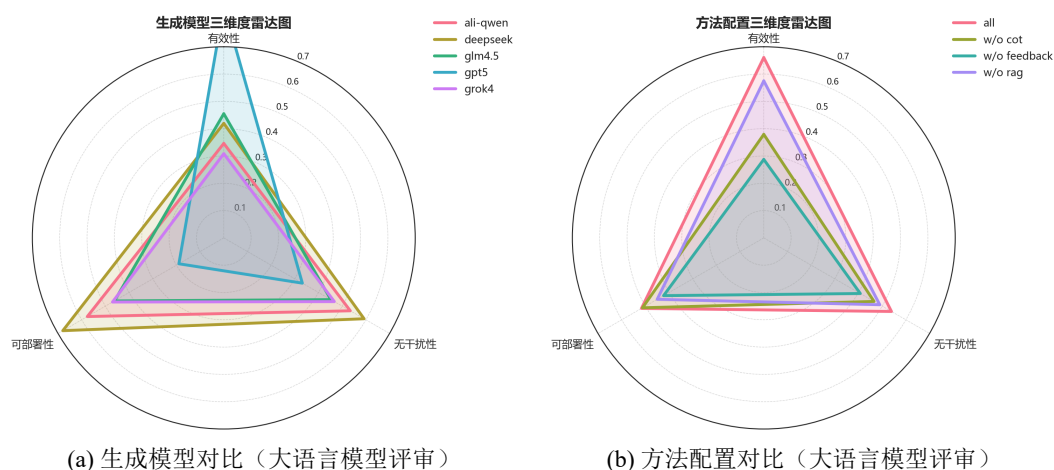


图 5-10: 大语言模型评审的三维度雷达图对比

Fig. 5-10: Comparison of three-dimensional radar charts based on large-language-model evaluation

表5-10给出了校准参数。有效性和无干扰性的 a 介于 0.35 到 0.52 之间，说明自动评审分数整体偏高，压缩后才更接近人工分数。可部署性的 a 只有 0.08，说明这一维度里的自动评审信号较弱，预测更多依赖常数项。

表 5-10: 校准参数与拟合质量
Tab. 5-10: Calibration parameters and fitting quality

维度	a	b	训练 MSE	5-fold MSE
有效性	0.52	0.24	0.025	0.034
无干扰性	0.35	0.33	0.020	0.026
可部署性	0.08	0.46	0.034	0.040

综合来看，线性校准对有效性和无干扰性是有帮助的，但对可部署性的作用有限。可部署性本身分歧更大，与人工结果的线性对应也更弱，因此很难通过一组简单参数把它校准好。

这一部分的直接限制还是样本量偏小。每个维度只有 20 个观测点，很难支撑更复杂的拟合。当前模型也只使用了排名分数这一类特征，信息量本身有限。后续如果要继续提高拟合质量，更现实的做法是先扩大样本，再引入评审置信度、反馈轮次中的分数变化、文本长度和配置复杂度等辅助信号，然后再考虑更复杂的校准模型。

5.5.4 小结

综合内部一致性与人工对照结果，当前多评审聚合方案足以支撑前文的主要比较。限制主要集中在可部署性维度。该维度的评审分歧更大，与人工结果的对应关系也更弱。后续若要提高这一部分的可靠性，首先应细化评价标准，其次再考虑更稳健的聚合方法。

5.6 RQ4: 静态扫描的处置价值

前几节讨论的是工作流内部模块、生成模型和评审方法，RQ4 则引入外部基线，考察传统静态扫描能否为临时漏洞处置提供同样直接的线索。为此，选取 Kubescape、KubeSec、kube-linter 和 SLI-Kube 作为对照工具，在同一批 Kubernetes 清单上运行后，将报告条目按统一风险类型归并统计，结果见表5-11。

表5-11显示，四类工具的报告主要集中在配置基线与通用加固项，例如资源配置缺失、身份与密钥暴露、网络隔离和非 Root 运行等。这类结果对日常治理和合规检查仍有价值，但与本文收集的 55 个权限提升漏洞逐一对应时，能够直接指向漏洞触发前提的情况并不多。实际对照后，只有 CVE20204062 与 CVE202431989 能在报告中找到较明确的关联线索，而且都落在网络隔离

表 5-11: 静态扫描工具检测结果
Tab. 5-11: Detection results of static scanning tools

风险类型	关联一级分类	Kubescape	KubeSec	kube-linter	SLI-Kube
资源配置缺失	配置缺陷	110	47	73	0
非 Root 运行	配置缺陷	54	53	51	0
根文件系统非只读	配置缺陷	39	34	59	0
网络隔离	配置缺陷	132	0	0	15
特权与权限提升	配置缺陷	39	0	12	0
主机级暴露	配置缺陷	30	0	9	3
身份与密钥	敏感泄露	192	58	0	2
系统调用隔离	配置缺陷	0	58	0	1
探针配置	配置缺陷	92	0	15	0

项上，分别由 Kubescape 与 SLI-Kube 命中。其余样本的关键攻击路径通常依赖具体组件行为、权限边界或运行时条件，超出了静态规则直接覆盖的范围。就本文关注的补丁窗口期临时处置而言，静态扫描更适合作为背景性的安全治理工具，而不是主要的决策依据。

5.7 RQ5: 资源消耗与效率评估

RQ4 说明静态扫描难以直接替代策略生成，RQ5 则进一步考察生成式流程的资源代价。这里分别统计令牌消耗和运行时间，因为前者直接对应调用成本，后者决定在漏洞响应窗口内能否及时完成一轮生成与评审。

表5-12给出了不同方法配置与生成模型下的平均令牌消耗及对应成本：

从表5-12看，方法配置对令牌成本的影响首先体现在是否保留反馈环节。完整工作流的平均成本为 12.99 元，去掉反馈后降到 9.88 元，降幅最明显；去掉 RAG 和 CoT 后则分别为 11.10 元和 11.29 元。说明多轮反馈是额外令牌开销的主要来源。不同模型之间的差异更大。GPT-5 在四种配置下的成本都明显高于其他模型，完整配置达到 50.40 元；DeepSeek 处于中间；Qwen3-Max 与 GLM-4.5 的费用相对较低。也就是说，流程复杂度决定了成本会增加多少，而模型选择则直接拉开了总支出水平。

时间开销呈现出相近的结构。表5-13统计了 56 个 CVE 样本在不同方法配置和生成模型下的平均耗时，单位为秒：

表 5-12: 令牌消耗与经济成本统计
Tab. 5-12: Token usage and economic cost statistics

方法	模型	输入 Token	输出 Token	输入成本 (CNY)	输出成本 (CNY)	总成本 (CNY)	平均成本 (CNY)
All	qwen3-max	228,842	92,360	0.57	0.92	1.50	12.99
	deepseek	213,174	360,923	0.51	2.60	3.11	
	glm4.5	217,216	158,493	0.17	0.32	0.49	
	gpt5	376,812	672,931	3.30	47.11	50.40	
	grok4	246,549	108,188	2.96	6.49	9.45	
W/O CoT	qwen3-max	147,128	44,759	0.37	0.45	0.82	11.29
	deepseek	143,191	310,209	0.34	2.23	2.58	
	glm4.5	138,905	141,923	0.11	0.28	0.39	
	gpt5	201,623	646,417	1.76	45.25	47.01	
	grok4	165,321	61,481	1.98	3.69	5.67	
W/O Feedback	qwen3-max	129,241	44,841	0.32	0.45	0.77	9.88
	deepseek	120,138	280,609	0.29	2.02	2.31	
	glm4.5	122,551	155,095	0.10	0.31	0.41	
	gpt5	178,449	554,871	1.56	38.84	40.40	
	grok4	145,538	62,736	1.75	3.76	5.51	
W/O RAG	qwen3-max	184,820	46,291	0.46	0.46	0.92	11.10
	deepseek	170,295	318,002	0.41	2.29	2.70	
	glm4.5	170,290	116,108	0.14	0.23	0.37	
	gpt5	338,456	603,583	2.96	42.25	45.21	
	grok4	203,726	64,326	2.44	3.86	6.30	

从表5-13看，反馈环节同样是时间增长的主要来源。去掉反馈后，方法均值降至 86.98 秒，其余三种配置都在 132 到 136 秒之间。按模型区分时，Grok 4 与 Qwen3-Max 在各配置下都维持在几十秒量级，GLM-4.5 居中，DeepSeek 与 GPT-5 明显更慢，其中 GPT-5 在完整配置下达到 344.48 秒。时间统计与令牌成本基本一致，说明更高质量的生成通常伴随着更长的推理和更高的调用支出。

综合成本和时间结果，并不存在对所有场景都占优的选择。若响应窗口较短、预算也有限，Qwen3-Max 和 Grok 4 的资源负担较轻；若更看重漏洞封堵的充分性，GPT-5 的代价最高，但也给出了最强的有效性得分；DeepSeek 介于两者之间，以更长的处理时间换取较好的可部署性和无干扰性。因此，这一节给出的不是单一最优组合，而是不同资源约束下可接受的取舍范围。

表 5-13: 时间成本统计
Tab. 5-13: Time cost statistics

方法配置	模型	平均耗时 (秒)	方法均值 (秒)
W/O RAG	qwen3-max	27.57	132.48
	deepseek	212.84	
	glm4.5	55.34	
	gpt5	340.42	
	grok4	26.23	
W/O CoT	qwen3-max	26.22	135.64
	deepseek	219.34	
	glm4.5	74.22	
	gpt5	328.47	
	grok4	29.93	
W/O Feedback	qwen3-max	15.94	86.98
	deepseek	140.94	
	glm4.5	35.51	
	gpt5	228.50	
	grok4	14.00	
All	qwen3-max	28.93	136.39
	deepseek	215.20	
	glm4.5	65.78	
	gpt5	344.48	
	grok4	27.55	

6 案例分析

为验证本文提出的多智能体协同安全策略生成框架在真实云原生权限提升漏洞场景下的有效性，本章选取若干典型 CVE 案例进行深入分析。每个案例包括**漏洞复现**、**策略生成**与**三维质量验证**三个阶段：首先在可控环境中重现漏洞攻击路径并确认风险；随后使用本文方法自动生成**监控策略**与**防护策略**候选集合；最后从有效性 E 、无干扰性 I 与可部署性 D 三个维度对生成策略进行实证评估，以验证策略能否真实阻断攻击、是否引入业务干扰以及部署难度是否可控。

6.1 案例分析实验设计

6.1.1 CVE 案例选择标准

为保证案例的代表性与可复现性，本文依据以下标准筛选案例 CVE：

风险等级与影响范围：优先选择 CVSS 评分 ≥ 5.0 的高危漏洞，且在 Kubernetes 生态中具有广泛影响（如 Kubernetes 核心组件漏洞或高使用率的第三方应用漏洞）。

成因类型覆盖：确保所选案例覆盖本文提出的两级成因分类框架中的主要类别，包括配置缺陷（如过度权限配置、网络隔离不足）、安全逻辑缺陷（如访问控制缺陷、身份验证绕过）、敏感信息泄露与代码注入等，以全面检验方法的适用性。

可复现性：漏洞具备公开的攻击 PoC（Proof of Concept）或详尽技术细节，可在隔离环境中安全复现；同时受影响版本与修复版本明确，便于对比验证。

防控价值：漏洞场景具有典型的临时防控需求（如修复补丁尚未发布、业务无法立即升级或需要多层防御），使得智能生成策略具有实际应用价值。

6.1.2 实验环境与复现流程

环境搭建。所有漏洞复现与策略验证在隔离的 Kubernetes 测试集群中进行。为便于复核与复现，实验环境的软硬件配置与安全组件版本统一汇总如表 6-2。

表 6-1: 本章所选 CVE 案例一览
Tab. 6-1: Overview of selected CVE cases in this chapter

序号	CVE 编号	影响组件	一级分类	二级分类
1	CVE-2022-24768	Argo CD	安全逻辑缺陷	访问控制缺陷
2	CVE-2025-2241	Hive	敏感信息泄露	API 端口暴露
3	CVE-2023-30512	CubeFS CSI	配置缺陷	冗余权限
4	CVE-2020-4062	ConjurOSS	配置缺陷	网络隔离不足
5	CVE-2022-24877	Flux	代码注入	路径遍历
6	CVE-2022-29171	Sourcegraph	代码注入	命令注入
7	CVE-2023-22482	Argo CD	安全逻辑缺陷	身份验证绕过

漏洞复现流程。对每个 CVE，按以下步骤进行复现：

1. **基线环境准备：**部署受影响版本的目标组件，并创建模拟的业务负载与用户身份（如低权限 ServiceAccount）；
2. **攻击执行：**根据公开 PoC 或技术分析，模拟攻击者执行权限提升攻击，记录攻击步骤与观测到的异常行为；
3. **影响确认：**验证攻击是否成功实现权限提升（如获取集群管理员权限、访问受限资源、执行特权操作等）；

策略生成与验证。复现成功后，使用本文方法生成安全策略并进行三维评价：

1. **安全上下文构建（检索增强（RAG））：**将 CVE 公告、受影响组件文档、攻击日志与社区讨论输入向量知识库，检索与漏洞相关的证据；
2. **策略生成（思维链推理（CoT）与反馈优化（Feedback））：**基于检索增强上下文与漏洞分析结果，生成候选策略并进行评审反馈优化；
3. **策略部署与复测：**在测试集群中部署生成的安全策略（如 NetworkPolicy、RBAC 规则、Gatekeeper 策略等），重新执行攻击以验证策略是否有效阻断；
4. **三维质量评价：**
 - **有效性（E）：**策略能否成功阻断原攻击路径，是否存在绕过可能；
 - **无干扰性（I）：**策略对正常业务流量的影响（如误报率、性能开销、运维复杂度）；
 - **可部署性（D）：**策略配置的完整性与可执行性，是否包含清晰的

表 6-2: 实验环境软硬件配置与工具版本汇总

Tab. 6-2: Summary of hardware and software environment configuration and tool versions

项目	配置
计算资源	本地虚拟机集群, 96 核 CPU, 96GB 内存, 200GB 存储
集群工具与版本	Minikube v1.18.1; Kubernetes v1.18.3
节点规模与规格	1 个控制节点与 2 个工作节点; 每节点 4 核 CPU/8GB 内存
CNI/网络策略	Calico v3.28.2, Kubernetes-Network-Policy v1.5.5
准入控制	OPA/Gatekeeper v3.14.0; Kyverno v1.11.0
运行时安全监控	Falco v0.38.1
包管理工具	Helm v3

验证与回滚方案。

6.2 案例一：CVE-2022-24768 内部权限问题

6.2.1 漏洞详解与复现

漏洞背景

Argo CD 是 Kubernetes 生态中广泛使用的声明式 GitOps 持续交付工具。其典型部署形态为：Argo CD 控制平面以相对高权限身份（对集群资源拥有创建/更新/删除能力）持续对齐 Git 仓库中的“期望状态”。该架构提升交付一致性的同时，也带来显著的安全挑战：一旦普通用户能够影响 Argo CD 的控制决策，便可能借助控制平面权限实现权限边界放大。

CVE 描述：CVE-2022-24768 为 Argo CD 的不当访问控制（Improper Access Control）漏洞。所有未修复版本（自 1.0.0 起）均存在风险，0.5.x 与 0.8.x 系列中包含该问题的受限版本。攻击者需要满足以下前置条件之一：(i) 对某个 Application 的源 Git/Helm 仓库具备 push 权限；或 (ii) 在 Argo CD 中对该 Application 具备 sync 与 override 权限。满足条件后，攻击者可通过构造/修改仓库内容或应用配置，诱导 Argo CD 以其控制平面权限执行超出攻击者原始 RBAC 边界的操作，进而潜在提升至 admin 级别能力。官方已在 Argo CD 2.1.14、2.2.8、2.3.2 中发布补丁。缓解措施可降低风险但不能替代升级。

主要成因分类：安全逻辑缺陷 / 访问控制缺陷（已授权用户的权限放大）。
攻击链条描述：其抽象攻击链可概括为：

1. **合法身份阶段**: 攻击者以普通 Argo CD 用户身份登录;
2. **配置影响阶段**: 凭借对 Application 的 sync+override 或对源码仓库的 push 权限, 注入恶意资源清单;
3. **控制平面滥用阶段**: Argo CD 按 GitOps 同步逻辑将恶意资源应用到集群;
4. **权限提升阶段**: 借助被创建的高权限 RBAC 绑定获得集群级管理员能力。

漏洞复现

本节给出可复现的实验流程: 构造包含”正常资源 + 恶意 RBAC 绑定”的 Git 仓库, 为低权用户配置 sync 与 override 权限, 触发 Argo CD 同步, 在集群中创建管理员级绑定实现权限提升。

环境配置: Kubernetes 测试集群可选 kind 或 minikube。受影响组件为 Argo CD v2.3.1, 补丁版本为 2.3.2、2.2.8、2.1.14。测试负载使用仓库 https://github.com/Ivanbeethoven/bad_repo, 包含 app.yaml 与 hack-admin.yaml。

攻击步骤:

本文复现包括以下关键环节: 部署漏洞版本并创建宽松 AppProject; 授予低权用户 sync+override; 创建指向恶意仓库的 Application 并触发同步; 最后验证权限变化。为避免正文过长, 关键命令与清单见附录B。

攻击结果: 综合上述验证可观察到权限边界放大。低权用户 bob 原本仅具备对特定项目范围内应用的 sync/override 能力, 但通过影响 Git 期望状态或同步动作, 间接诱导控制平面在集群中创建高权限 RBAC 绑定, 最终表现为在 --as bob 条件下 `kubectl auth can-i '*' '*'` 返回 yes。

```
r2cn-ubuntu-01 → ~ kubectl describe clusterrolebinding bob-admin-binding
Name:          bob-admin-binding
Labels:        app.kubernetes.io/instance=bob-app
Annotations:   <none>
Role:
  Kind: ClusterRole
  Name: cluster-admin
Subjects:
  Kind  Name  Namespace
----  ---  -
User   bob
```

图 6-1: 漏洞复现实验结果示意 (验证权限变化)

Fig. 6-1: Experimental result of vulnerability reproduction showing privilege change verification

```
# 检查 bob 对 namespaces 的权限
kubectl auth can-i create namespaces --as bob
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  annotations:
    kubernetes.io/last-applied-configuration: |
      {"apiVersion":"rbac.authorization.k8s.io/v1","kind":"ClusterRoleBinding","metadata":{"annotations":{},"labels":{"app.kubernetes.io/instance":"bob-app"},"name":"bob-admin-binding"},"roleRef":{"apiGroup":"rbac.authorization.k8s.io","kind":"ClusterRole","name":"cluster-admin"},"subjects":[{"apiGroup":"rbac.authorization.k8s.io","kind":"User","name":"bob"}]}
      creationTimestamp: "2025-12-17T12:02:56Z"
      labels:
        app.kubernetes.io/instance: bob-app
        name: bob-admin-binding
        resourceVersion: "529144"
        uid: 45b0bb4b-605e-4b00-a4d7-9fe650c3171e
  roleRef:
    apiGroup: rbac.authorization.k8s.io
    kind: ClusterRole
    name: cluster-admin
  subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: User
    name: bob
yes
yes
yes
yes
Warning: resource 'namespaces' is not namespace scoped
```

图 6-2: 漏洞复现实验结果示意（验证是否有命令空间级别权限）

Fig. 6-2: Experimental result of vulnerability reproduction showing namespace-level privilege verification

6.2.2 策略部署

本节给出可落地的缓解策略。该策略遵循纵深防御思路，强调限制配置可达性、限制配置表达力以及约束授权来源，通过补偿性控制压制攻击路径。

策略概述：从身份与权限、策略执行、准入控制以及审计治理四个层面协同落地。方法侧以 Kubernetes RBAC 收敛与 Argo CD RBAC 加固为基础，在 AppProject 与权限策略的创建更新环节施加准入约束，并通过身份组映射白名单、审计告警、基线与回滚机制降低权限漂移风险。相关能力可由 Kubernetes RBAC、Argo CD RBAC、Gatekeeper 或 Kyverno 以及审计与告警系统实现。

策略配置：为便于复用与自动化，可按先审计后强制的节奏分阶段落地。集群侧通过 Kubernetes RBAC 收敛 AppProject 的创建更新删除权限，应用侧通过 Argo CD RBAC 将默认权限设置为只读并显式隔离 projects、repositories、clusters、exec 等高风险能力。策略侧在 AppProject 创建更新环节校验策略表达，限制通配符放权并约束权限授予范围。治理侧将关键配置纳入 Git 基线并接入审计告警，降低权限漂移风险。

为避免正文堆叠大量清单，Kubernetes RBAC、Argo CD RBAC、Gatekeeper/Kyverno 准入策略的完整配置与应用命令见附录B。

6.2.3 三因素分析

有效性 (Effectiveness): 该策略在攻击链的多个关键点形成阻断: Kubernetes RBAC 对 AppProject 写入进行限制,可降低放开 sourceRepos、destinations 与白名单导致的爆炸半径。Argo CD RBAC 默认只读并隔离高危资源写权限,使低权用户难以获得可用于放大控制面的关键操作集合。准入控制对策略表达施加硬约束,防止隐蔽通配符放权与权限漂移。身份治理约束授权来源,可降低组注入或错误映射造成的隐式权限提升。在复现实验中,若上述策略生效,则攻击者即使拥有 sync 权限,也难以通过修改 AppProject/仓库/策略来诱导 Argo CD 创建集群级高权限绑定,攻击路径被显著收敛。

无干扰性 (Interference): 该策略以“最小必要变更”为原则,主要影响集中在管理面与配置流程:对业务同步的直接影响可控,策略保留项目内正常的 sync 能力,仅收紧 override、高危资源写权限与高风险策略表达。准入控制建议先审计后强制,以降低误拦截对业务的冲击。额外运维开销主要来自 RBAC 精细化与准入规则维护,但可通过模板化与 GitOps 基线降低长期成本。

可部署性 (Deployability): 该策略依赖的能力均为云原生常用组件,具有较强可落地性:Kubernetes RBAC 与 Argo CD RBAC 均为原生能力,配置可版本化并可审计。Gatekeeper 或 Kyverno 属于成熟的准入控制方案,可按项目逐步推广,并与审计治理配套形成闭环。

6.3 案例二: CVE-2025-2241 (Hive ClusterProvision 敏感信息泄露)

6.3.1 漏洞详解与复现

漏洞背景

Hive 是 Red Hat Multicloud Engine (MCE) 与 Advanced Cluster Management (ACM) 中的集群生命周期管理组件,常用于在多集群场景下自动化创建与托管 OpenShift/Kubernetes 集群。在 vSphere 场景中, Hive 需要使用 vCenter 凭据完成集群的基础设施供应与后续管理。

CVE 描述: CVE-2025-2241 指出 Hive 在完成 vSphere 集群供应 (provisioning) 后,会将 vCenter 凭据暴露在 ClusterProvision 对象中。由于 ClusterProvision 属于自定义资源 (CR),拥有其读取权限的用户可直接

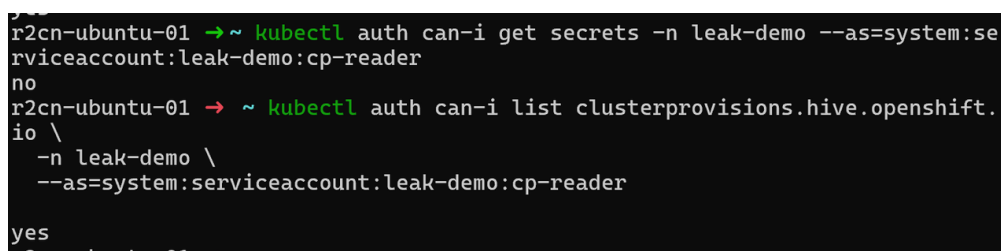
从该对象中提取敏感凭据，即使该用户并不具备对 Kubernetes Secret 的访问权限。该问题会导致未授权 vCenter 访问、基础设施与集群管理风险，并可能进一步演化为权限提升。

CVSS 信息：CNA (Red Hat) 给出的向量为 CVSS:3.1/AV:N/AC:H/PR:L, 基准分 8.2 (High)。其含义可理解为：攻击者需要一定前置权限 (PR:L) 和较高攻击复杂度 (AC:H)，但一旦满足条件，影响范围跨越安全边界 (S:C)，并造成高机密性与完整性影响 (C:H/I:H)。

主要成因分类：敏感信息泄露 / 配置与对象生命周期缺陷 (“Secret 旁路”：敏感数据出现在可读的 CR 状态字段中)。

攻击链条描述：该漏洞可抽象为 “Secret 隔离被 CR 可读性绕过” 的链条：

1. **权限前置阶段：**攻击者拥有对 ClusterProvision 的 get/list/watch 权限 (通常来自聚合角色 view/edit 或错误的绑定)；
2. **数据旁路阶段：**Hive 将 vCenter 凭据写入 ClusterProvision, 位于 status 字段；
3. **凭据提取阶段：**攻击者通过读取 CR YAML/JSON 直接获得凭据；
4. **后续风险阶段：**利用 vCenter 凭据对虚拟化平台执行未授权操作，进而影响集群节点与工作负载，造成横向移动或权限提升。



```
r2cn-ubuntu-01 → ~ kubectl auth can-i get secrets -n leak-demo --as=system:serviceaccount:leak-demo:cp-reader
no
r2cn-ubuntu-01 → ~ kubectl auth can-i list clusterprovisions.hive.openshift.io \
-n leak-demo \
--as=system:serviceaccount:leak-demo:cp-reader
yes
```

图 6-3: CVE-2025-2241: vCenter 凭据通过 ClusterProvision 暴露的风险示意

Fig. 6-3: CVE-2025-2241: illustration of vCenter credential exposure through ClusterProvision

漏洞复现

本案例复现目标是验证：无 Secret 访问权限但可读 ClusterProvision 的用户，是否可从对象中提取到 vCenter 凭据。由于该漏洞涉及敏感凭据泄露，本文仅给出受控测试环境的复现要点，详细 YAML 与命令序列见附录 C。

环境配置：测试集群为 OpenShift 或 Kubernetes，并已部署 ACM/MCE 与 Hive，具体版本以厂商公告与实验环境为准。基础设施侧需要可用的 vSphere 环境与测试专用的 vCenter 账号，建议采用最小权限配置。权限模型侧需准备

至少一个只读主体，其具备 `ClusterProvision` 的读取权限但不具备 `Secret` 的读取权限，用于验证旁路泄露条件。

攻击步骤（摘要）：

1. 创建 `vSphere` 集群供应配置（`Hive ClusterDeployment` 等相关对象），触发供应流程；
2. 使用低权限但可读 `ClusterProvision` 的身份读取对象并导出 `YAML`；
3. 在输出中定位 `vCenter` 用户名/密码等敏感字段（例如 `status.vCenter.credentials.*` 或同类结构）。

攻击结果：若在 `ClusterProvision` 的可读字段中发现明文凭据，则说明“敏感信息通过 `CR` 状态旁路泄露”成立；该风险不依赖 `Secret` 读取权限，属于权限边界外泄。

6.3.2 策略部署

本节给出可落地的缓解策略。本案例侧重敏感信息泄露的多层次控制：通过**防护策略**收敛读权限面，通过**监控策略**缩短暴露窗口。

策略概述：从**防护策略**（身份与权限、准入检测）与**监控策略**（审计与告警、资源生命周期治理）四个层面协同落地。具体方法包括：

- **防护策略：**收敛 `ClusterProvision` 的读取权限；采用 `Gatekeeper` 或 `Kyverno` 以 `dryrun/Audit` 模式对对象中疑似敏感字段进行检测提示。
- **监控策略：**启用读取行为审计并联动告警；在供应完成后对对象进行清理以缩短暴露窗口；在发现泄露风险后执行 `vCenter` 凭据轮换与强化。

策略配置与部署步骤：为避免正文过长，本案例的复现详细步骤、`RBAC` 清单、审计策略片段、自动化清理示例与 `Gatekeeper/Kyverno` 检测规则完整代码见附录C。

6.3.3 三因素分析

有效性（Effectiveness）：

`RBAC` 收敛可直接降低非必要主体读取 `ClusterProvision` 的概率，是风险抑制的关键手段。审计与告警用于发现异常读取行为，并可在凭据轮换后用于验证风险是否仍然存在。对象清理及其自动化可以缩短凭据暴露窗口，降低可利用时间。审计型准入检测则有助于在配置漂移或版本回退时及时发现敏感字段再次出现。

无干扰性 (Interference):

RBAC 收敛主要影响运维与排障可见性，需要以按需、限时、可审计的授权流程替代全局可读。审计与告警对业务链路通常无直接影响，但会增加日志量与告警治理成本。自动化清理需避免破坏 Hive 的对账或回收流程，建议先在预生产验证，并以标签或状态条件精确匹配对象后再删除。

可部署性 (Deployability):

防护策略层：RBAC 收敛属于平台原生能力，可在多集群中以模板化方式发布。准入**防护策略**建议从 `dryrun/Audit` 起步，用于检测与辅助治理，通常不建议强制拦截控制器写入 `status` 字段，以避免影响供应流程。**监控策略层：**审计与告警对业务链路通常无直接影响，但会增加日志量与告警治理成本。自动化清理属运维流程治理，建议与 GitOps 或流水线集成，在供应完成后打标签并由定时任务触发清理。

6.4 案例三：CVE-2023-30512 (CubeFS CSI 过度权限配置)

6.4.1 漏洞详解与复现

漏洞背景

CubeFS 是面向云原生场景的分布式存储系统，常通过 CSI (Container Storage Interface) 插件与 Kubernetes 集成。在 Kubernetes 中，CSI 节点插件通常以 DaemonSet 形式在每个节点上运行 (例如 `cfs-csi-node`)，控制器组件以 Deployment 形式运行 (例如 `cfs-csi-controller`)。这类组件需要与 Kubernetes API 交互，但其权限应遵循最小权限原则。

CVE 描述：CVE-2023-30512 的核心问题是 CubeFS CSI 插件的服务账号 (如 `cfs-csi-service-account`) 被绑定到了包含**过度权限**的集群级角色 (ClusterRole)，使其具备对集群范围 Secret 的读取能力 (如 `get/list`)。一旦攻击者在某节点获得对 CSI 节点插件 Pod 的控制 (例如通过节点被入侵、容器逃逸、或对系统命名空间特权工作负载的执行能力)，便可能滥用该服务账号的身份边界，读取全局敏感信息并进一步演化为集群级权限提升。

主要成因分类：配置缺陷 / 过度授权 (Over-privileged RBAC)。

攻击链条描述：该漏洞可抽象为“系统组件身份被过度授权 → 被攻陷后可枚举集群秘密”的链条：

1. **身份前置阶段：**CSI 组件使用默认或被误配置的 ServiceAccount 运行；

2. **权限放大阶段：**ServiceAccount 被绑定到含 secrets 读取能力的 ClusterRole；
3. **控制获取阶段：**攻击者控制某节点上的 CSI Pod（例如 cfs-csi-node）并获得其运行时上下文；
4. **数据获取阶段：**攻击者以该 ServiceAccount 身份读取集群 Secret，从而获取高价值凭据（管理员 Token、云凭据等），实现从“节点/Pod 控制”向“集群控制”的升级。

[illegible]

图 6-4: CVE-2023-30512: 攻击成功证据（基于 CSI ServiceAccount 的越权访问结果）
Fig. 6-4: CVE-2023-30512: evidence of successful attack based on unauthorized access through a CSI ServiceAccount

漏洞复现（验证性复现）

考虑到本文面向论文复现实验，且该漏洞的攻击细节容易被滥用，本文采用**验证性复现**思路：以“RBAC 权限证据 + 受控权限验证”为主，证明 `cfs-csi-service-account` 具备不应存在的 `secrets` 读取能力；同时在策略部署后复测，验证该能力被有效移除。完整命令与策略清单见附录D。

环境配置: Kubernetes 测试集群可选 Kind、Minikube 或实验集群，版本以实验环境为准。需要部署受影响版本的 CubeFS CSI，部署方式可为 Helm 或 YAML，命名空间按实际为准。同时应明确 CSI 使用的 ServiceAccount 以及其绑定的 ClusterRole 与 ClusterRoleBinding，以便开展权限证据提取与复测。

复现步骤（摘要）:

1. 盘点并导出 CSI 的 RBAC 绑定，确认 ClusterRole 规则含 `resources: [secrets]` 且 `verbs` 包含 `get/list/watch`;
2. 以该 ServiceAccount 身份执行权限验证 (`kubectl auth can-i`)，确认其具备跨命名空间读取 `Secret` 的能力;
3. 按“最小权限 + 准入防回归 + 审计检测”部署策略后，重复第 2 步验证

权限应为 no。

```
r2cn-ubuntu-01 → ~ curl -k -H "Authorization: Bearer $TOKEN" $API_SERVER/api/v1/
{
  "kind": "APIResourceList",
  "groupVersion": "v1",
  "resources": [
    {
      "name": "bindings",
      "singularName": "binding",
      "namespaced": true,
      "kind": "Binding",
      "verbs": [
        "create"
      ]
    },
    {
      "name": "componentstatuses",
      "singularName": "componentstatus",
      "namespaced": false,
      "kind": "ComponentStatus",
      "verbs": [
        "get",
        "list"
      ],
      "shortNames": [
        "cs"
      ]
    },
    {
      "name": "configmaps",
      "singularName": "configmap",
      "namespaced": true,
      "kind": "ConfigMap",
      "verbs": [
        "create",
        "delete",
        "deletecollection",
        "get",
        "list",
        "patch",
        "update",

```

图 6-5: CVE-2023-30512: 攻击成功证据 (Secrets 读取能力验证/权限提升结果截图)

Fig. 6-5: CVE-2023-30512: evidence of successful attack showing secret-reading capability and privilege escalation results

攻击结果 (摘要): 若服务账号具备集群范围 Secret 读取权限, 则攻击者一旦控制 CSI Pod/节点, 存在获取高价值凭据并进一步获得集群控制的风险。

6.4.2 策略部署

本案例侧重移除不必要的 Secrets 读取权限并防止权限回归: 通过防护策略 (RBAC 基线治理 + 准入策略) 与监控策略 (审计检测) 的组合, 在配置漂移或误授权场景下提供纵深防护。

策略概述: 从防护策略 (身份与权限、策略执行、网络收敛) 与监控策略 (审计与检测) 等层面协同落地。核心方法包括:

- **防护策略:** 对 CSI 相关 RBAC 进行最小化与清理, 按需评估 ServiceAccount 硬化措施; 在准入侧阻断再次引入含 secrets 读取动词的 ClusterRole; 必要时进一步收敛 CSI 工作负载的出站访问面。

- **监控策略：**对 secrets 的枚举行为建立可观测与告警。

策略配置与部署步骤：为避免正文过长，RBAC 最小化示例、准入控制规则、审计策略片段与验证命令见附录D。

6.4.3 三因素分析

有效性(Effectiveness)：RBAC 最小化可直接移除攻击链关键条件，即 secrets 读取能力，从根源降低风险。准入防回归用于阻断误配置、版本回滚或安装脚本重复执行导致的过度授权再次出现。审计检测可在异常访问出现时提供证据并触发凭据轮换等止血动作。

无干扰性(Interference)：RBAC 收敛可能影响 CSI 的部分诊断与自愈路径，应在预生产环境验证 PVC/PV 生命周期以及 attach/detach 等关键流程。准入策略若采用 deny 强制模式，需要避免误拦截平台必需组件的合法权限，建议先以 dryrun/audit 运行并结合白名单迭代。审计与检测会增加日志量与告警治理成本，但通常不影响业务链路。

可部署性(Deployability)：RBAC 与审计属于平台原生能力，适合在多集群场景中模板化推广。准入策略与网络收敛属于长期基线治理内容，可通过 GitOps 与策略即代码持续交付。工程落地时建议提供回滚与灰度路径，先在测试与预生产验证后再逐步推广至生产。

6.5 案例四：CVE-2020-4062 Conjur OSS 缺乏网络隔离策略

6.5.1 漏洞详解与复现

漏洞背景

Conjur OSS 是云原生场景下常用的机密管理系统，属于 Secrets Management 系统。在典型部署中，Conjur Server 负责鉴权与策略校验，后端数据库 PostgreSQL 用于持久化存储策略、审计与机密相关数据。Kubernetes 默认网络模型在多数容器网络接口 CNI 配置下呈扁平互通状态，Pod 间通信默认允许。因此，对于数据库等敏感组件，需要显式配置 NetworkPolicy 等隔离策略，形成默认拒绝策略 Default Deny，以控制最小暴露面。

CVE 描述：CVE-2020-4062 影响 Conjur OSS Helm Chart 2.0.0 之前版本。旧版 Chart 默认未提供对 Postgres 的访问控制策略，例如缺失 NetworkPolicy，导致数据库服务在集群内对任意 Pod 可达。即使数据库未通过 NodePort 或

LoadBalancer 暴露到公网，攻击者一旦获得集群内任意工作负载的执行上下文，仍可能横向移动并直连数据库，绕过应用层安全控制，从而造成机密数据泄露、策略被篡改与审计记录被破坏等后果。

严重等级：Critical。

主要成因分类：配置缺陷 / 网络访问控制缺失 Missing NetworkPolicy。

攻击链条描述：该漏洞可抽象为集群内横向移动与直连数据库绕过应用层安全的链条：

1. **初始立足点：**攻击者控制集群内任意 Pod，例如通过业务漏洞或错误授权获得容器执行能力；
2. **侦察发现：**在目标命名空间中发现 Postgres Service 与 5432 端口；
3. **网络直连：**由于缺失 NetworkPolicy，攻击者可从任意 Pod 直连 Postgres；
4. **影响扩展：**若获取数据库凭据或数据密钥（来源可能包括过宽 RBAC、配置泄露或备份泄露），风险可进一步扩展为机密窃取、权限篡改与审计删除。

漏洞复现（验证性复现）

本案例复现以**网络暴露面验证**为主：证明在旧版 Chart 部署下，Postgres 在集群内可被非 Conjur 组件访问，随后通过部署 NetworkPolicy 将该访问面收敛。考虑到该场景涉及数据库直连与敏感数据风险，本文不展开可被滥用的数据库操作细节，完整的环境准备与策略清单见附录E。

环境配置：测试集群为 Kubernetes（Minikube）与 Helm v3，受影响组件为 Conjur OSS Helm Chart < 2.0.0。同时要求集群 CNI 支持 NetworkPolicy；若不支持，则本文所述隔离策略无法生效。

复现步骤：

1. 在独立命名空间部署旧版 Conjur OSS Chart，并确认 Postgres Service 端口 5432 存在；
2. 证明该命名空间中不存在限制 Postgres 入站的 NetworkPolicy；
3. 进行受控连通性验证：从非 Conjur 组件工作负载发起到 Postgres 5432 的 TCP 探测，用于证明集群内可达；
4. 部署默认拒绝并仅允许 Conjur Server 访问 Postgres 的 NetworkPolicy，复测应不可达。

攻击结果（摘要）：若未隔离 Postgres，攻击者一旦获得集群内任意 Pod 的执行上下文，即存在横向移动并直连数据库的机会，形成从业务容器到机

密管理后端的跨域突破。

6.5.2 策略部署

该漏洞的治理重点是**收敛数据库暴露面并防止隔离策略回归**。可采用以 NetworkPolicy 为核心的缓解路径：以默认拒绝策略 Default Deny 收敛数据库入站访问面，仅允许 Conjur Server 访问 Postgres 5432 端口，并与命名空间隔离、RBAC 收敛及审计治理配套，以降低横向直连与配置漂移带来的风险。

策略概述：从网络隔离、身份与权限、审计与治理三个层面协同落地。网络层通过 NetworkPolicy 实现默认拒绝与精确放通，身份层通过 RBAC 限制对数据库凭据相关资源的读取面，治理层以基线检测与告警机制保证策略长期有效。

策略配置与部署步骤：为避免正文过长，NetworkPolicy 示例、验证命令与注意事项见附录E。

6.5.3 三因素分析

有效性 (Effectiveness)：NetworkPolicy 能直接收敛数据库暴露面，将任意 Pod 可达降至仅 Conjur Server 可达，从网络层阻断横向直连路径。命名空间隔离与 RBAC 收敛可进一步降低数据库凭据被读取或滥用的概率，与网络隔离形成互补。基线检测用于在 Helm 回滚或配置漂移时，及时发现 NetworkPolicy 缺失、规则被弱化等回归风险。

无干扰性 (Interference)：NetworkPolicy 的引入可能影响 Conjur 组件间通信与运维排障，需在预生产完成连通性与回归验证，并准备回滚方案。若集群 CNI 不支持 NetworkPolicy，则相关策略无法执行，应优先通过命名空间隔离等手段降低暴露面，或在平台层面选用支持策略的 CNI。精确放通依赖正确的标签选择器，标签不一致可能造成误拦截，需要与 Chart 模板保持一致。

可部署性 (Deployability)：NetworkPolicy 属于 Kubernetes 原生资源，便于纳入 GitOps 流程并在多集群环境中复制推广。由于数据库服务与端口相对稳定，规则维护成本较低。为避免遗漏与回归，可将 NetworkPolicy 纳入 Helm values 或部署流水线的强制检查项，并与基线检测形成闭环。

6.6 案例五：CVE-2022-24877 (Flux kustomize-controller 路径穿越)

6.6.1 漏洞详解与复现

漏洞背景

Flux 是云原生 GitOps 交付体系中的常用组件，典型部署包含 kustomize-controller 等控制器，用于在集群中持续对齐 Git 仓库的期望状态。在该工作流中，控制器会拉取仓库内容并调用 Kustomize 解析 kustomization.yaml，进而生成并应用资源清单。由于 kustomization.yaml 以“文件路径”为核心配置对象，控制器必须将其中指定的资源、补丁与组件路径解析为实际文件系统访问。若路径规范化与边界检查不足，则恶意构造的 kustomization.yaml 可能触发路径穿越，使控制器读取超出预期工作目录的文件，从而造成敏感信息暴露，并在多租户部署中形成隔离边界破坏的风险。

CVE 描述：CVE-2022-24877 是 Flux kustomize-controller 的路径穿越漏洞。攻击者可通过恶意 kustomization.yaml 触发控制器读取其 Pod 文件系统中的非预期文件，从而暴露敏感数据；在多租户部署下，该能力可能进一步放大为权限提升风险。该漏洞已在 kustomize-controller v0.24.0 中修复，并随 flux2 v0.29.0 集成发布。工程侧常见缓解手段包括在 CI/CD 流水线中自动校验 kustomization.yaml 是否符合安全策略。

主要成因分类：输入校验缺陷 / 路径穿越 (Path Traversal, 路径规范化与目录边界检查不足)。

攻击链条描述：该漏洞可抽象为“仓库内容被信任 → 路径解析越界 → 敏感数据外泄”的链条：攻击者将恶意 kustomization.yaml 提交至可被控制器同步的仓库；控制器在构建阶段解析并访问其中指定的路径；若路径可越界访问控制器容器文件系统，则非预期文件内容可能被注入到构建产物、事件输出或后续流程中，造成敏感信息暴露，并在多租户环境中破坏隔离边界。

漏洞复现 (验证性复现)

本案例复现采用**验证性复现**思路，重点验证控制器是否会在构建过程中读取超出预期目录边界的文件，并以构建日志与产物行为作为证据，复现的观测点包括：控制器在构建阶段出现与异常路径解析相关的错误或告警信息，或在严格受控的测试条件下出现“构建产物包含非预期内容”的迹象。为避免涉及敏感文件读取的可滥用细节，正文仅给出复现边界与证据判据。完整

的环境准备、受控验证步骤与证据判据见附录F。

6.6.2 策略部署

本节给出针对该类“路径解析越界导致的数据暴露”的优先级缓解策略。治理思路以前置校验与 GitOps 变更治理为核心，用于降低多租户场景下的输入面风险。

策略概述：第一，在 CI/CD 或代码评审环节对 `kustomization.yaml` 进行策略化校验，阻断异常路径引用进入仓库。第二，在多租户环境中收敛对 Git 仓库写入与 Flux 资源变更的权限。第三，通过运行时硬化与最小权限配置降低异常路径被利用后的后果。

策略配置与部署步骤：为避免正文堆叠大量清单，CI 校验示例、权限收敛要点与验证建议见附录F。

6.6.3 三因素分析

有效性 (Effectiveness)：CI 校验用于在提交阶段阻断异常路径引用进入仓库，显著降低触发概率。权限与变更治理用于限制恶意 `kustomization.yaml` 进入同步面，运行时硬化与最小权限配置则在多租户场景下缩小爆炸半径。

无干扰性 (Interference)：CI 校验可能对历史仓库中不规范写法产生阻断，应采用先告警后强制的方式灰度推进，并提供例外与审批流程。多租户下的权限收敛可能影响自助交付效率，需要以标准化模板与可审计的变更机制替代高权限直写。运行时硬化通常对业务逻辑影响较小，但需验证与现有控制器权限、文件访问行为及构建流程的兼容性。

可部署性 (Deployability)：上述措施以工程流程与平台能力为主，便于标准化推广：CI 校验可作为流水线质量门禁复用；多租户权限收敛与运行时硬化可通过 GitOps 模板化交付，形成持续治理闭环。

6.7 案例六：CVE-2022-29171 (Sourcegraph gitserver 远程代码执行)

6.7.1 漏洞详解与复现

漏洞背景

Sourcegraph 是面向大规模代码仓库的搜索与导航平台，其典型部署包含

负责仓库拉取与 Git 操作的 `gitserver` 服务。在企业集成场景中，Sourcegraph 可接入多种代码托管与协作系统。为获得额外元数据，部分集成允许管理员在系统中配置外部命令，由后端服务在特定路径下执行并返回结果。此类机制具有天然的高风险：若命令可被高权限用户任意改写，且执行环境与网络边界缺少隔离，则可能形成配置驱动的命令执行入口。

CVE 描述：CVE-2022-29171 指出 Sourcegraph `gitserver` 在 Gitolite 与 Phabricator 联动集成中存在远程代码执行风险。在受影响版本(< 3.38.0)中，站点管理员可配置用于获取 Phabricator 元数据的 `callsignCommand`；若攻击者同时具备对 Sourcegraph 实例的站点管理员权限，以及对其内置 Grafana 监控实例的管理员权限，则可进一步获取内部服务地址信息，并将上述命令改写为任意系统命令，从而在 `gitserver` 上触发远程执行。该问题已在 3.38.0 版本修复，官方仍建议即使采取临时缓解也应尽快升级。

主要成因分类：安全逻辑缺陷 / 命令注入（配置驱动的命令执行缺乏约束）与权限边界缺陷（高敏感配置权限与观测面权限向执行面耦合）。

攻击链条描述：该漏洞可抽象为“高权限配置 → 执行面触达 → 基础设施控制”的链条。攻击者首先获得可编辑或新增 Gitolite `code host` 的管理权限；随后借助 Grafana 管理权限获取 `gitserver` 的内部寻址信息；最终改写 `callsignCommand` 并触发执行。该链条反映了管理面配置、观测面信息与执行面能力之间的耦合风险。

漏洞复现（验证性复现）

本文采用**验证性复现**思路，在隔离环境中验证外部命令执行风险。复现不提供可复用攻击载荷，以审计日志与任务执行记录为主：在受影响版本中，若可观察到 `gitserver` 因配置项变更而执行外部命令的迹象，则可判定存在执行面风险；升级至修复版本或禁用相关集成后，该迹象应消失。完整环境准备、验证判据与回归检查步骤见附录G。

6.7.2 策略部署

本案例的治理重点是**切断命令执行入口并隔离执行面与观测面**。考虑到攻击链对权限前置条件要求较高，治理目标应落在降低误授权限后的可达性与影响范围，避免配置权限与观测面权限共同指向执行面。

策略概述：第一，若业务不依赖 Gitolite 集成，禁用或移除 Gitolite `code host`，消除 `callsignCommand` 对应的输入面。第二，对 `gitserver` 实施网络

隔离，采用默认拒绝并按需放通的方式收敛可达面，降低内部寻址信息被利用后的横向触达能力。第三，进行权限分离与最小化，严格限制外部服务配置的变更主体，并收敛 Grafana 的暴露面与管理员权限，降低观测面泄露内部拓扑的风险。策略配置要点与验证建议见附录G。

上述措施可形成分层防护：功能禁用用于收敛输入面，网络隔离与权限最小化用于限制影响范围。落地时应将相关功能开关、网络策略与权限配置纳入基线，并在变更后复测关键判据，避免回退或旁路路径导致风险回归。

6.7.3 三因素分析

有效性 (Effectiveness)：禁用 Gitolite 集成可移除关键输入面，适用于不依赖该功能的部署。网络隔离与权限最小化属于纵深防御：即使出现站点管理员权限误授或观测面越权，也能显著提高攻击链的达成难度并降低爆炸半径。

无干扰性 (Interference)：禁用 Gitolite 会影响依赖该集成的仓库同步与元数据功能，需要评估替代接入方式。网络隔离需要精确梳理 gitserver 的入站/出站依赖，初期可能带来误拦截风险，建议先在预生产灰度并配套可观测性。权限分离会提升变更门槛，但对高敏感配置属于必要治理。

可部署性 (Deployability)：功能禁用多数部署中可通过标准化配置管理实现。NetworkPolicy 等网络隔离能力依赖集群 CNI 支持，但在 Kubernetes 环境中易于模板化推广。权限分离与 Grafana 暴露面治理属于组织流程与平台基线配置，适合纳入 GitOps 基线与审计机制形成持续治理闭环。

6.8 案例七：CVE-2023-22482 (Argo CD OIDC aud 未校验导致越权访问)

6.8.1 漏洞详解与复现

漏洞背景

Argo CD 是 Kubernetes 场景中常用的声明式 GitOps 持续交付工具，提供 Web 与 API 接口供用户进行应用同步、回滚与差异审计等操作。在企业环境中，Argo CD 常通过 OIDC 与统一身份提供方对接，由身份提供方为 Argo CD 签发包含用户身份与组信息的令牌。在 OIDC 令牌中，aud (audience) 声明用于标识该令牌的预期接收方。通常情况下，服务端除验证签名与发行者外，还需要校验 aud 是否包含本服务的受众标识，以避免将“为其他服务签发的

令牌”误用到本服务上。

CVE 描述: CVE-2023-22482 指出 Argo CD 在部分版本中存在不恰当授权缺陷。其能够验证令牌由已配置的 OIDC 提供方签名,但未对令牌的 `aud` 声明进行校验,从而可能接受并非面向 Argo CD 的有效令牌。若同一身份提供方还为其他系统签发令牌,攻击者一旦获得其他系统的有效令牌,就可能将其用于访问 Argo CD。Argo CD 将基于令牌中的 `groups` 声明授予权限,即使这些组信息并非为 Argo CD 的授权场景设计。该问题在 2.6.0-rc3、2.5.6、2.4.19 与 2.3.13 及以上版本修复;受影响范围包含 $\geq 1.8.2$ 且低于上述修复版本的版本分支。

主要成因分类: 安全逻辑缺陷 / 身份与授权校验缺失 (令牌受众 `aud` 未校验导致跨受众令牌复用)。

攻击链条描述: 该漏洞可抽象为”多受众身份提供 → 跨受众令牌复用 → 基于组声明的越权访问”的链条。攻击者获得同一 OIDC 提供方为其他受众签发的有效令牌;将其提交给 Argo CD 的接口;若 Argo CD 未校验 `aud`,则会接受该令牌并根据 `groups` 映射授予权限,进而造成对应用资源与交付流程的非预期访问。该缺陷同时放大了令牌泄露事件的影响范围。

漏洞复现 (验证性复现)

本文采用**验证性复现**思路,验证“`aud` 不匹配的有效令牌是否会被 Argo CD 接受”这一关键安全性质。复现过程不展开可直接复用的令牌获取与请求构造细节,而以配置证据、令牌声明证据与接口返回结果为主:在受影响版本中,若 Argo CD 在已验签前提下接受 `aud` 不匹配的令牌并返回受 RBAC 控制的资源,则可判定存在受众校验缺失;升级至修复版本后,同样令牌应在校验阶段被拒绝。完整的环境准备、验证判据与回归检查步骤见附录H。

```
luxian@MAIN:~$ curl -sk -H "Authorization: Bearer ${ID_TOKEN:-<id_token_with_wrong_aud>}" https://argocd.example.com/api/v1/applications
HTTP/1.1 200 OK
{
  "items": []
}
luxian@MAIN:~$
```

图 6-6: 验证性复现证据示意 (审计/接口响应截图的经处理摘要)

Fig. 6-6: Illustration of validation-oriented reproduction evidence based on processed summaries of audit and API response screenshots

6.8.2 策略部署

该漏洞的治理重点是**降低跨系统令牌复用的可行性与后果**。在补丁窗口期内，可通过身份提供方配置、前置网关校验与 RBAC 收敛构建补偿控制，压制攻击路径并降低风险暴露面。

策略概述：第一，在身份提供方侧为 Argo CD 配置独立客户端与唯一受众，限制该客户端签发的组声明范围，避免与其他系统的组命名空间混用。第二，在 Argo CD 前置入口实施令牌受众强制校验，例如通过 API 网关对 iss、aud 与签名进行一致性验证，拒绝 aud 不匹配的请求。第三，收敛 Argo CD 的 RBAC 与项目边界，限制高权限组的授予范围，并将关键操作纳入审计与告警，降低令牌误用后的爆炸半径。策略配置要点与验证建议见附录H。

6.8.3 三因素分析

有效性 (Effectiveness)：身份提供方侧的客户端隔离可减少可被复用的令牌来源，前置网关的 aud 强制校验可在应用层之前阻断异常令牌进入。RBAC 与项目边界收敛则用于限制误用令牌的权限上限，三者形成互补的纵深防御。

无干扰性 (Interference)：前置网关校验对访问路径有一定侵入性，需评估对 CLI、Web 与自动化流水线的影响并提供灰度策略。身份提供方的客户端隔离与组声明裁剪可能影响既有用户组映射，需要配套变更沟通与回滚预案。RBAC 收敛可能降低自助交付效率，应通过标准化角色与审批流程平衡安全与效率。

可部署性 (Deployability)：身份提供方侧配置与网关校验可作为平台能力复用，在多集群场景中可模板化推广。RBAC 与项目边界策略可通过 GitOps 管理并持续审计，形成长期的基线治理闭环。

6.9 小节

通过本章的分析，本文所提多智能体协同的安全策略生成框架能够有效应对多种云原生环境中的权限提升漏洞。通过对典型 CVE 案例的复现与分析，验证了自动生成的**监控策略**与**防护策略**在真实攻击场景中的有效性与适用性，为云原生应用的安全防护提供了有力支撑。

7 结论与展望

7.1 总结

随着云原生技术在企业生产环境中的广泛应用，权限提升漏洞在补丁窗口期内带来的安全风险日益突出。本文围绕这一问题，提出了基于大语言模型的多智能体协同漏洞策略化防控框架，将漏洞语义理解、安全知识检索、策略生成与质量评估有机融合，实现了从 CVE 公告到可执行防控策略的全流程自动化。通过在 55 个真实 CVE 样本上的实验验证，该框架能够有效生成高质量的监控策略与防护策略，为补丁窗口期内的安全防控提供了切实可行的技术方案。

本文的主要贡献如下：

(1) 提出了监控策略与防护策略的两分类体系及四元组形式化表示，建立了基于排名的三维质量评价机制与多评审主体共识聚合模型，为策略生成与质量度量提供了统一的理论基础。

(2) 构建了面向防控的云原生权限提升漏洞两级成因分类框架与多源安全知识库，设计了知识检索 → 威胁分析 → 策略生成 → 评审反馈四阶段多智能体协同生成流水线，采用检索增强（RAG）、思维链推理（CoT）与反馈优化（Feedback）相结合的技术路线实现可控生成，并通过静态评估与动态验证相结合的两阶段质量控制保障策略产出质量。

(3) 设计并实现了 KPEAgent 系统原型，涵盖 CVE 知识库管理、智能策略生成、策略优化与反馈、专家评审与排序以及策略部署管理等核心功能模块，将前述方法论落地为面向安全工程师与专家评审员的可交互工具，支撑从漏洞分析到策略部署的完整闭环。

(4) 在 55 个云原生权限提升漏洞样本上开展了系统性实验，结果表明本框架中各 workflow 模块的正向贡献。同时，选取覆盖访问控制缺陷、冗余权限、命令注入、身份验证绕过等多种权限提升漏洞类型中 7 个典型 CVE 案例，通过漏洞复现、策略生成与三维质评估，进一步验证了框架在真实漏洞场景下的实际应用效果。

本文构建了一个从理论建模到工程落地的完整防控方案，为安全团队在补丁尚未就绪阶段提供了及时、有效的临时防控能力，也为生成式 AI 在安全

工程中的应用探索了可行路径。

7.2 展望

本文工作已取得初步成效，但仍有进一步完善与扩展的空间。针对现有框架的局限性，提出以下工作展望：

（1）**高危漏洞的自动化监控与知识库实时更新**。当前知识库采用离线批量构建方式，无法及时响应新披露漏洞。未来可对 GitHub Security Advisory、NVD 等公开数据源建立持续监控与订阅机制，实现增量知识库更新与策略重评估的自动化触发，缩短从漏洞披露到策略交付的响应时间。

（2）**漏洞利用的自动化复现与验证**。当前策略验证主要依赖人工模拟或静态推理，缺乏真实利用环境下的自动化验证能力。未来可基于漏洞披露信息自动搭建包含漏洞的 Kubernetes 测试集群，结合 PoC 代码执行攻击重放，量化评估策略的阻断成功率与误报率，为策略质量评价提供更客观的证据支撑。

（3）**研究范围向其他漏洞类型扩展**。本文聚焦于权限提升漏洞，未来可将方法体系扩展至拒绝服务漏洞、供应链与镜像安全漏洞等云原生环境中的其他高危类型，通过扩展分类框架与优化评价维度，构建更全面的智能化安全防控能力。

通过上述工作的持续推进，可推动云原生安全防控从被动应对向主动防御、从单点防护向体系化防御的演进。

参考文献

- [1] 吴逸文, 张洋, and 王涛 and 王怀民. 从 docker 容器看容器技术的发展: 一种系统文献综述的视角. 软件学报, 34(12):5527–5551, 2023.
- [2] Kubernetes. Kubernetes documentation. Available online: <https://kubernetes.io/docs/home/>, 2024. Accessed: November 18, 2024.
- [3] Alibaba Cloud. Alibaba container service: From serverless containers to serverless kubernetes. Available online: https://www.alibabacloud.com/blog/from-serverless-containers-to-serverless-kubernetes_596533, 2020. Accessed: November 18, 2024.
- [4] Adrienn Lawson and Jeffrey Sica. Cncf annual cloud native survey: The infrastructure of ai’s future. Technical report, The Linux Foundation, January 2026. URL <https://www.cncf.io/reports/the-cncf-annual-cloud-native-survey/>. Foreword by Jonathan Bryce.
- [5] David Ferraiolo, Janet Cugini, D Richard Kuhn, et al. Role-based access control (rbac): Features and motivations. In *Proceedings of the 11th Annual Computer Security Applications Conference*, pages 241–48, 1995.
- [6] Nanzi Yang, Wenbo Shen, Jinku Li, Xunqi Liu, Xin Guo, and Jianfeng Ma. Take over the whole cluster: Attacking kubernetes via excessive permissions of third-party applications. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security, CCS ’23*, page 3048–3062, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700507. doi: 10.1145/3576915.3623121. URL <https://doi.org/10.1145/3576915.3623121>.
- [7] Chang-ai Sun, Xiaoyang Han, and Yufei Gong. Kubeguard: A systematic permission-oriented risk detection approach for kubernetes applications. In *2025 IEEE International Conference on Web Services (ICWS)*, pages 855–865, July 2025. doi: 10.1109/ICWS67624.2025.00111.

- [8] Shazibul Islam Shamim, Hanyang Hu, and Akond Rahman. On prescription or off prescription? an empirical study of community-prescribed security configurations for kubernetes. In *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2432–2444, April 2025. doi: 10.1109/ICSE55347.2025.00170.
- [9] Akond Rahman, Shazibul Islam Shamim, Dibyendu Brinto Bose, and Rahul Pandita. Security misconfigurations in open source kubernetes manifests: An empirical study. *ACM Trans. Softw. Eng. Methodol.*, 32(4), May 2023. ISSN 1049-331X. doi: 10.1145/3579639. URL <https://doi.org/10.1145/3579639>.
- [10] Falco (CNCF). Falco: Cloud native runtime security. Available online: <https://falco.org/>, 2025. Accessed: January 26, 2026.
- [11] Open Policy Agent (CNCF). Open policy agent: Policy-based control for cloud native environments. Available online: <https://www.openpolicyagent.org/>, 2025. Accessed: January 26, 2026.
- [12] Kyverno (CNCF). Kyverno: Kubernetes native policy management. Available online: <https://kyverno.io/>, 2025. Accessed: January 26, 2026.
- [13] Francesco Minna, Agathe Blaise, Filippo Rebecchi, Balakrishnan Chandrasekaran, and Fabio Massacci. Understanding the security implications of kubernetes networking. *IEEE Security & Privacy*, 19(5):46–56, Sep. 2021. ISSN 1558-4046. doi: 10.1109/MSEC.2021.3094726.
- [14] Gerald Budigiri, Christoph Baumann, Jan Tobias Mühlberg, Eddy Truyen, and Wouter Joosen. Network policies in kubernetes: Performance evaluation and security analysis. In *2021 Joint European Conference on Networks and Communications & 6G Summit (EuCNC/6G Summit)*, pages 407–412, June 2021. doi: 10.1109/EuCNC/6GSummit51104.2021.9482526.
- [15] Shazibul Islam Shamim, Hanyang Hu, and Akond Rahman. Dynamic application security testing for kubernetes deployment: An experience report from industry. In *Proceedings of the 33rd ACM International Conference on the*

- Foundations of Software Engineering*, FSE Companion '25, page 514–519, New York, NY, USA, 2025. Association for Computing Machinery. ISBN 9798400712760. doi: 10.1145/3696630.3728573. URL <https://doi.org/10.1145/3696630.3728573>.
- [16] Yue Gu, Xin Tan, Yuan Zhang, Siyan Gao, and Min Yang. Epscan: Automated detection of excessive rbac permissions in kubernetes applications. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 3199–3217, May 2025. doi: 10.1109/SP61157.2025.00011.
- [17] Nicholas Pecka, Lotfi Ben Othmane, and Altaz Valani. Privilege escalation attack scenarios on the devops pipeline within a kubernetes environment. In *Proceedings of the International Conference on Software and System Processes and International Conference on Global Software Engineering*, ICSSP '22, page 45 – 49, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450396745. doi: 10.1145/3529320.3529325. URL <https://doi.org/10.1145/3529320.3529325>.
- [18] Charles P. Wright, Jay Dave, Puja Gupta, Harikesavan Krishnan, David P. Quigley, Erez Zadok, and Mohammad Nayyer Zubair. Versatility and unix semantics in namespace unification. *ACM Trans. Storage*, 2(1):74 – 105, February 2006. ISSN 1553-3077. doi: 10.1145/1138041.1138045. URL <https://doi.org/10.1145/1138041.1138045>.
- [19] Abhishek Verma, Luis Pedrosa, Madhukar Korupolu, David Oppenheimer, Eric Tune, and John Wilkes. Large-scale cluster management at google with borg. In *Proceedings of the Tenth European Conference on Computer Systems*, EuroSys '15, page 1–17. ACM, April 2015. doi: 10.1145/2741948.2741964. URL <http://dx.doi.org/10.1145/2741948.2741964>.
- [20] NANCY R Mead. Computer security: Art and science. *IEEE Security & Privacy*, 1(3):14–14, 2003.
- [21] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. The express data path: Fast programmable packet processing in the operating system kernel.

In Proceedings of the 14th international conference on emerging networking experiments and technologies, pages 54–66, 2018.

- [22] Aqua Security. Trivy: A simple and comprehensive vulnerability scanner for containers. Available online: <https://github.com/aquasecurity/trivy>, 2025. Accessed: January 15, 2026.
- [23] Anchore. Gype: A vulnerability scanner for container images and filesystems. Available online: <https://github.com/anchore/gype>, 2025. Accessed: January 15, 2026.
- [24] Anchore. Syft: A cli tool and library for generating sboms from container images. Available online: <https://github.com/anchore/syft>, 2025. Accessed: January 15, 2026.
- [25] kube-linter. kube-linter: Static analysis for kubernetes yaml and helm charts. Available online: <https://github.com/stackrox/kube-linter>, 2025. Accessed: January 15, 2026.
- [26] Kubescape. Kubescape: Kubernetes security scanning and hardening tool. Available online: <https://github.com/kubescape/kubescape>, 2025. Accessed: January 15, 2026.
- [27] kube-score. kube-score: Kubernetes object analysis tool for best practices. Available online: <https://github.com/zegl/kube-score>, 2025. Accessed: January 15, 2026.
- [28] KubeSec. Kubesec: Security risk analysis for kubernetes resources. Available online: <https://kubesec.io/>, 2025. Accessed: January 15, 2026.
- [29] Fairwinds. Polaris: Kubernetes best-practices and configuration checks. Available online: <https://github.com/FairwindsOps/polaris>, 2025. Accessed: January 15, 2026.
- [30] Bridgecrew (Checkov). Checkov: Infrastructure as code (iac) static analysis tool. Available online: <https://github.com/bridgecrewio/checkov>, 2025. Accessed: January 15, 2026.

- [31] CyberArk. Kubiscan: Kubernetes risk assessment tool. Available online: <https://github.com/cyberark/KubiScan>, 2024. Accessed: January 15, 2026.
- [32] Appvia. Krane: Kubernetes rbac static analysis & visualisation tool. Available online: <https://github.com/appvia/krane>, 2024. Accessed: January 15, 2026.
- [33] Palo Alto Networks. Kubernetes privilege escalation: Excessive permissions in popular platforms. Available online: <https://www.paloaltonetworks.com/resources/whitepapers/kubernetes-privilege-escalation-excessive-permissions-in-popular-platforms>, 2022. Accessed: January 15, 2026.
- [34] J.H. Saltzer and M.D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [35] Greg O’Shea. Redundant access rights. *Computers & Security*, 14(4):323–348, 1995. ISSN 0167-4048.
- [36] Ian Molloy, Youngja Park, and Suresh Chari. Generative models for access control policies: applications to role mining over logs with attribution. In *Proceedings of the 17th ACM Symposium on Access Control Models and Technologies, SACMAT ’12*, page 45–56, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450312950. doi: 10.1145/2295136.2295145. URL <https://doi.org/10.1145/2295136.2295145>.
- [37] Matthew W. Sanders and Chuan Yue. Minimizing privilege assignment errors in cloud services. In *Proceedings of the Eighth ACM Conference on Data and Application Security and Privacy, CODASPY ’18*, page 2 – 12, New York, NY, USA, 2018. Association for Computing Machinery. ISBN 9781450356329. doi: 10.1145/3176258.3176307. URL <https://doi.org/10.1145/3176258.3176307>.
- [38] Lucas Davi, Alexandra Dmitrienko, Ahmad-Reza Sadeghi, and Marcel

- Winandy. Privilege escalation attacks on android. In *Information Security Conference*, 2010.
- [39] S. Hutchinson and C. Varol. A survey of privilege escalation detection in android. In *2018 9th IEEE Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*, 2018.
- [40] Daoyuan Wu, Debin Gao, Yingjiu Li, and Robert H Deng. Seccomp: Towards practically defending against component hijacking in android applications. In *Proceedings of the 32nd Annual Conference on Computer Security Applications (ACSAC)*, 2016.
- [41] Mohamed A. El-Zawawy and Aya Hamdy. Detection of hidden privilege escalations in android. *Automated Software Engineering*, 32(2), 2025.
- [42] Vincent F. Taylor, Alastair R. Beresford, and Ivan Martinovic. Intra-library collusion: A potential privacy nightmare on smartphones. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2017.
- [43] Kathy Wain Yee Au, Yi Fan Zhou, Zhen Huang, and David Lie. Pscout: analyzing the android permission specification. In *Proceedings of the 2012 ACM Conference on Computer and Communications Security, CCS '12*, page 217 – 228, New York, NY, USA, 2012. Association for Computing Machinery. ISBN 9781450316514. doi: 10.1145/2382196.2382222. URL <https://doi.org/10.1145/2382196.2382222>.
- [44] Hamid Bagheri, Alireza Sadeghi, Joshua Garcia, and Sam Malek. Deldroid: An automated approach for determination and enforcement of least-privilege architecture in android. *Journal of Systems and Software*, 2019.
- [45] Yang Xu, Guojun Wang, Ju Ren, and Yaoxue Zhang. An adaptive and configurable protection framework against android privilege escalation threats. *Future Generation Computer Systems*, 92:210–224, 2019.
- [46] Ahmed K. H. Hussain, M. Kakavand, and M. Silval. A novel android security

- framework to prevent privilege escalation attacks. *International Journal of Computer Network and Information Security (IJCNIS)*, 12(1):20–26, 2020.
- [47] S. Sharmeen, S. Huda, J. H. Abawajy, and M. M. Hassan. An adaptive framework against android privilege escalation threats using deep learning and semi-supervised approaches. *Applied Soft Computing*, 2020.
- [48] Omer Tripp, Marco Pistoia, Stephen J. Fink, Manu Sridharan, and Omri Weisman. Taj: effective taint analysis of web applications. In *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI 2009)*, page 87–97, New York, NY, USA, 2009. ISBN 9781605583921. doi: 10.1145/1542476.1542486.
- [49] William Enck, Peter Gilbert, Seungyeop Han, Vasant Tendulkar, Byung-Gon Chun, Landon P. Cox, Jaeyeon Jung, Patrick McDaniel, and Anmol N. Sheth. Taintdroid: An information-flow tracking system for realtime privacy monitoring on smartphones. *ACM Transactions on Computer Systems*, 32(2):1–29, June 2014. ISSN 0734-2071. doi: 10.1145/2619091.
- [50] Wei You, Bin Liang, Wenchang Shi, Peng Wang, and Xiangyu Zhang. Taintman: An art-compatible dynamic taint analysis framework on unmodified and non-rooted android devices. *IEEE Transactions on Dependable and Secure Computing*, 17(1):209–222, 2020. doi: 10.1109/TDSC.2017.2740169.
- [51] J. Zhang, L. Yang, Y. Han, et al. A small leak will sink many ships: Vulnerabilities related to mini-programs permissions. In *2023 IEEE 47th Annual Computers, Software, and Applications Conference (COMPSAC)*, pages 595–606. IEEE, 2023.
- [52] Yin Wang, Ming Fan, Hao Zhou, Haijun Wang, Wuxia Jin, Jiajia Li, Wenbo Chen, Shijie Li, Yu Zhang, Deqiang Han, and Ting Liu. Minichecker: Detecting data privacy risk of abusive permission request behavior in mini-programs. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering, ASE '24*, page 1667–1679, New York, NY, USA, 2024. Association for Computing Machinery. ISBN 9798400712487. doi: 10.1145/3691620.3695534. URL <https://doi.org/10.1145/3691620.3695534>.

- [53] Md. Shazibul Islam Shamim, Farzana Ahamed Bhuiyan, and Akond Rahman. Xi commandments of kubernetes security: A systematization of knowledge related to kubernetes security practices. In *2020 IEEE Secure Development (SecDev)*, pages 58–64, 2020. doi: 10.1109/SecDev45635.2020.00025.
- [54] Shazibul Islam Shamim. Mitigating security attacks in kubernetes manifests for security best practices violation. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ESEC/FSE 2021, page 1689–1690, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450385626. doi: 10.1145/3468264.3473495. URL <https://doi.org/10.1145/3468264.3473495>.
- [55] J. Alexander Curtis and Nasir U. Eisty. The kubernetes security landscape: Ai-driven insights from developer discussions. In *2025 IEEE/ACIS 23rd International Conference on Software Engineering Research, Management and Applications (SERA)*, pages 179–186, May 2025. doi: 10.1109/SERA65747.2025.11154503.
- [56] Ahmed Zerouali, Ruben Opdebeeck, and Coen De Roover. Helm charts for kubernetes applications: Evolution, outdatedness and security risks. In *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, pages 523–533, May 2023. doi: 10.1109/MSR59073.2023.00078.
- [57] Agathe Blaise and Filippo Rebecchi. Stay at the helm: secure kubernetes deployments via graph generation and attack reconstruction. In *2022 IEEE 15th International Conference on Cloud Computing (CLOUD)*, pages 59–69, July 2022. doi: 10.1109/CLOUD55607.2022.00022.
- [58] Francesco Minna, Fabio Massacci, and Katja Tuma. Analyzing and mitigating (with llms) the security misconfigurations of helm charts from artifact hub. *Empirical Software Engineering*, 30(5):132, 2025.
- [59] Mubin Ul Haque, M. Mehdi Kholoosi, and M. Ali Babar. Kgseconfig: A knowledge graph based approach for secured container orchestrator configuration. In *2022 IEEE International Conference on Software Analysis*,

- Evolution and Reengineering (SANER)*, pages 420–431, March 2022. doi: 10.1109/SANER53432.2022.00057.
- [60] Giorgio Dell’ Immagine, Jacopo Soldani, and Antonio Brogi. Kubehound: Detecting microservices’ security smells in kubernetes deployments. *Future Internet*, 15(7), 2023. ISSN 1999-5903. doi: 10.3390/fi15070228. URL <https://www.mdpi.com/1999-5903/15/7/228>.
- [61] Bom Kim, Jinwoo Kim, and Seungsoo Lee. Exploring security enhancements in kubernetes cni: A deep dive into network policies. *IEEE Access*, 13:35322–35338, 2025. ISSN 2169-3536. doi: 10.1109/ACCESS.2025.3543841.
- [62] Carmine Cesarano and Roberto Natella. Kubefence: Security hardening of the kubernetes attack surface. *arXiv preprint arXiv:2504.11126*, 2025.
- [63] Hui Zhu and Christian Gehrman. Kub-sec, an automatic kubernetes cluster apparmor profile generation engine. In *2022 14th International Conference on COMMunication Systems & NETWORKS (COMSNETS)*, pages 129–137, Jan 2022. doi: 10.1109/COMSNETS53615.2022.9668504.
- [64] Michael V. Le, Salman Ahmed, Dan Williams, and Hani Jamjoom. Securing container-based clouds with syscall-aware scheduling. In *Proceedings of the 2023 ACM Asia Conference on Computer and Communications Security*, ASIA CCS ’23, page 812 – 826, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400700989. doi: 10.1145/3579856.3582835. URL <https://doi.org/10.1145/3579856.3582835>.
- [65] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.
- [66] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. Mpnet: Masked and permuted pre-training for language understanding. *Advances in Neural Information Processing Systems*, 33:16857–16867, 2020.

- [67] Matthijs Douze et al. The faiss library. *arXiv preprint arXiv:2401.08281*, 2024.
- [68] Xuansheng Wu, Padmaja Pravin Saraf, Gyeonggeon Lee, Ehsan Latif, Ninghao Liu, and Xiaoming Zhai. Unveiling scoring processes: Dissecting the differences between llms and human graders in automatic scoring. *Technology, Knowledge and Learning*, pages 1–16, 2025.
- [69] Peiyi Wang, Lei Li, et al. Large language models are not fair evaluators. *arXiv preprint arXiv:2305.17926*, 2023.
- [70] Yerin Hwang, Dongryeol Lee, Taegwan Kang, Yongil Kim, and Kyomin Jung. Can you trick the grader? adversarial persuasion of llm judges. *arXiv preprint arXiv:2508.07805*, 2025.
- [71] Jorge Velez et al. Evaluating large language models as raters in large-scale writing assessments. *Learning and Instruction*, 2025.
- [72] Yang Liu et al. G-eval: Nlg evaluation using gpt-4 with better human alignment. *arXiv preprint arXiv:2303.16634*, 2023.
- [73] Yidong Wang, Yunze Song, Tingyuan Zhu, Xuanwang Zhang, Zhuohao Yu, Hao Chen, Chiyu Song, Qiufeng Wang, Cunxiang Wang, Zhen Wu, et al. Trustjudge: Inconsistencies of llm-as-a-judge and how to alleviate them. *arXiv preprint arXiv:2509.21117*, 2025.

附录 A 云原生权限提升漏洞分类与 CWE 对应关系

表 A-1: 云原生权限提升漏洞分类框架与 CWE 对应关系

Tab. A-1: Mapping between the classification framework of cloud-native privilege escalation vulnerabilities and CWE

一级分类	二级分类	主要 CWE	CWE 描述
配置缺陷	冗余权限	CWE-269	不当访问控制（通用型）
		CWE-266	权限分配错误
		CWE-732	关键资源权限分配错误
	网络隔离不足	CWE-653	隔离措施不足
		CWE-668	资源暴露至错误安全域
		CWE-669	资源在不同安全域间转移错误
安全逻辑缺陷	访问控制缺陷	CWE-863	授权验证错误
		CWE-284	不当访问控制
		CWE-862	缺失授权机制
	身份验证绕过	CWE-287	不当身份验证
		CWE-290	硬编码密码进行身份验证
		CWE-327	使用存在缺陷或高风险的加密算法
	输入验证不足	CWE-20	不当输入验证
		CWE-74	输出内容中特殊元素的中和处理不当
		CWE-79	Web 页面生成过程中输入的中和处理不当
	授权验证不完善	CWE-285	授权验证不当
CWE-250		以非必要权限执行操作	
CWE-268		特权操作执行不当	
敏感信息泄露	API 端口暴露	CWE-200	敏感信息泄露给未授权主体
		CWE-358	动态识别资源的操作限制不当
		CWE-276	默认权限配置错误
	日志泄露	CWE-532	敏感信息写入日志文件
		CWE-214	存储/传输前敏感信息清除不当
		CWE-497	敏感系统信息泄露至未授权控制域
	配置文件泄露	CWE-922	敏感信息存储不安全
		CWE-200	敏感信息泄露给未授权主体
		CWE-270	权限上下文切换错误
代码注入	命令注入	CWE-78	操作系统命令中特殊元素的中和处理不当
		CWE-94	代码生成控制不当
		CWE-601	URL 重定向至不可信站点
	路径遍历	CWE-22	路径名限制不当，未限定在指定目录
		CWE-36	绝对路径遍历
		CWE-74	输出内容中特殊元素的中和处理不当

A.1 KubeGuard 权限提升检测规则汇总

为便于正文聚焦方法主线，本节补充给出 KubeGuard 在四类权限提升场景下采用的代表性检测规则汇总。

表 A-2: 多种权限提升类型及其检测规则
Tab. A-2: Types of privilege escalation and their detection rules

类型	机制	检测规则
凭证窃取	密钥访问	检测请求的源 IP 地址或服务账户与资源所属 Pod 不一致的 API 请求
	Pod 执行	
	容器逃逸	
账户冒用	服务账户模拟	检测服务账户、操作和资源的组合是否属于该账户的正常使用行为集合
RBAC 操纵	角色修改	检测对角色或角色绑定资源的创建、修补或更新操作
	角色绑定修改	
间接执行	工作负载修改	检测对工作负载资源的创建、修补或更新操作，以及对 Pod 的 exec 操作
	Pod 执行	

附录 B CVE202224768 复现步骤与策略清单

本附录给出案例一的复现详细步骤与纵深防御策略代码清单，用于复核与工程复现。为降低对生产环境的风险，建议仅在隔离测试集群中执行。

B.1 漏洞复现详细步骤

步骤 1：部署漏洞版本 Argo CD (v2.3.1)

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argocd/v2.3.1/manifests/install.yaml
kubectl rollout status deployment/argocd-server -n argocd
```

步骤 2：获取初始管理员密码并登录 UI/CLI

```
kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath={.data.password} | base64 -d && echo
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

步骤 3：创建宽松的 AppProject（用于演示风险）

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: demo-project
  namespace: argocd
spec:
  destinations:
  - namespace: '*'
    server: '*'
  sourceRepos:
  - '*'
  clusterResourceWhitelist:
  - group: '*'
    kind: '*'
EOF
```

步骤 4: 创建低权用户 bob，并授予其对目标 Application 的 sync 与 override 权限在 argocd-rbac-cm 中添加如下策略（注意缩进）:

```
policy.default: role:readonly
policy.csv: |
  p, role:syncer, applications, sync, demo-project/*, allow
  p, role:syncer, applications, override, demo-project/*, allow
  g, bob, role:syncer
```

并在 argocd-cm 中开启账户:

```
accounts.bob: apiKey, login
```

随后重启 Argo CD Server 使配置生效:

```
kubectl rollout restart deployment/argocd-server -n argocd
```

步骤 5: 创建 Application 指向恶意仓库

```
cat <<EOF | kubectl apply -f -
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: bob-app
  namespace: argocd
spec:
  project: demo-project
  source:
    repoURL: https://github.com/Ivanbeethoven/bad_repo
    path: .
    targetRevision: HEAD
  destination:
    server: https://kubernetes.default.svc
    namespace: default
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
EOF
```

步骤 6: 模拟 bob 触发同步（或依赖自动同步）

```
argocd login localhost:8080 --username bob --password
↪ <token-or-password>
argocd app sync bob-app --force
```

步骤 7: 验证是否权限提升成功

```
kubectl auth can-i '*' '*' --as bob
```

若输出为 yes, 说明 bob 获得了集群范围的高权限 (该结果用于说明风险, 生产环境严禁在真实集群中进行)。

B.2 策略代码清单

B.2.1 Kubernetes RBAC: 收敛 AppProject 的写权限

将 appprojects.argoproj.io 的创建/更新/删除限制在平台管理员组, 租户仅可只读 (示例中组名可按实际替换):

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: argocd-appproject-edit
rules:
- apiGroups: ["argoproj.io"]
  resources: ["appprojects"]
  verbs: ["get", "list", "watch", "create", "update", "patch", "delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: argocd-appproject-readonly
rules:
- apiGroups: ["argoproj.io"]
  resources: ["appprojects"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
```

```
    name: argocd-appproject-edit-platform-admins
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: argocd-appproject-edit
subjects:
- kind: Group
  name: argo:platform-admin
  apiGroup: rbac.authorization.k8s.io
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: argocd-appproject-readonly-tenants
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: argocd-appproject-readonly
subjects:
- kind: Group
  name: argo:tenant-readonly
  apiGroup: rbac.authorization.k8s.io
```

B.2.2 Argo CD RBAC: 默认只读 + 高风险能力隔离

建议将 `argocd-rbac-cm` 纳入 Git 基线管理, 并在修改后滚动重启 `argocd`

```
-server:

apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.csv: |
    g, argo:platform-admin, role:admin
    g, argo:tenant-readonly, role:readonly
```

```

p, role:admin, projects, *, *, allow
p, role:admin, repositories, *, *, allow
p, role:admin, clusters, *, *, allow
p, role:admin, applications, *, *, allow
p, role:admin, exec, *, *, allow
p, role:admin, logs, *, *, allow

p, role:readonly, applications, get, *, allow
p, role:readonly, repositories, get, *, allow
p, role:readonly, projects, get, *, allow
p, role:readonly, clusters, get, *, allow
p, role:readonly, logs, get, *, allow
p, role:readonly, exec, *, *, deny
policy.default: role:readonly

```

B.2.3 准入控制：Gatekeeper（推荐）

目标：在 AppProject 创建/更新时，对 `spec.roles[].policies[]` 进行一致性校验。本策略覆盖三类约束：

- 禁止出现 `*, *, allow` 的全局通配符放权；
- 要求 `policy subject` 绑定到项目域（`proj:<project>:<role>` 前缀）；
- 禁止在 `project` 内授予 `projects/repositories/clusters/exec` 等高危资源写权限。

（1）ConstraintTemplate

```

apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sargocdappprojectpolicies
spec:
  crd:
    spec:
      names:
        kind: K8sArgoCDAppProjectPolicies
      validation:
        openAPIV3Schema:
          properties:

```

```

        enforceProjPrefix:
            type: boolean
targets:
- target: admission.k8s.gatekeeper.sh
  rego: |
      package k8sargocdappprojectpolicies

      trim_space(s) = out {
          out := regex.replace("^\\s+|\\s+$", "", s)
      }

      # 1) 禁止出现 "..., *, *, allow" 的全局通配符放权
      deny[msg] {
          input.review.kind.group == "argoproj.io"
          input.review.kind.kind == "AppProject"
          roles := input.review.object.spec.roles
          some i
          policies := roles[i].policies
          some j
          policy := policies[j]
          regex.match(".*,\\s*\\s*,\\s*\\s*,\\s*allow\\s*$", policy)
          msg := sprintf("AppProject policy contains global wildcard
              ↪ allow: %s", [policy])
      }

      # 2) 要求 subject 采用 proj:<project>:<role> 前缀（可选开关）
      deny[msg] {
          input.review.kind.group == "argoproj.io"
          input.review.kind.kind == "AppProject"
          input.parameters.enforceProjPrefix == true
          ap := input.review.object.metadata.name

          roles := input.review.object.spec.roles
          some i
          policies := roles[i].policies
          some j
          policy := policies[j]

```

```

    startswith(policy, "p,")
    parts := split(policy, ",")
    count(parts) >= 2
    subject := trim_space(parts[1])
    not startswith(subject, sprintf("proj:%s:", [ap]))
    msg := sprintf("AppProject policy subject must start with
    ↪ 'proj:%s:' : %s", [ap, policy])
}

# 3) 禁止授予 projects/repositories/clusters/exec 的写权限
deny[msg] {
    input.review.kind.group == "argoproj.io"
    input.review.kind.kind == "AppProject"
    roles := input.review.object.spec.roles
    some i
    policies := roles[i].policies
    some j
    policy := policies[j]
    regex.match("^p,.*, (projects|repositories|clusters|exec),\\s*(
    ↪ create|update|patch|delete|\\*),.*, (allow)\\s*$", policy)
    msg := sprintf("AppProject policy grants write on global
    ↪ resource: %s", [policy])
}

```

(2) Constraint

```

apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sArgoCDAppProjectPolicies
metadata:
  name: restrict-argocd-appproject-policies
spec:
  match:
    kinds:
      - apiGroups: ["argoproj.io"]
        kinds: ["AppProject"]
  parameters:
    enforceProjPrefix: true

```

B.2.4 准入控制：Kyverno（替代方案）

若集群已使用 Kyverno，可用如下 ClusterPolicy 实现同类校验。建议先将 validationFailureAction 设为 Audit，稳定后再切换为 Enforce。

```
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: restrict-argocd-appproject-policies
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: forbid-global-wildcard-allow
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/AppProject
      validate:
        message: "forbid global wildcard allow in AppProject role
        ↪ policies"
        foreach:
          - list: "request.object.spec.roles[].policies[]"
            deny:
              conditions:
                any:
                  - key: "{{ regex_match('.*,\\s*\\s*,\\s*\\s*,\\s*allow\\s*$',
                  ↪ element) }}"
                  operator: Equals
                  value: true

    - name: enforce-proj-prefix
      match:
        any:
          - resources:
              kinds:
                - argoproj.io/v1alpha1/AppProject
```



```

validate:
  message: "policy subject must use proj:<project>:<role> prefix"
  foreach:
    - list: "request.object.spec.roles[].policies[]"
      deny:
        conditions:
          all:
            - key: "{{ regex_match('^p,\\s*proj:[^:]+:[^,]+,', element)
              ↪ }}"
              operator: Equals
              value: false

- name: forbid-global-resource-write
  match:
    any:
      - resources:
          kinds:
            - argoproj.io/v1alpha1/AppProject
  validate:
    message: "forbid write on projects/repositories/clusters/exec in
      ↪ AppProject policies"
    foreach:
      - list: "request.object.spec.roles[].policies[]"
        deny:
          conditions:
            any:
              - key: "{{ regex_match('^p,.*(projects|repositories|clusters|
                ↪ exec),\\s*(create|update|patch|delete|\\s*),.*(all|
                ↪ ow)\\s*$', element) }}"
                operator: Equals
                value: true

```


附录 C CVE-2025-2241 复现步骤与策略清单

本附录给出案例二（CVE-2025-2241）的复现要点与缓解策略清单，用于复核与工程落地。该漏洞属于敏感信息泄露：vCenter 凭据出现在 Hive 的 ClusterProvision 对象可读字段中，导致“无 Secret 权限也可读到凭据”的旁路风险。请仅在隔离测试环境中操作。

C.1 漏洞复现详细步骤

前提条件

- 已部署 ACM/MCE 与 Hive，并启用 vSphere 集群供应能力（具体版本与安装方式依赖环境，以厂商公告与实验环境为准）；
- 至少存在一个“只读身份”。该身份不具备 secrets 读取权限，但具备 clusterprovisions.hive.openshift.io 的 get/list/watch 权限（该条件用于验证旁路泄露）。

步骤 1：触发一次 vSphere 集群供应通过 ACM 控制台或 YAML 创建 Hive 的 ClusterDeployment（示例仅展示关键字段，真实环境中密码应引用 Secret；本漏洞的问题在于后续对象中出现明文暴露）：

```
apiVersion: hive.openshift.io/v1
kind: ClusterDeployment
metadata:
  name: vsphere-cluster
  namespace: your-namespace
spec:
  baseDomain: example.com
  platform:
    vsphere:
      vCenter: vc.example.com
      username: administrator@vsphere.local
      password: your-password
  provisioning:
    installConfigSecretRef:
      name: your-install-config
    imageSetRef:
```

```
name: img4.14.0
```

步骤 2：用“可读 ClusterProvision 但不可读 Secret”的身份读取对象先确认权限面，再读取 ClusterProvision：

```
# 谁能读 clusterprovisions (OpenShift)
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision

# 低权用户自检 (Kubernetes 通用)
kubectl auth can-i get clusterprovision --as=<user> --all-namespaces
kubectl auth can-i get secrets --as=<user> --all-namespaces

# 导出对象 (名称可能带随机后缀, 按实际替换)
kubectl get clusterprovision -n your-namespace
kubectl get clusterprovision <name> -n your-namespace -o yaml
```

步骤 3：定位敏感字段在输出中查找类似结构（位置因版本实现而异，示例以 `status.vCenter.credentials` 为代表）：

```
status:
  vCenter:
    credentials:
      username: ...
      password: ...
```

若凭据以明文形式出现，即满足漏洞复现判据。

C.2 策略代码清单

本节策略以“收敛读取面 + 缩短暴露窗口 + 审计发现与轮换止血”为主线。由于敏感信息出现在控制器写入的 `status` 字段中，准入控制更适合作为审计/检测手段，不建议强制拦截控制器更新（否则可能影响供应流程）。

C.2.1 1) 立刻收敛 ClusterProvision 的读取权限 (RBAC)

风险盘点命令 (OCP)

```
oc adm policy who-can get clusterprovision
oc adm policy who-can list clusterprovision
oc adm policy who-can watch clusterprovision
```

最小可读权限示例（仅绑定 Hive 控制器服务账号）说明：Hive 控制器的具体 ServiceAccount/namespace 需按实际部署确认。

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: hive-clusterprovision-read-min
rules:
- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["get","list","watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: hive-clusterprovision-read-min-binding
subjects:
- kind: ServiceAccount
  name: hive-controllers
  namespace: hive
roleRef:
  kind: ClusterRole
  name: hive-clusterprovision-read-min
  apiGroup: rbac.authorization.k8s.io
```

C.2.2 2) 对读取行为启用审计并联动告警

Kubernetes 审计策略片段（加载方式依平台而定）：

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get","list","watch"]
  resources:
- group: "hive.openshift.io"
  resources: ["clusterprovisions"]
omitStages: ["RequestReceived"]
```

建议在 SIEM/日志平台中将 Hive 控制器主体加入白名单，仅对非白名单主体的读取行为告警。

C.2.3 3) 缩短暴露窗口：供应完成后清理 ClusterProvision

手动清理

```
oc get clusterprovision -n <ns>
oc delete clusterprovision -n <ns> <name>
```

自动化清理（示例 CronJob）说明：删除条件建议通过标签或状态字段精确匹配，避免误删；示例以标签 `hive.openshift.io/retain=false` 为条件。

```
apiVersion: v1
kind: ServiceAccount
metadata:
  name: cleanup-sa
  namespace: hive
---
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  name: cleanup-role
  namespace: hive
rules:
- apiGroups: ["hive.openshift.io"]
  resources: ["clusterprovisions"]
  verbs: ["list","get","delete"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: cleanup-rb
  namespace: hive
subjects:
- kind: ServiceAccount
  name: cleanup-sa
roleRef:
```

```

kind: Role
name: cleanup-role
apiGroup: rbac.authorization.k8s.io
---
apiVersion: batch/v1
kind: CronJob
metadata:
  name: cleanup-clusterprovision
  namespace: hive
spec:
  schedule: "0 * * * *"
  jobTemplate:
    spec:
      template:
        spec:
          serviceAccountName: cleanup-sa
          containers:
            - name: cleaner
              image: registry.access.redhat.com/ubi9/ubi:latest
              command: ["/bin/sh", "-c"]
              args:
                - |
                  set -eu
                  oc delete clusterprovision -n hive -l
                  ↪ hive.openshift.io/retain=false --ignore-not-found
          restartPolicy: OnFailure

```

C.2.4 4) vCenter 凭据轮换与强化（止血）

建议在确认泄露后立即轮换 vCenter 凭据，并启用最小权限、MFA、来源 IP 允许列表等约束。轮换完成后更新 Hive 使用的 Secret（示例命令需按实际 Secret 名称/namespace 调整）：

```

oc -n hive create secret generic vcenter-credentials \
  --from-literal=username=<new-user> \
  --from-literal=password=<new-pass> \
  --dry-run=client -o yaml | oc apply -f -

```

C.2.5 5) “敏感字段守护”检测（Gatekeeper/Kyverno, 建议审计模式）

Gatekeeper (dryrun)：检测 ClusterProvision 中是否出现敏感键名 (password/token/secret/credential)。

```
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sclustprovkeysensitive
spec:
  crd:
    spec:
      names:
        kind: K8sClustProvKeySensitive
  targets:
    - target: admission.k8s.gatekeeper.sh
      rego: |
        package k8sclustprovkeysensitive

        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "password")
        }
        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "token")
        }
        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "secret")
        }
        sensitive_key(k) {
          lk := lower(k)
          contains(lk, "credential")
        }

        # 深度遍历 object, 收集所有键
```



```

walk(obj, path, val) {
  is_object(obj)
  some k
  v := obj[k]
  walk(v, array.concat(path, [k]), val)
}

walk(obj, path, val) {
  is_array(obj)
  some i
  v := obj[i]
  walk(v, array.concat(path, [sprintf("%v", [i])]), val)
}

walk(obj, path, obj) {
  not is_object(obj)
  not is_array(obj)
}

deny[msg] {
  input.review.kind.group == "hive.openshift.io"
  input.review.kind.kind == "ClusterProvision"
  walk(input.review.object, [], _)
  some k
  _ = input.review.object[k]
  sensitive_key(k)
  msg := sprintf("ClusterProvision contains sensitive key at
    ↪ top-level: %s", [k])
}

deny[msg] {
  input.review.kind.group == "hive.openshift.io"
  input.review.kind.kind == "ClusterProvision"
  walk(input.review.object, path, _)
  some i
  k := path[i]
  sensitive_key(k)
  msg := sprintf("ClusterProvision contains sensitive key on path
    ↪ %v", [path])
}

```

```

    }
---
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sClustProvKeySensitive
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  enforcementAction: dryrun
  match:
    kinds:
      - apiGroups: ["hive.openshift.io"]
        kinds: ["ClusterProvision"]

```

Kyverno (Audit): 对对象文本进行检测，以提示可能存在的敏感键名。
该策略更适用于告警与排查，不建议直接用于阻断控制器更新。

```

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: audit-clusterprovision-sensitive-keys
spec:
  validationFailureAction: Audit
  background: true
  rules:
    - name: detect-secret-like-keys
      match:
        any:
          - resources:
              kinds:
                - hive.openshift.io/v1/ClusterProvision
      validate:
        message: ClusterProvision 可能包含敏感字段 password token secret
        ↪ credential。
      deny:
        conditions:
          any:
            - key: "{{ to_string(request.object) }}"
              operator: AnyIn

```

```
value:  
- password  
- token  
- secret  
- credential
```


附录 D CVE-2023-30512 复现步骤与策略清单

本附录给出案例三（CVE-2023-30512, CubeFS CSI 插件过度授权导致的集群权限提升风险）的**验证性复现与策略清单**。为避免提供可被滥用的细粒度利用操作，复现部分以“权限证据 + 受控权限验证”为主，重点证明 CSI ServiceAccount 被绑定了不必要的 secrets 读取能力；策略部分给出可落地的 RBAC 收敛、准入防回归与审计检测。

D.1 漏洞复现详细步骤

变量约定（按实际替换）

```
NAMESPACE=<cubeFS-namespace>
SA=cfs-csi-service-account
CLUSTERROLE=cfs-csi-cluster-role
```

如 SA 名称不同，请先定位：

```
kubectl get sa -A | findstr /i "csi cfs cubeFS"
kubectl get clusterrole,clusterrolebinding -A | findstr /i "csi cfs
↪ cubeFS"
```

步骤 1：导出 RBAC 证据（确认 secrets 权限存在）

```
kubectl get clusterrole $CLUSTERROLE -o yaml
kubectl get clusterrolebinding -o yaml | findstr /i "cfs-csi cubeFS"
```

重点检查 ClusterRole 规则中是否出现：

```
# resources: ["secrets"]
# verbs: ["get","list","watch"]
```

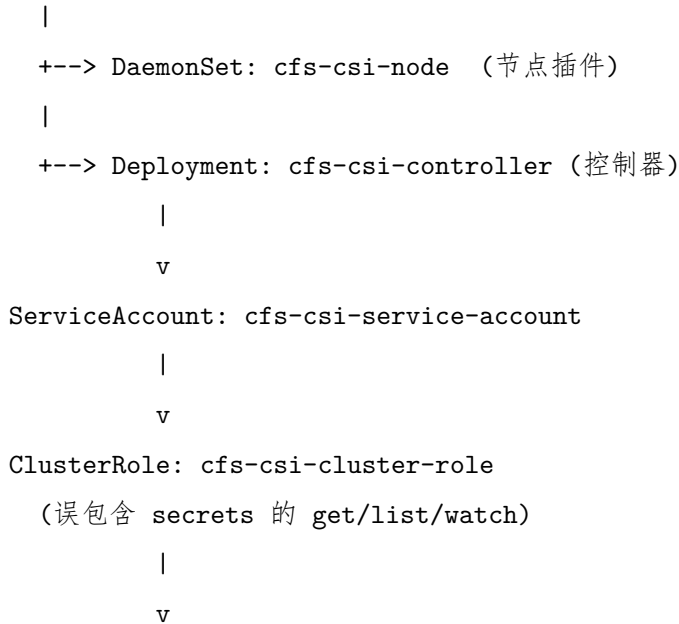
步骤 2：受控权限验证（不执行实际数据提取）

直接验证该 SA 是否具备跨命名空间读取 secrets 的能力

```
kubectl auth can-i get secrets
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
kubectl auth can-i list secrets
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
kubectl auth can-i watch secrets
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

权限提升路径（文字图，便于复盘）

CubeFS CSI 安装环境



一旦攻击者控制节点或 CSI Pod => 可滥用该 SA 的身份边界

（风险：读取高价值凭据 -> 演化为集群级权限提升）

D.2 策略代码清单

本节策略遵循“最小权限 + 防回归 + 可观测”主线：先移除 CSI 不必要的 `secrets` 读取权限；再用准入策略阻断未来再次引入过度授权；最后启用审计检测以便快速发现异常访问并触发止血流程。

D.2.1 1) RBAC 最小化：移除 ClusterRole 中的 secrets 权限

说明：CSI 需要的资源/动词与具体实现有关，建议在预生产以 PVC/PV 创建、挂载、扩容、attach/detach 等流程回归验证后再推广到生产。以下示例展示“覆盖式”更新 ClusterRole 的方式，明确不包含 `resources: ["secrets"]`。

```

cat <<'EOF' | kubectl apply -f -
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: cfs-csi-cluster-role
  labels:
    app.kubernetes.io/name: cfs-csi
  
```

```
rules:
- apiGroups: [""]
  resources: ["nodes","persistentvolumes","pods"]
  verbs: ["get","list","watch"]
- apiGroups: ["storage.k8s.io"]
  resources: ["volumeattachments","csinodes"]
  verbs: ["get","list","watch"]
# 注意：不应包含 resources:["secrets"]
EOF

# 复测（期望输出 no）
kubectl auth can-i list secrets
↪ --as=system:serviceaccount:$NAMESPACE:$SA --all-namespaces
```

D.2.2 2) ServiceAccount 硬化（按需评估）

若 CSI 驱动不需要直接调用 Kubernetes API，可评估禁用自动挂载令牌以降低“被攻陷即可滥用身份”的风险：

```
# 将 automountServiceAccountToken 设为 false（需评估 CSI 功能是否依赖
↪ token）
kubectl -n $NAMESPACE patch serviceaccount $SA --type='merge' -p
↪ '{"automountServiceAccountToken":false}'
```

D.2.3 3) 准入防回归（OPA Gatekeeper）：禁止创建包含 secrets 读取动词的 ClusterRole

说明：该策略用于防止未来误配置再次引入过度授权。建议先以 dryrun 运行观察影响，再逐步强制。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: templates.gatekeeper.sh/v1beta1
kind: ConstraintTemplate
metadata:
  name: k8sdenysecretsrbac
spec:
  crd:
    spec:
```

```

    names:
      kind: K8sDenySecretsRBAC
  targets:
  - target: admission.k8s.gatekeeper.sh
    rego: |
      package k8sdenysecretsrbac

      has_secrets_rule(rule) {
        rule.resources[_] == "secrets"
        rule.verbs[_] == "get"
      }

      has_secrets_rule(rule) {
        rule.resources[_] == "secrets"
        rule.verbs[_] == "list"
      }

      has_secrets_rule(rule) {
        rule.resources[_] == "secrets"
        rule.verbs[_] == "watch"
      }

      violation[{"msg": msg}] {
        input.review.kind.kind == "ClusterRole"
        some i
        rule := input.review.object.rules[i]
        has_secrets_rule(rule)
        msg := "ClusterRole must not grant get/list/watch on secrets"
      }
EOF

cat <<'EOF' | kubectl apply -f -
apiVersion: constraints.gatekeeper.sh/v1beta1
kind: K8sDenySecretsRBAC
metadata:
  name: deny-clusterrole-secrets-read
spec:
  enforcementAction: dryrun
  match:

```



```
kinds:
- apiGroups: ["rbac.authorization.k8s.io"]
  kinds: ["ClusterRole"]
EOF
```

D.2.4 4) 审计与检测：对 secrets 枚举行为进行可观测化

Kubernetes 审计策略片段（加载方式依平台而定）：

```
apiVersion: audit.k8s.io/v1
kind: Policy
rules:
- level: Metadata
  verbs: ["get","list","watch"]
  resources:
  - group: ""
    resources: ["secrets"]
  omitStages: ["RequestReceived"]
```

建议在 SIEM/日志平台中对来自 CSI ServiceAccount 的 secrets 枚举进行聚合告警，并结合阈值与白名单降低噪音。

D.2.5 5) 网络收敛（可选）：限制 CSI Pod 出站访问面

说明：NetworkPolicy 能降低被攻陷工作负载的横向移动能力，但在不同 CNI 下支持情况不同；同时 API Server 可能位于外部 LB，需按实际环境调整。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: cfs-csi-egress-lockdown
  namespace: $NAMESPACE
spec:
  podSelector:
    matchLabels:
      app: cfs-csi
  policyTypes: ["Egress"]
  egress:
```

```
- to:
  - namespaceSelector:
      matchLabels:
        kubernetes.io/metadata.name: kube-system
    podSelector:
      matchLabels:
        k8s-app: kube-dns
  ports:
    - protocol: UDP
      port: 53
EOF
```

附录 E CVE-2020-4062 复现步骤与策略清单

本附录给出案例四（CVE-2020-4062，Conjur OSS Helm Chart 旧版本缺失网络隔离导致 Postgres 在集群内可达）的复现要点与缓解策略清单。考虑到该问题涉及数据库直连带来的敏感数据风险，复现环节仅进行网络暴露面验证，用于确认 5432 端口在集群内的可达性以及网络策略的生效性，不展开数据库操作细节。

E.1 漏洞复现详细步骤

前提条件

- 集群 CNI 支持 NetworkPolicy，例如 Calico。
- Helm v3 可用。
- 受影响的 Conjur OSS Helm Chart 版本为 < 2.0.0。

步骤 1：在隔离命名空间部署旧版 Chart，并进行必要的兼容性适配旧版 Chart 可能使用过时 API，例如 apps/v1beta2。在较新 Kubernetes 版本上复现时，可在保持网络策略缺失这一特性不变的前提下，将 Deployment API 适配为 apps/v1 并补齐 selector 字段以完成安装。

步骤 2：确认 Postgres Service 与 5432 端口暴露

```
NAMESPACE=conjur-vuln
```

```
kubectl get ns $NAMESPACE
```

```
kubectl -n $NAMESPACE get svc
```

```
# 重点确认 Postgres Service（名称以实际为准）与 5432 端口存在
```

```
kubectl -n $NAMESPACE get svc | findstr /i "postgres 5432 conjur"
```

步骤 3：确认未配置 NetworkPolicy

```
kubectl -n $NAMESPACE get networkpolicy
```

若不存在任何约束 Postgres 入站的策略，且集群默认允许 Pod 间互通，则可判定数据库对集群内工作负载开放。

步骤 4：连通性验证在不执行数据库操作的前提下，使用 TCP 连通性探

测验证非 Conjur 组件工作负载是否能够与 Postgres 的 5432 端口建立连接，以此作为暴露面存在的验证依据。

E.2 策略代码清单

E.2.1 1) 缓解措施：NetworkPolicy，默认拒绝并仅放通 Conjur Server 访问 Postgres 5432

下述策略在命名空间内实现两点：其一，默认拒绝 Postgres Pod 的进站流量；其二，仅允许 Conjur Server 访问 Postgres 的 5432 端口。其中标签选择器需与实际部署对象一致，示例使用 `app=conjur-oss` 与 `app=conjur-oss-postgres`，可按实际标签调整。

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-default-deny
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
      app: conjur-oss-postgres
  policyTypes:
  - Ingress
  ingress: []
EOF
```

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: conjur-postgres-allow-conjur
  namespace: conjur-vuln
spec:
  podSelector:
    matchLabels:
```

```

    app: conjur-oss-postgres
policyTypes:
- Ingress
ingress:
- from:
  - podSelector:
    matchLabels:
      app: conjur-oss
ports:
- protocol: TCP
  port: 5432
EOF

```

策略验证

```
kubectl -n conjur-vuln get networkpolicy
```

```

# 复测：从非 Conjur 组件工作负载到 Postgres 5432 的连通性应失败
# 从 Conjur Server 到 Postgres 5432 的连通性应成功

```

E.2.2 2) 命名空间隔离与 RBAC 收敛

- 将 Conjur 部署在独立命名空间，并通过 NetworkPolicy 限制跨命名空间访问。
- 收敛对数据库凭据相关 Secret 的读取权限，降低网络可达与凭据可读叠加带来的风险。

附录 F CVE-2022-24877 复现步骤与策略清单

本附录给出案例五（CVE-2022-24877，Flux kustomize-controller 通过恶意 kustomization.yaml 触发路径穿越，导致控制器 Pod 文件系统敏感数据暴露，并在多租户部署中可能引发权限提升）的**验证性复现与缓解策略清单**。为避免提供可直接滥用的利用细节，复现部分以“路径越界行为证据与判据”为主，不提供可复用的攻击脚本。

F.1 漏洞复现详细步骤

复现目标验证控制器在构建过程中是否会因配置文件中的异常路径引用而产生目录边界越界读取风险。重点关注构建阶段的路径规范化与边界约束，并以日志行为与构建产物的异常特征作为证据判据。本案例涉及的控制器为 kustomize-controller，相关配置文件为 kustomization.yaml。

环境与前提

建议在隔离测试集群中进行，并使用受影响版本的 kustomize-controller。该类问题在多租户集群中可能放大隔离破坏风险，故不建议在生产环境进行实验。修复信息：该漏洞已在 kustomize-controller v0.24.0 修复。该版本随 flux v0.29.0 集成发布。

步骤要点（受控）

1. 在隔离仓库中准备最小化示例，包含 kustomization.yaml 及少量合法资源清单，用于建立可对比的基线构建产物。
2. 在受控条件下引入异常路径引用（例如包含父目录引用或绝对路径风格的条目），触发控制器重新拉取并执行构建。
3. 观察控制器日志与事件输出，记录与路径解析、文件读取失败或异常引用相关的报错/告警。
4. 对比构建产物与基线差异，检查是否出现“非预期文件内容进入产物”的迹象。该步骤应仅使用无敏感性的测试素材进行验证。

证据判据与逻辑解释该类漏洞的关键证据在于“路径解析是否受目录边界约束”。若控制器在构建阶段对异常路径引用给出明确的拒绝与报错，并且构建产物未出现越界读取痕迹，可视为风险被抑制。若在严格受控的测试条件下观察到异常路径引用导致非预期内容进入构建产物，则可判定目录边

界约束存在缺口，需立即升级组件并补齐流程治理措施。

F.2 策略代码清单（优先级方案）

本节选取对该类风险**最直接且工程可落地**的缓解措施：CI/CD 前置校验，并辅以多租户下的变更治理与运行时硬化。

F.2.1 1) 工作区缓解：在 CI/CD 中校验 kustomization.yaml 路径策略

在补丁窗口期或多团队协作场景下，可在 CI/CD 流水线中加入配置校验门禁。校验对象为 kustomization.yaml，要求其资源路径满足安全策略，例如：禁止绝对路径风格、禁止父目录引用、限制引用的文件类型与目录范围。下述示例为 conftest (OPA) 策略骨架，用于展示如何对 kustomization.yaml 的路径字段进行约束；可按组织规范扩展字段列表与白名单策略。

```
package kustomizepolicy

deny[msg] {
  some p
  p := input.resources[_]
  risky_path(p)
  msg := sprintf("kustomization.yaml: resources contains risky path:
  ↪ %s", [p])
}

deny[msg] {
  some i
  p := input.patches[i].path
  risky_path(p)
  msg := sprintf("kustomization.yaml: patches[%d].path contains risky
  ↪ path: %s", [i, p])
}

risky_path(p) {
  startswith(p, "/")
}
```



```
risky_path(p) {  
    contains(p, "..")  
}
```

说明：为降低误报，可进一步将 `contains(p, "..")` 改为按路径分段匹配“父目录段”，并对允许的相对路径前缀建立白名单。

F.2.2 2) 多租户变更治理与运行时硬化（纵深防御）

在多租户集群中，应严格控制谁可以向 Flux 同步源提交变更、谁可以修改 Flux 相关资源，并通过代码评审与分支保护降低恶意配置进入同步面的概率。同时可对控制器工作负载启用非特权运行、只读根文件系统、默认 `seccomp` 配置与能力集收敛，以降低异常路径下的后续风险。

附录 G CVE-2022-29171 复现步骤与策略清单

本附录给出案例六（CVE-2022-29171，Sourcegraph 在 Gitolite 与 Phabricator 联动集成中存在配置驱动的命令执行风险）的**验证性复现与缓解策略清单**。为避免提供可直接滥用的利用细节，复现部分以执行路径证据与回归判据为主，不给出可直接用于远程执行的命令载荷与请求模板。

G.1 漏洞复现详细步骤

复现目标验证在受影响版本中，特定集成配置项（`callsignCommand`）是否会被 `gitserver` 在处理仓库元数据获取流程时执行。复现以日志、审计与可观测证据为主，并在禁用集成后复测，确认风险被消除。

环境与前提仅在隔离测试环境开展。前置条件包括：（i）Sourcegraph 版本处于受影响范围；（ii）启用了 Gitolite code host 并与 Phabricator 元数据获取路径相关联；（iii）存在可编辑/新增 Gitolite code host 的管理权限；（iv）存在对内置 Grafana 的高权限访问，用于说明观测面可能降低内部寻址成本。

步骤要点（受控）

1. 部署受影响版本 Sourcegraph，并确认 `gitserver` 作为独立工作负载运行（记录其命名空间与标签信息）。
2. 在管理面将 Gitolite 与 Phabricator 集成配置为可用状态，保证仓库元数据获取流程可被触发。
3. 在严格受控条件下对 `callsignCommand` 进行**无害探针式**变更（例如仅返回固定标识、且不访问外部网络、不读取敏感文件）。
4. 触发一次与该集成相关的元数据获取/同步流程，并观察 `gitserver` 日志、任务记录或审计日志中是否出现外部命令执行的迹象。
5. 禁用 Gitolite code host 后重复触发流程，确认相同迹象不再出现。

证据判据与逻辑解释若在受影响版本中，`gitserver` 的运行日志或可观测数据中出现因集成配置而执行外部命令的明确迹象，可判定存在执行面风险。禁用相关集成后，该执行路径应被移除或被显式限制；若仍可观察到可疑执行迹象，则需进一步检查是否存在配置残留或旁路路径。

G.2 策略清单

本节仅选取 3 条对该类风险最直接且工程可落地的策略：功能禁用、网络隔离与权限分离/暴露面收敛。

G.2.1 1) 快速缓解：禁用或移除 Gitolite code host（若业务允许）

若业务不依赖 Gitolite，可移除所有 Gitolite code host 配置并保持禁用状态，从输入面直接消除 `callsignCommand` 的风险来源。

G.2.2 2) 网络隔离：限制 gitserver 的可达面（默认拒绝并按需放通）

在 Kubernetes 部署中，建议对 gitserver 应用 NetworkPolicy：仅允许 Sourcegraph 内部必要组件访问其入站端口；出站仅放通 DNS 与必要的代码托管出口。该策略用于降低内部拓扑信息被获取后的横向触达能力，并为权限误授提供纵深缓冲。

策略示例 示例分别约束入站与出站。入站仅允许 Sourcegraph 内部调用方访问 gitserver；出站仅放通 DNS，并对代码托管出口按需单独放通。选择器与端口应按实际部署对象调整。

NAMESPACE=sourcegraph

1) 入站收敛：仅允许 Sourcegraph 内部组件访问 gitserver

```
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: gitserver-ingress-allow-internal
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
  policyTypes: ["Ingress"]
  ingress:
```

```

- from:
  - podSelector:
      matchLabels:
        app: sourcegraph-frontend
  - podSelector:
      matchLabels:
        app: repo-updater
  ports:
    - protocol: TCP
      port: 3178
EOF

# 2) 出站收敛: 仅放通 DNS; 代码托管出口按需另行放通
cat <<'EOF' | kubectl apply -f -
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: gitserver-egress-dns-only
  namespace: sourcegraph
spec:
  podSelector:
    matchLabels:
      app: gitserver
  policyTypes: ["Egress"]
  egress:
    - to:
        - namespaceSelector:
            matchLabels:
              kubernetes.io/metadata.name: kube-system
        podSelector:
            matchLabels:
              k8s-app: kube-dns
  ports:
    - protocol: UDP
      port: 53
    - protocol: TCP
      port: 53

```

EOF

```
# 验证：策略已创建，且 gitserver 的出站/入站符合预期
kubectl -n $NAMESPACE get networkpolicy | findstr /i "gitserver"
```

验证要点建议先在预生产灰度启用，并结合 gitserver 的错误日志与调用链路指标观察误拦截情况。对确需访问的代码托管出口，按最小集合逐项放通，并保留变更记录。

G.2.3 3) 权限分离与暴露面收敛：限制管理面与观测面的高权限

严格限制可编辑外部服务配置（尤其是 Gitolite code host 配置）的主体数量，并启用变更审批与审计。同时收敛 Grafana 的暴露面，例如仅内网访问、受控 Ingress、强认证与最小管理员集合，并将 Grafana 管理员与 Sourcegraph 站点管理员职责分离，避免观测面权限放大攻击链的信息优势。

落地要点在平台流程上，可将外部服务配置变更与观测面账号授权纳入同一类高风险变更，要求双人复核、强制留痕与定期回收。在技术控制上，可对管理入口启用强认证并限制来源网络，同时在日志平台中对关键配置项变更进行集中审计与告警。

附录 H CVE-2023-22482 复现步骤与策略清单

本附录给出案例七（CVE-2023-22482，Argo CD 在 OIDC 场景下未校验令牌受众 `aud`，导致跨受众令牌可能被接受）的**验证性复现与缓解策略清单**。为避免提供可直接复用的越权访问细节，复现部分以配置证据、令牌声明证据与接口返回判据为主，不展开可直接用于获取令牌与构造请求的具体命令。

H.1 漏洞复现详细步骤

复现目标验证在受影响版本中，Argo CD 是否会接受 `aud` 不匹配但签名与发行者合法的 OIDC 令牌，并据此返回受 RBAC 控制的资源响应。

环境与前提仅在隔离测试环境开展。前置条件包括：（i）部署受影响版本的 Argo CD（例如 2.5.4），并启用 OIDC 登录；（ii）OIDC 提供方能够为至少两个不同受众签发令牌，且其中一个受众并非 Argo CD；（iii）Argo CD 已配置为信任该 OIDC 提供方，并启用 `groups` 相关声明用于 RBAC 映射；（iv）具备可观察的审计与日志渠道，用于对登录校验与资源访问结果进行证据采集。

步骤要点（受控）

1. 固化基线配置：记录 Argo CD 版本与 OIDC 配置要点（`issuer`、`clientID`、所请求的 `scope`），确认 Argo CD 能够正常使用 OIDC 完成会话建立。
2. 获取一枚由同一 OIDC 提供方签发、但 `aud` 指向其他受众的有效令牌。该令牌应满足验签可通过且包含可用于 RBAC 映射的 `groups` 声明。
3. 提取令牌声明证据：对该令牌解析其 `payload`，保留 `iss`、`aud`、`exp` 与 `groups` 等字段的记录，证明其受众并非 Argo CD。
4. 使用该令牌访问 Argo CD 的受保护接口，选择一个只读且便于判定的 API 作为验证点，记录响应结果与 Argo CD 侧日志。

证据判据与逻辑解释若在受影响版本中，Argo CD 在验签通过的前提下接受 `aud` 不匹配的令牌，并返回受 RBAC 控制的资源响应，可判定存在受众校验缺失。

H.2 策略清单

本节仅选取 3 条对该类风险最直接且工程可落地的策略：身份提供方客户端隔离、前置受众校验与 RBAC 收敛。

H.2.1 1) 身份提供方治理：独立客户端与唯一受众

为 Argo CD 配置独立 OIDC 客户端与唯一受众标识，避免与其他业务系统共享同一受众空间。同时对该客户端签发的 groups 声明进行裁剪，确保仅包含与 Argo CD 授权相关的组，并建立清晰的组命名规范，避免跨系统复用带来的权限含义漂移。

Argo CD OIDC 配置示例(唯一 clientID)说明:以下示例展示 argocd-cm 中的关键配置项，重点在于为 Argo CD 使用独立客户端标识，并显式声明基于 groups 的 RBAC 映射来源。

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-cm
  namespace: argocd
data:
  url: https://argocd.example.com
  oidc.config: |
    name: CorporateOIDC
    issuer: https://idp.example.com/realms/platform
    clientID: argocd-prod
    clientSecret: $oidc.corporate.clientSecret
    requestedScopes: ["openid", "profile", "email", "groups"]
    requestedIDTokenClaims:
      groups:
        essential: true
    logoutURL: https://idp.example.com/realms/platform/protocol/openid_
↵ -connect/logout
```

身份提供方客户端侧约束示意说明:不同身份提供方的配置格式不同,以下仅示意“为 Argo CD 单独分配客户端与受众、限制组声明来源”的关键字段。


```

clientId: argocd-prod
audience: argocd-prod
redirectUri:
  - https://argocd.example.com/auth/callback
protocol: openid-connect
defaultClientScopes:
  - openid
  - profile
  - email
  - groups
groupClaimFilter:
  prefix: argo:
  allow:
    - argo:platform-admin
    - argo:project-readonly
    - argo:project-deployer

```

H.2.2 2) 入口补偿控制：前置强制校验 aud

在 Argo CD 前置入口（如 API 网关或反向代理）对令牌进行一致性校验，至少包含发行者、签名与受众校验。落地时应提供灰度与回滚机制，并覆盖 Web 与自动化访问路径。

前置网关示例：Envoy JWT 认证过滤器说明：该示例展示如何在进入 Argo CD 前对 JWT 的 issuer 与 audiences 执行强制校验。若令牌的 aud 不包含 argocd-prod，请求将在到达 Argo CD 之前被拒绝。

```

static_resources:
  listeners:
    - name: argocd_listener
      address:
        socket_address:
          address: 0.0.0.0
          port_value: 8443
      filter_chains:
        - filters:
            - name: envoy.filters.network.http_connection_manager
              typed_config:

```

```

"@type": type.googleapis.com/envoy.extensions.filters.network.
↪ k.http_connection_manager.v3.HttpConnectionManager
stat_prefix: ingress_https
route_config:
  virtual_hosts:
    - name: argocd
      domains: ["argocd.example.com"]
      routes:
        - match: { prefix: "/" }
          route:
            cluster: argocd_server
http_filters:
- name: envoy.filters.http.jwt_authn
  typed_config:
    "@type": type.googleapis.com/envoy.extensions.filters.ht
    ↪ tp.jwt_authn.v3.JwtAuthentication
  providers:
    corporate_oidc:
      issuer: https://idp.example.com/realms/platform
      audiences:
        - argocd-prod
      remote_jwks:
        http_uri:
          uri: https://idp.example.com/realms/platform/pro
          ↪ tocol/openid-connect/certs
          cluster: idp_jwks
          timeout: 5s
      rules:
        - match: { prefix: "/api/" }
          requires:
            provider_name: corporate_oidc
- name: envoy.filters.http.router
clusters:
- name: argocd_server
  type: STRICT_DNS
  load_assignment:
    cluster_name: argocd_server

```

```

endpoints:
- lb_endpoints:
  - endpoint:
    address:
      socket_address:
        address: argocd-server.argocd.svc.cluster.local
        port_value: 80

```

H.2.3 3) 权限收敛：RBAC 最小化与项目边界约束

收敛 `argocd-rbac-cm` 中的角色授权，将高权限组限制在最小集合，并通过 `AppProject` 限定可部署的命名空间、资源类型与来源仓库。同时将关键接口的访问失败与异常登录事件纳入审计与告警，以便在令牌泄露或误用时快速发现与止血。

Argo CD RBAC 收敛示例

说明：以下示例将高权限组限制为 `argo:platform-admin`。普通项目用户仅保留只读与有限同步能力，从而降低跨受众令牌误用后的权限上限。

```

apiVersion: v1
kind: ConfigMap
metadata:
  name: argocd-rbac-cm
  namespace: argocd
data:
  policy.default: role:readonly
  policy.csv: |
    g, argo:platform-admin, role:admin
    g, argo:project-readonly, role:readonly
    g, argo:project-deployer, role:deployer

    p, role:admin, applications, *, /*, allow
    p, role:admin, projects, *, *, allow
    p, role:admin, repositories, *, *, allow
    p, role:admin, clusters, *, *, allow

    p, role:readonly, applications, get, /*, allow
    p, role:readonly, projects, get, *, allow

```

```
p, role:readonly, repositories, get, *, allow

p, role:deployer, applications, get, my-project/*, allow
p, role:deployer, applications, sync, my-project/*, allow
p, role:deployer, applications, action/*, my-project/*, deny
```

AppProject 边界约束示例说明：通过限制目标命名空间、来源仓库与可管理资源范围，进一步压缩误用令牌在项目维度上的横向影响面。

```
apiVersion: argoproj.io/v1alpha1
kind: AppProject
metadata:
  name: my-project
  namespace: argocd
spec:
  description: bounded project for tenant applications
  sourceRepos:
    - https://github.com/example-org/app-configs.git
  destinations:
    - namespace: tenant-a
      server: https://kubernetes.default.svc
  clusterResourceWhitelist:
    - group: ""
      kind: Namespace
  namespaceResourceWhitelist:
    - group: apps
      kind: Deployment
    - group: ""
      kind: Service
  roles:
    - name: readonly
      description: read-only access for tenant reviewers
      policies:
        - p, proj:my-project:readonly, applications, get, my-project/*,
          ↪ allow
      groups:
        - argo:project-readonly
    - name: deployer
```

```
description: deployment access for approved operators
policies:
- p, proj:my-project:deployer, applications, get, my-project/*,
  ↪ allow
- p, proj:my-project:deployer, applications, sync, my-project/*,
  ↪ allow
groups:
- argo:project-deployer
```


致 谢

时光荏苒，岁月如梭。在导师的悉心指导和亲友的默默支持下，本论文终于得以顺利完成。值此定稿之际，我谨向所有在我的硕士学习和生活期间给予我帮助与支持的人，致以最诚挚的谢意。

首先，我要向我的导师孙昌爱教授表达最衷心的感谢和崇高的敬意。在我的整个硕士学习阶段，从论文的选题、开题、研究工作的开展到最终的定稿，孙老师都倾注了大量的心血，给予了我悉心的指导。孙老师严谨的治学态度、渊博的专业知识以及对科研的热忱，使我受益匪浅，也将成为我今后工作和学习中的宝贵财富。

其次，感谢北京科技大学为我提供了良好的学习和科研环境。感谢在此期间教导过我的所有老师，感谢你们的传道授业解惑。我要特别感谢实验室的同门们以及我的室友。我们不仅在科研上互相切磋、共同进步，在日常生活中也彼此照应，让我的研究生时光充满了欢乐与温暖。

此外，我还要特别感谢在北京的各位高中同学和朋友们。在紧张的科研之余，是你们的陪伴与鼓励，给了我莫大的放松与慰藉，让我在求学之路上不觉孤单。

最后，我要深深感谢我的家人。正是你们多年如一日的默默付出、理解与无私的爱，为我撑起了一片充满温暖的天空，让我能够全心全意地投入到学业之中。你们永远是我坚实的后盾和不断前行的最大动力。

感谢开源社区的所有贡献者们，你们的无私奉献和创新精神为我的研究提供了宝贵的资源和灵感。

感谢所有关心和帮助过我的人！

作者简历及在学研究成果

一、主要教育经历/工作经历（从大学起，到硕士入学止）

起止年月	学习或工作单位	备注
2019 年 9 月至 2023 年 6 月	在北京科技大学计算机科学与技术专业攻读学士学位	

二、在学期间从事的科研工作

三、在学期间所获的科研奖励

四、在学期间发表的论文

- [1] Sun C, **Han X**, Gong Y. KubeGuard: A Systematic Permission-Oriented Risk Detection Approach for Kubernetes Applications[C]//2025 IEEE International Conference on Web Services (ICWS). IEEE, 2025: 855-865.

- [2] 第二作者（导师一作）.ICWS.CCF-B.2025.

盲审说明（正式写作请删除此说明）：

盲审论文需要隐藏掉所有会影响盲审结果的论文作者及其导师的信息，以便论文评阅人能够公正的进行评阅。

当学校要求提供盲审论文时，请按如下方法制作。

1) 对于论文中下列学生和导师信息。请将学生姓名、学生学号、导师姓名，依次全部替换为[本论文作者]、[论文作者学号]、[本论文导师]。论文中上述信息均需要替换，包括作者研究成果等部分的有关信息。

2) 在研究成果中，论文作者发表的文章列表中应隐去所有作者的名字，只标明论文期刊名，级别，发表年份。

3) 盲审论文，请不要填写致谢，致谢页除标题、页眉、页码外请保持空

白。

4) 其他会影响盲审结果的信息, 请采用类似方式处理。

5) 提交的盲审论文应为正式论文, 除了上述替换后的信息外, 应为可以评阅的正式论文。

6) 论文封面, 请填写专业名称、论文题目等, 将学号和姓名项目保持空白不填。制作盲审论文时, 论文书脊、封二、题名页请删除。

替换前后将文档分别保存, 以便盲审论文与其他论文分开管理。

盲审样例 (正式写作请删除此样例):

五、在学期间发表的论文:

[1] **第一作者**. 中国矿业. 中文核心期刊.2020.

[2] **第二作者 (导师一作)**. 中国矿业. 中文核心期刊.2020.

独创性声明

本人声明所呈交的论文是我个人在导师指导下进行的研究工作及取得的
研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，论文中不包
含其他人已经发表或撰写过的研究成果，也不包含为获得北京科技大学或其
它教育机构的学位或证书而使用过的材料。与我一同工作的同志对本研究所
做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：_____ 日期：_____ 年_____ 月_____ 日

备注：提交时须有作者亲笔签名！

关于论文使用授权的说明

本人完全了解北京科技大学有关保留、使用学位论文的规定，即：学校有权保留送交论文的复印件，允许论文被查阅和借阅；学校可以公布论文的全部或部分内容，可以采用影印、缩印或其他复制手段保存论文。

（保密的论文在解密后应遵循此规定）

签名：_____ 导师签名：_____ 日期：_____

学位论文数据集

关键词 *	密级 *	中图分类号 *	UDC	论文资助
学位授予单位名称 *		学位授予单位代 码 *	学位类别 *	学位级别 *
北京科技大学		10008		
论文题名 *		并列题名		论文语种 *
作者姓名 *			学号 *	
培养单位名称 *		培养单位代码 *	培养单位地址	邮编
北京科技大学		10008	北京市海淀区学 院路 30 号	100083
学科专业 *		研究方向 *	学制 *	学位授予年 *
论文提交日期 *				
导师姓名 *			职称 *	
评阅人	答辩委员会主席	答辩委员会成员		
	*			
电子版论文提交格式文本 () 图像 () 视频 () 音频 () 多媒体 () 其他 () 推荐格式: application/msword; application/pdf				
电子论文出版 (发布) 者		电子论文出版 (发布) 地		权限声明
论文总页数 *				
共 33 项, 其中带 * 为必填数据, 为 22 项。				